《算法设计》习题与答案

第 1、2、3、4、5、6、8 章

Jon Kleinberg & Éva Tardos, *Algorithm Design*

题目：中文翻译　答案：简要解答

《算法设计》习题与答案（第1、2、3、4、5、6、8章）

> 原书：Jon Kleinberg & Éva Tardos, *Algorithm Design* 题目：中文翻译；答案：据官方解答手册整理的简要解答。

目录

- 第1章 引言：若干典型问题
- 第2章 算法分析基础
- 第3章 图
- 第4章 贪心算法
- 第5章 分治
- 第6章 动态规划
- 第8章 NP 与计算难解性

第1章 引言：若干典型问题

习题 1

题目： 判断下列说法是真是假。若为真，给出简短说明；若为假，给出反例。

真或假？在稳定匹配问题（Stable Matching Problem）的每个实例中，都存在一个稳定匹配，其中含有一对 (m, w)，使得 m 在 w 的偏好表上排第一，且 w 在 m 的偏好表上排第一。

答案： 假。反例：两名男子 m、m' 与两名女子 w、w'，偏好为 m: $w \succ w'$，m': $w' \succ w$，w: $m' \succ m$，w': $m \succ m'$。此时没有任何一对男女互列第一，故任何稳定匹配都不可能含有这样的对。

习题 2

题目： 判断下列说法是真是假。若为真，给出简短说明；若为假，给出反例。

真或假？考虑稳定匹配问题的一个实例，其中存在男子 m 和女子 w，使得 m 在 w 的偏好表上排第一，且 w 在 m 的偏好表上排第一。那么对该实例的每个稳定匹配 S，都有 $(m, w) \in S$。

答案： 真。若某稳定匹配 S 不含 (m, w)，则它含 (m, w') 与 (m', w)。由于 m 与 w 互列第一，二人都更偏好对方而非 S 中的伴侣，于是 (m, w) 构成不稳定对，与 S 稳定矛盾。

习题 3

题目： 还有许多其他场景可以提出与某种"稳定性"原则相关的问题。下面是其中之一，涉及两个企业之间的竞争。

设有两个电视网，我们称之为 $\mathcal{A}$ 和 $\mathcal{B}$。共有 n 个黄金时段节目档，每个电视网各有 n 个电视节目。每个电视网都希望制定一份节目表（schedule）——把每个节目分配给一个不同的节目档——以尽可能吸引更大的市场份额。

在给定双方节目表的情况下，我们按如下方式判定两个电视网的相对表现。每个节目有一个固定的收视率（rating），它基于去年观看该节目的人数；我们假定没有两个节目的收视率完全相同。若某电视网在某个节目档安排的节目收视率高于另一电视网在该档安排的节目，则该电视网赢得该节目档。每个电视网的目标是赢得尽可能多的节目档。

假设在秋季首播周，电视网 $\mathcal{A}$ 公布了节目表 S，电视网 $\mathcal{B}$ 公布了节目表 T。基于这对节目表，每个电视网按上述规则赢得若干节目档。若两个电视网都不能单方面改变自己的节目表而赢得更多节目档，则称节目表对 (S, T) 是稳定的（stable）。即不存在节目表 S' 使得 $\mathcal{A}$ 在 (S', T) 下赢得的档数多于在 (S, T) 下；对称地，也不存在节目表 T' 使得 $\mathcal{B}$ 在 (S, T') 下赢得的档数多于在 (S, T) 下。

对这种稳定性，Gale–Shapley 问题的类比是：对任意一组电视节目及其收视率，是否总存在稳定的节目表对？通过完成以下两件事之一来回答：

(a) 给出一个算法，对任意一组电视节目及相关收视率，产生一对稳定的节目表；或

(b) 给出一组电视节目及相关收视率的例子，使其不存在稳定的节目表对。

答案： 不存在稳定对，故 (b) 成立。反例：$\mathcal{A}$ 有两个节目 a_1（收视率 20）、a_2（40），$\mathcal{B}$ 有 d_1（10）、d_2（30），只有两个节目档。若 a_1 对 d_1、a_2 对 d_2，则 $\mathcal{B}$ 交换两节目后由赢 0 档变为赢 1 档；若 a_1 对 d_2、a_2 对 d_1，则 $\mathcal{A}$ 交换两节目后由赢 1 档变为赢 2 档；其余两种节目表组合同理。故任何节目表对都有一方可以单方面改进。

习题 4

题目： Gale 和 Shapley 于 1962 年发表了关于稳定匹配问题的论文；但该算法的一个版本已被美国国家住院医师匹配计划（National Resident Matching Program）用于医学生分配医院，长达十年之久。

基本情况如下。有 m 所医院，每所医院有若干可用的住院医师职位。有 n 名医学生将在某年毕业，每人都有兴趣加入其中一所医院。每所医院对申请学生有一个偏好排序，每个学生对医院也有一个偏好排序。我们假定毕业的学生人数多于 m 所医院可提供的职位总数。

自然，目标是找到一种把每个学生分配到至多一所医院的方案，使得所有医院的可用职位都被填满。（由于假定学生过剩，会有一些学生未被分配到任何医院。）

若下列情形都不出现，则称学生到医院的分配是稳定的：

- 第一类不稳定：存在学生 s、学生 s' 和医院 h，使得
- s 被分配到 h，且
- s' 未被分配到任何医院，且
- h 相比 s 更偏好 s'。
- 第二类不稳定：存在学生 s、学生 s' 和医院 h、h'，使得
- s 被分配到 h，
- s' 被分配到 h'，且
- h 相比 s 更偏好 s'，且
- s' 相比 h' 更偏好 h。

所以我们基本上面对的就是稳定匹配问题，只不过 (i) 医院通常想要不止一名住院医师，(ii) 医学生存在过剩。

证明总存在稳定的学生–医院分配，并给出一个求它的算法。

答案： 把 Gale–Shapley 算法推广为"多职位"版本。任一时刻学生处于"已承诺"或"自由"状态，医院处于"有空位"或"满员"状态。当某医院 h_i 还有空位时，向偏好表上尚未拒绝过它的下一位学生 s_j 发出邀请：若 s_j 自由则接受；若 s_j 已承诺于 h_k，则比较二者，更偏好 h_k 就留下，否则改投 h_i（h_k 空位加 1，h_i 空位减 1）。每所医院对每名学生至多邀请一次，故算法在 $O(mn)$ 步内终止；因学生人数多于职位总数，所有职位必被填满；所得分配稳定，因为两类不稳定都会推出"该学生曾拒绝该医院"或"该医院曾邀请该学生而被拒"的矛盾。

习题 5

题目： 如正文所述，稳定匹配问题假定所有男女都有完全严格的偏好序。本题考虑一个变体，其中男女可以在某些选项之间无差异（indifferent）。和之前一样，设有 n 名男子的集合 M 和 n 名女子的集合 W。假定每个男子和每个女子都对异性成员排序，但现在允许排序中出现并列。例如（$n = 4$），某女子可以说 m_1 排在第一位；第二位是 m_2 与 m_3 并列（她对二者无偏好）；m_4 排在最后。若 m 在她的偏好表上排在 m' 之前（二者不并列），我们说 w 相比 m' 更喜欢 m。

在存在并列的情况下，稳定性有两种自然的定义。对每一种，我们都可以问稳定匹配是否存在，如下。

(a) 完美匹配 S 中的强不稳定（strong instability）由一名男子 m 和一名女子 w 组成，使得 m 和 w 各自都更喜欢对方而不是 S 中自己的伴侣。是否总存在无强不稳定的完美匹配？或者给出一组男女及其偏好表的例子，使每个完美匹配都有强不稳定；或者给出一个保证能找到无强不稳定的完美匹配的算法。

(b) 完美匹配 S 中的弱不稳定（weak instability）由一名男子 m 和一名女子 w 组成，使得他们在 S 中的伴侣分别为 w' 和 m'，且下列之一成立： - m 相比 w' 更喜欢 w，且 w 相比 m 更喜欢 m'，或对二者无差异；或 - w 相比 m' 更喜欢 m，且 m 相比 w 更喜欢 w'，或对二者无差异。

换言之，m 与 w 的配对要么被双方偏好，要么被一方偏好而另一方无差异。是否总存在无弱不稳定的完美匹配？或者给出一组男女及其偏好表的例子，使每个完美匹配都有弱不稳定；或者给出一个保证能找到无弱不稳定的完美匹配的算法。

答案： (a) 是。任取一种打破并列的方式（如按编号字典序）得到严格偏好表，运行 Gale–Shapley 算法；若所得匹配有强不稳定，则这对男女在打破并列的表中仍互相严格更偏好，从而构成该表下的不稳定对，与算法输出的稳定性矛盾。

(b) 否。反例：$n = 2$，m_1 对 w_1、w_2 无差异，w_1、w_2 都更偏好 m_1 而非 m_2（m_2 的偏好任意）。任何完美匹配中总有一名女子与 m_2 配对，她与 m_1 构成弱不稳定：她严格更偏好 m_1，而 m_1 对她与现任伴侣无差异。

习题 6

题目： 周游航运公司（Peripatetic Shipping Lines, Inc.）拥有 n 艘船，为 n 个港口提供服务。每艘船有一份日程表（schedule），说明当月每一天它正在访问哪个港口，或者是否在海上。（可以假定"月"有 m 天，其中 $m > n$。）每艘船在当月恰好访问每个港口一天。出于安全考虑，PSL 公司有如下严格要求：

(†) 任何两艘船不能在同一天处于同一港口。

公司希望在本月对所有船只进行维护，方案如下。他们想截断（truncate）每艘船的日程：对每艘船 S_i，总会有某一天它抵达日程中的某个港口，此后就一直留在那里（进行维护）。这意味着 S_i 当月不再访问其日程中剩余的港口（如果有的话），但这没问题。于是 S_i 日程的截断就是它原日程到某个指定日期为止的部分，该日它位于港口 P；截断后的其余部分就是一直留在港口 P。

现在公司向你提出的问题是：给定每艘船的日程，为每艘船找一个截断，使条件 (†) 仍然成立：任何两艘船不会在同一天处于同一港口。

证明这样一组截断总是存在，并给出求它们的算法。

（题目附例：两艘船、两个港口，"月"有 4 天。第一艘船日程为 P_1；海上；P_2；海上。第二艘船日程为 海上；P_1；海上；P_2。则唯一的截断选择是：让第一艘船从第 3 天起留在 P_2，让第二艘船从第 2 天起留在 P_1。）

答案： 把船与港口构成稳定匹配：每艘船按到访港口的先后次序给港口排序，每个港口按到访它的船的逆时序给船排序；求稳定匹配，把每艘船匹配到的港口作为其"停靠港"，其后原地停留即得截断方案。若方案不满足 (†)，即有船 S_i 在 S_j 已停靠之后仍经过港口 P_k，则 S_i 更偏好 P_k 而非自己的停靠港，而 P_k 更偏好 S_i 而非 S_j（S_i 到得更晚），与匹配的稳定性矛盾。稳定匹配必存在，用 Gale–Shapley 算法求出。

习题 7

题目： 你的一些朋友在 CluNet 工作，该公司建设大型通信网络，他们正在研究某类输入/输出交叉开关（crossbar）的切换算法。

情形如下。有 n 条输入线和 n 条输出线，每条线都从源端（source）指向终端（terminus）。每条输入线与每条输出线恰好相交于一个独特的点，这个点位于一块称为交点箱（junction box）的特殊器件中。线上的点沿从源到终端的方向自然排序：对同一条线上两个不同的点 x 和 y，若 x 比 y 更靠近源，则称 x 在 y 的上游（upstream），否则称 x 在 y 的下游（downstream）。一条输入线与各输出线相交的次序，不必与另一条输入线与这些输出线相交的次序相同。（各输出线与输入线相交的次序亦然。）图 1.8 给出了这样一组输入线和输出线的例子。

现在的切换问题是：每条输入线承载一路不同的数据流，这路数据流必须被切换（switch）到某条输出线上。若输入 i 的数据流在交点箱 B 处切换到输出 j，则该数据流经过输入 i 上 B 上游的所有交点箱，再经过 B，然后经过输出 j 上 B 下游的所有交点箱。哪路输入数据流切换到哪条输出线无所谓，但每路输入数据流必须切换到一条不同的输出线。此外——这是棘手的约束——切换之后，不能有两路数据流经过同一个交点箱。

最后是问题：证明对输入线与输出线相交的任意给定模式（每对恰好相交一次），总能找到一种有效的数据流切换——每路输入数据流切换到不同的输出，且切换后没有两路数据流经过同一个交点箱。此外，给出一个求这种有效切换的算法。

答案： 把输入线与输出线构成稳定匹配：每条输入线按从源到终端与各输出线相遇的次序给输出线排序，每条输出线按相遇次序的逆序给输入线排序；匹配到的输出即为该输入数据流切换的目标。若切换后两路数据流在输入 i 与输出 j 的交点箱相撞，则另一路来自某输入 k 并切换到输出 j；因 i 在 k 的下游与 j 相交，输出 j 更偏好 i 而非 k；因 j 在 i 实际切换到的输出 ℓ 的上游，输入 i 更偏好 j 而非 ℓ，构成阻碍对，矛盾。偏好表可在 $O(n^2)$ 时间内构造，再用 Gale–Shapley 算法在 $O(n^2)$ 时间内求出匹配。

习题 8

题目： 本题探讨稳定匹配问题、特别是 Gale–Shapley 算法中的诚实性（truthfulness）问题。基本问题是：男人或女人能否通过谎报自己的偏好而得到更好的结果？更具体地，假定每个参与者都有一个真实偏好序。考虑某女子 w。设 w 更偏好男子 m 而非 m'，但 m 和 m' 在她的偏好表上都排得很低。是否可能：她交换表中 m 与 m' 的顺序（即谎称自己更偏好 m' 而非 m），并用这份虚假偏好表运行算法后，w 最终得到的男子 m'' 是她在真实偏好下比 m 和 m' 都更喜欢的？（对男子也可以问同样的问题，但本题只关注女子的情形。）

通过完成以下两件事之一来回答：

(a) 证明：对任意一组偏好表，交换表中一对元素的顺序都不能改善某女子在 Gale–Shapley 算法中的伴侣；或

(b) 给出一组偏好表的例子，其中某女子作这样的交换后伴侣得到改善。

答案： (a) 不成立，故无此证明。(b) 反例（3 男 3 女）：真实偏好为 m_1: $w_3 \succ w_1 \succ w_2$，m_2: $w_1 \succ w_3 \succ w_2$，m_3: $w_3 \succ w_1 \succ w_2$；w_1: $m_1 \succ m_2 \succ m_3$，w_2: $m_1 \succ m_2 \succ m_3$，w_3: $m_2 \succ m_1 \succ m_3$。按真实偏好运行 G–S 得 (m_1, w_3)、(m_2, w_1)、(m_3, w_2)，w_3 只得到第二志愿 m_1。若 w_3 谎称 $m_2 \succ m_3 \succ m_1$，则运行 G–S 时她先接受 m_1，再接受 m_3，最后接受 m_2，最终与 m_2 配对——恰是她的真实第一志愿。故女子可以通过谎报偏好改善伴侣。

第2章 算法分析基础

习题 1

题目： 假设你有一些算法，其运行时间如下所列（假定这些就是精确的运行时间）。当 (a) 输入规模加倍，或 (b) 输入规模增加 1 时，这些算法的运行时间分别会变慢多少？运行时间为：(a) n^2；(b) n^3；(c) $100n^2$；(d) $n \log n$；(e) 2^n。

答案： 规模加倍时：(a) 变为 4 倍；(b) 变为 8 倍；(c) 变为 4 倍；(d) $T(2n) = 2T(n) + 2n$，即约 2 倍再加 $2n$；(e) 变为原运行时间的平方（$T(2n) = T(n)^2$）。规模加 1 时：(a) 增加 $2n + 1$；(b) 增加 $3n^2 + 3n + 1$；(c) 增加 $200n + 100$；(d) 增加 $\log(n+1) + n \cdot \log((n+1)/n)$；(e) 变为 2 倍。

习题 2

题目： 假设有六个算法，其运行时间如下所列（假定所列即为作为输入规模 n 的函数的精确操作次数）。你有一台每秒可执行 10^{10} 次操作的计算机，并要求最多在一小时的计算时间内得到结果。对每个算法，你能在一小时内得到结果的最大输入规模 n 是多少？(a) n^2；(b) n^3；(c) $100n^2$；(d) $n \log n$；(e) 2^n；(f) $2^{(2^n)}$。

答案： 一小时可做约 3.6×10^{13} 次操作。(a) 6,000,000；(b) 33,015；(c) 600,000；(d) 约 9×10^{11}；(e) 45；(f) 5。

习题 3

题目： 将下列函数按增长率递增的次序排列，即若 g(n) 紧接在 f(n) 之后，则应有 f(n) = O(g(n))：$f_1(n) = n^{2.5}$，$f_2(n) = \sqrt{2n}$，$f_3(n) = n + 10$，$f_4(n) = 10^n$，$f_5(n) = 100^n$，$f_6(n) = n^2 \log n$。

答案： $f_2 \prec f_3 \prec f_6 \prec f_1 \prec f_4 \prec f_5$。多项式（含非整数次幂）比指数增长慢；多项式的次序看最高次幂，$\sqrt{2n}$ 低于 n，$n^2 \log n$ 低于 $n^{2.5}$；指数按底数排序，10^n 先于 100^n。

习题 4

题目： 将下列函数按增长率递增的次序排列，即若 g(n) 紧接在 f(n) 之后，则应有 f(n) = O(g(n))：$g_1(n) = 2^{(\sqrt{\log n})}$，$g_2(n) = 2^n$，$g_3(n) = n(\log n)^3$，$g_4(n) = n^{(4/3)}$，$g_5(n) = n^{(\log n)}$，$g_6(n) = 2^{(2^n)}$，$g_7(n) = 2^{(n^2)}$。

答案： 正确次序：$g_1 \prec g_3 \prec g_4 \prec g_5 \prec g_2 \prec g_7 \prec g_6$。要点（均取对数比较）： - $g_1 = 2^{(\sqrt{\log n})}$：指数 $\sqrt{\log n}$ 最小，故最慢； - $g_3 = n(\log n)^3$ 与 $g_4 = n^{(4/3)}$：同除 n 后比较 $(\log n)^3$ 与 $n^{(1/3)}$，前者更慢； - $g_5 = n^{(\log n)}$：取对数得 $(\log n)^2$，增长快于 $n^{(1/3)}$、慢于 2^n； - 之后是指数：$g_2 = 2^n \prec g_7 = 2^{(n^2)} \prec g_6 = 2^{(2^n)}$（比较对数 n、n^2、2^n）。

注：原解答页把 g_5 误当作 $n \log n$，排成了 $g_1 \prec g_5 \prec g_3 \prec g_4 \prec g_2 \prec g_7 \prec g_6$；按题面的 $n^{(\log n)}$，g_5 应在 g_4 与 g_2 之间。

习题 5

题目： 设有函数 f 和 g，满足 f(n) = O(g(n))。对下列各命题，判断其真假并给出证明或反例：(a) $\log_2 f(n) = O(\log_2 g(n))$；(b) $2^{f(n)} = O(2^{g(n)})$；(c) $f(n)^2 = O(g(n)^2)$。

答案： (a) 一般情形为假：取 g(n) = 1、f(n) = 2，则 $\log_2 g(n) = 0$，不可能有 $\log_2 f(n) \le c \cdot \log_2 g(n)$；但若要求 $g(n) \ge 2$（n 充分大），命题成立，因为 $\log_2 f(n) \le \log_2 g(n) + \log_2 c \le (\log_2 c)(\log_2 g(n))$。(b) 假：取 f(n) = 2n、g(n) = n，则 $2^{f(n)} = 4^n$，不是 $O(2^n)$。(c) 真：由 $f(n) \le c \cdot g(n)$ 两边平方得 $f(n)^2 \le c^2 g(n)^2$。

习题 6

题目： 考虑如下基本问题。给定含 n 个整数的数组 A[1], A[2], …, A[n]，要输出一个 n×n 的二维数组 B，其中 i < j 时 B[i, j] 为数组元素 A[i] 到 A[j] 之和，即 $A[i] + A[i+1] + \cdots + A[j]$（i ≥ j 时 B[i, j] 的取值不作规定，输出什么都无所谓）。下面是一个简单的求解算法：

For i = 1, 2, …, n　For j = i+1, i+2, …, n　　将 A[i] 到 A[j] 的数组元素相加　　把结果存入 B[i, j]　Endfor Endfor

(a) 选取某个你认为合适的函数 f，给出该算法在规模 n 的输入上运行时间（即算法执行的操作次数）的 O(f(n)) 上界。 (b) 对同一个 f，证明该算法在规模 n 的输入上的运行时间也是 Ω(f(n))（这给出运行时间 Θ(f(n)) 的渐近紧界）。 (c) 尽管 (a)(b) 中分析的算法是解决问题最自然的做法——它毕竟只是遍历数组 B 的相关项并逐一填值——但它含有若干极不必要的低效之处。请给出另一个算法，其运行时间渐近更优，即运行时间为 O(g(n)) 且 $\lim_{n\to\infty} g(n)/f(n) = 0$。

答案： (a) 取 $f(n) = n^3$：外层 n 次、内层至多 n 次，每次求和 $O(j - i + 1) = O(n)$，另有常数时间的存储，故运行时间为 $O(n^3)$。(b) 当 $i \le n/4$ 且 $j \ge 3n/4$ 时 $j - i + 1 > n/2$，此类 (i, j) 至少有 $(n/4)^2$ 对，每对至少需 n/2 次加法，故总操作数至少 $(n/4)^2 \cdot (n/2) = n^3/32$，即 $\Omega(n^3)$。(c) 用前缀和递推：先对每个 i 置 B[i, i+1] = A[i] + A[i+1]；再对 k = 2, 3, …, n−1 置 B[i, i+k] = B[i, i+k−1] + A[i+k]。每个 k 只需 O(n) 时间，共 $O(n^2)$，故 $g(n) = n^2$。

习题 7

题目： 有一类民谣和节日歌曲，其中每一段都由上一段再加上一行构成。《圣诞十二日》就有此性质：唱到第五段时，要在第四段各句之后再加上"五个金戒指"，以及四只鸣鸟、三只法国母鸡、两只斑鸠和梨树上的那只鹧鸪。逾越节家宴《哈德·加迪亚》（Had Gadya）等许多歌也如此。这类歌演唱时间很长，但脚本相对很短：你只需写明每一段新增的那一行，而不必把此前各段逐行写出（例如"五个金戒指"只写一次，尽管它会在第五段及以后各段出现）。这里有一个可作渐近分析的问题：设每行长度不超过常数 c，且整首歌演唱时共有 n 个词。请说明如何为这样一首歌设计一个长度为 f(n) 的脚本，使 f(n) 增长尽可能慢。

答案： 设互不相同的行共 L 行（每个词至少属于一行，故 $1 + 2 + \cdots + L \ge n$，即 $L \le 1 + \sqrt{(2n)}$）。脚本只需逐行写出这 L 行的文本（每行长度至多 c_2，即最长行 c 加上赋值语句所需的空间），再用常数长度的双层循环"对 i = 1..L，唱第 1 至 i 行"指明各段如何拼接；循环本身只占常数空间，故 $f(n) = O(\sqrt{n})$。

习题 8

题目： 你在对多种型号的玻璃罐做压力测试，以确定它们从多高落下而不碎。对某一型号的罐子，实验装置如下：有一架 n 级的梯子，你要找出从哪一级扔下罐子仍不碎的最高一级，称之为最高安全梯级。自然的想法是二分查找：从中间一级扔下，看是否打碎，再依结果在 n/4 或 3n/4 处递归尝试；但缺点是找答案的过程中可能打碎很多罐子。若首要目标是节省罐子，则可以采用另一策略：从第 1 级开始，逐级上升（第 1 级、第 2 级……），直到罐子打碎；这样只需一个罐子，但可能要扔 n 次（而非二分法的 log n 次）。于是存在一种权衡：若愿意打碎更多罐子，似乎就能减少抛掷次数。为定量理解这一权衡，考虑在固定"预算"$k \geq 1$ 个罐子的条件下进行实验：即必须确定正确答案（最高安全梯级），且整个过程至多打碎 k 个罐子。 (a) 设预算为 $k = 2$ 个罐子。请描述一种策略，使扔罐子的次数至多为 $f(n)$，其中 $f(n)$ 增长慢于线性（即 $\lim_{n\to\infty} f(n)/n = 0$）。 (b) 现设预算为 $k > 2$ 个罐子（k 给定）。请描述一种至多使用 k 个罐子的策略。若 $f_k(n)$ 表示你的策略所需的抛掷次数，则函数 $f_1, f_2, f_3, \ldots$ 应具有如下性质：每一个都比前一个渐近增长更慢，即对每个 k 有 $\lim_{n\to\infty} f_k(n)/f_{k-1}(n) = 0$。

答案： (a) 为简单起见设 n 是完全平方数。用第一个罐子从 $\sqrt{n}$ 的整数倍高度（$\sqrt{n}, 2\sqrt{n}, 3\sqrt{n}, \ldots$）依次扔下直到打碎；若到顶层仍未碎则已得答案。设它在 $j\sqrt{n}$ 处打碎，则最高安全梯级位于 $(j-1)\sqrt{n}$ 与 $j\sqrt{n}$ 之间，再用第二个罐子从 $(j-1)\sqrt{n}+1$ 级起逐级上升。两个罐子各至多扔 $\sqrt{n}$ 次，共 $O(\sqrt{n})$ 次（n 非完全平方时第一个罐子至多 $2\sqrt{n}$ 次，界不变）。故 $f_2(n) = O(\sqrt{n})$。(b) 归纳证明 $f_k(n) \leq 2k \cdot n^{1/k}$：第一个罐子每隔 $n^{(k-1)/k}$ 级扔一次，至多 $2n^{1/k}$ 次即可把候选范围缩小为长度不超过 $n^{(k-1)/k}$ 的区间；再在该区间上递归使用 $k-1$ 个罐子的策略，需 $2(k-1)\cdot(n^{(k-1)/k})^{1/(k-1)} = 2(k-1)n^{1/k}$ 次。两者合计 $2k \cdot n^{1/k}$，归纳完成。故 $f_k(n) = O(n^{1/k})$，从而 $f_k(n)/f_{k-1}(n) = n^{-1/(k(k-1))} \to 0$。

第3章 图

习题 1

题目： 考虑图 3.10 中的有向无环图 G，其顶点为 a, b, c, d, e, f，边为 a→b、a→d、b→c、c→f、d→e、e→f。它有多少个不同的拓扑序（topological ordering）？

答案： 共 6 个。a 必在首位、f 必在末位；中间四个节点只需满足 b 在 c 之前、d 在 e 之前，枚举得 b,c,d,e；b,d,c,e；b,d,e,c；d,e,b,c；d,b,e,c；d,b,c,e，共 6 种。

习题 2

题目： 给出一个算法，判定给定的无向图是否含圈。若含圈，算法应输出其中一个圈（不必输出图中所有的圈，一个即可）。对 n 个节点、m 条边的图，运行时间应为 O(m+n)。

答案： 先求连通分量（O(m+n)），在每个分量上从任一节点做 BFS 得 BFS 树 T。若 G 的每条边都出现在 T 中，则 G 就是这棵树，无圈；否则取一条不在 T 中的边 e=(v,w)，设 u 为 T 中 v、w 的最近公共祖先（沿父指针上溯求得），则 e 连同 T 中 v 到 u、u 到 w 的路径构成一个圈。总时间 O(m+n)。

习题 3

题目： 3.6 节中计算 DAG 拓扑序的算法反复找出没有入边的节点并将其删除，只要输入图确实是 DAG，这终将产生一个拓扑序。现在假设给定的是任意有向图 G，它可能是 DAG 也可能不是。请扩展该拓扑排序算法，使对输入的有向图 G 输出以下两者之一：(a) 一个拓扑序，从而证明 G 是 DAG；或 (b) G 中的一个圈，从而证明 G 不是 DAG。对 n 个节点、m 条边的有向图，算法运行时间应为 O(m+n)。

答案： 沿用原算法：若每轮都能找到入度为 0 的节点，则得到拓扑序。若某轮所有剩余节点都有入边，则从任一节点出发，反复沿一条入边走到其入邻居（取邻接表中第一条入边，故每步 O(1)），由于每个节点都有入边，至多走 n 步必重复访问某个节点 r；两次访问 r 之间经过的节点即构成一个圈（按行进方向的反向读出）。总时间 O(m+n)。

习题 4

题目： 受纳博科夫（Vladimir Nabokov）这位伟大康奈尔人的榜样启发，你的一些朋友成了业余鳞翅目昆虫学家（研究蝴蝶）。他们旅行归来带回标本时，往往很难说清捕到了多少个不同物种——因为许多物种彼此极其相似。有一天他们带回 n 只蝴蝶，相信每只都属于两个物种之一，以下称为 A 和 B。他们想把 n 个标本分成两群——属于 A 的和属于 B 的——但直接给任一标本贴标签非常困难，于是采用如下办法。对每一对标本 i 和 j，他们并排仔细研究；若判断足够有把握，就把这对 (i, j) 标为"相同"（认为二者来自同一物种）或"不同"（认为来自不同物种）；他们也可以对某一对不作判断，此时称这对是"含糊的"（ambiguous）。于是现在他们有 n 个标本，以及 m 条判断（对未被判为含糊的那些对的"相同"或"不同"）。他们想知道这些数据是否与"每只蝴蝶属于 A 或 B 之一"的想法相容。更具体地说，称这 m 条判断是**一致的**（consistent），若能把每个标本标为 A 或 B，使得：对每条标为"相同"的对 (i, j)，i 与 j 标号相同；对每条标为"不同"的对 (i, j)，i 与 j 标号不同。当他们正为此苦恼时，其中一人意识到你大概有算法能立刻回答这个问题。请给出运行时间为 O(m+n) 的算法，判定这 m 条判断是否一致。

答案： 建无向图：标本为顶点，每条判断为一条边并记为"相同"或"不同"。对每个连通分量从任一节点开始 BFS，起点任意标 A，沿边传播：遇"相同"边则邻点同标，遇"不同"边则邻点取反标号；与任何判断无关的孤立点任意标 A。标号完成后逐条检查判断是否被满足（"相同"边两端同号、"不同"边两端异号），全部满足则一致，否则不一致。O(m+n)。

习题 5

题目： 二叉树（binary tree）是每个节点至多有两个孩子的有根树。用归纳法证明：任意二叉树中，有两个孩子的节点数恰好比叶子数少 1。

答案： 对节点数归纳。单节点树只有 1 个叶子、0 个双孩子节点，成立。设 T 有 $n>1$ 个节点，取一个叶子 v，其父为 u，删除 v 得 T'。若 u 原本只有 v 一个孩子，则 T' 中 u 变成叶子，叶子数与双孩子节点数都不变；若 u 原有另一个孩子，则 u 从双孩子节点变为单孩子节点，叶子数减 1、双孩子节点数也减 1。两种情形都有 $n_0(T)-n_2(T) = n_0(T')-n_2(T')$，由归纳假设 $n_0(T') = n_2(T')+1$，故 $n_0(T) = n_2(T)+1$。

习题 6

题目： 设 G=(V,E) 是连通图，$u \in V$ 是指定顶点。假设以 u 为根做深度优先搜索（DFS）得到包含 G 全部节点的树 T；又以 u 为根做广度优先搜索（BFS），得到的仍是同一棵树 T。证明 G = T。（换言之，若 T 同时是以 u 为根的 DFS 树和 BFS 树，则 G 不可能含有不属于 T 的边。）

答案： 反设存在边 $e=\{a,b\} \in E$ 而不属于 T。因 T 是 DFS 树，e 的两端必有祖孙关系，设 a 是 b 的祖先。又因 T 是 BFS 树，a、b 到根 u 的距离至多相差 1；而 a 是 b 的祖先意味着 $\mathrm{dist}(u,b) \geq \mathrm{dist}(u,a)$，故 $\mathrm{dist}(u,b) = \mathrm{dist}(u,a)+1$，即 a 恰为 b 的直接父节点，于是 $\{a,b\}$ 是 T 的边，与假设矛盾。故 G 无 T 之外的边，G = T。

习题 7

题目： 你的朋友研究无线网络，正在考察 n 个移动设备组成的网络的性质。设备（随其主人）移动，他们在任一时刻这样定义图：n 个设备各为一个节点，若设备 i 与 j 的物理距离不超过 500 米，则连边（此时称 i 与 j "在范围内"）。他们希望网络始终连通，因此约束设备运动满足：任一时刻每台设备 i 都在至少 n/2 台其他设备 500 米范围内（设 n 为偶数）。他们想知道：这一性质本身是否保证网络保持连通？把问题形式化为下述关于图的命题。**命题：** 设 G 是 n 个节点的图，n 为偶数。若 G 的每个节点度数至少为 n/2，则 G 连通。判断该命题为真或为假，并给出该命题或其否定的证明。

答案： 命题为真。反设 G 不连通，取节点数最少的连通分量 S，因至少有两个分量故 $|S| \leq n/2$。S 中任一节点 u 的邻居全在 S 内，故 $\deg(u) \leq |S|-1 \leq n/2-1 < n/2$，与每个节点度数至少 n/2 矛盾。

习题 8

题目： 媒体上关于互联网与万维网结构的一些报道常围绕这样的问题：这些网络中典型节点之间相距多远？细读会发现，许多报道混淆了网络的**直径**（diameter）与**平均距离**（average distance），常把二者当作同一回事来回替换。如正文所述，图 G=(V,E) 中两个节点 u 和 v 之间的**距离** $\mathrm{dist}(u,v)$ 是连接二者的路径所含边数的最小值；G 的**直径** $\mathrm{diam}(G)$ 是任意一对节点间距离的最大值。再定义一个相关量，称为 G 中的**平均点对**

距离（average pairwise distance），记作 apd(G)：在所有 C(n,2)（即从 n 个点中取 2 个的组合数）个不同的节点对 {u,v} 上取 u 与 v 之间距离的平均值，即

apd(G) = [∑_{ {u,v}⊆V } dist(u,v)] / C(n,2)。

一个简单例子可说明存在 diam(G) ≠ apd(G) 的图：设 G 是三个节点 u, v, w 的图，只有两条边 {u,v} 和 {v,w}，则 diam(G) = dist(u,w) = 2，而 apd(G) = [dist(u,v)+dist(u,w)+dist(v,w)]/3 = 4/3。当然，对这个三节点图两数相差并不大，于是自然会问二者之间是否总有紧密关系。下述命题试图把这一点说精确。**命题：** 存在正整数 c，使得对所有连通图 G 都有 diam(G)/apd(G) ≤ c。判断该命题为真或为假，并给出该命题或其否定的证明。

答案： 命题为假，比值可任意大。对任意 c，取 k−1 个节点排成路径 $u_1,u_2,\dots,u_{k-1}$，再取 n−k+1 个叶子 $v_1,\dots,v_{n-k+1}$，每个只与 u_1 连一条边。此时 diam(G) = dist(v_1,u_{k−1}) = k。含 $u_1\dots u_{k-1}$ 中至少一个端点的点对至多 kn 个，距离至多 k，其余点对距离至多 2，故 apd(G) < [2·C(n,2) + k^2n]/C(n,2) < 2 + $2k^2/(n-1)$。取 n−1 > $2k^2$ 得 apd(G) < 3，再取 k > 3c 即得 diam(G)/apd(G) > 3c/3 = c。故不存在这样的常数 c。

习题 9

题目： 一种自然的直觉是：通信网络中相距很远（相隔很多跳）的两个节点，其连接比靠得很近的两个节点更脆弱。有不少算法结果在某种程度上基于对这一直觉的不同精确化方式，下面这个涉及路径对节点删除的敏感性。设 n 个节点的无向图 G=(V,E) 含有两个节点 s 和 t，且 s 与 t 之间的距离严格大于 n/2。证明必存在某个节点 v（v ≠ s 且 v ≠ t），使得从 G 中删除 v 会破坏所有 s-t 路径（换言之，删除 v 后所得图中不存在从 s 到 t 的路径）。并给出运行时间为 O(m+n) 的算法找出这样的 v。

答案： 从 s 做 BFS，设 t 出现在第 d 层，则 d > n/2。若第 1 至 d−1 层每层都至少有 2 个节点，则这些层已含至少 2(n/2) = n 个节点，而 s、t 都不在其中，矛盾；故存在某层 L_i 只含单个节点 v。令 X = {s} ∪ L_1 ∪ … ∪ L_{i−1}，则 s ∈ X、t ∉ X，且由 BFS 性质 X 发出的边都终止于 L_i。任一 s-t 路径必在某处离开 X，从而必经 L_i，而 L_i 只有 v，故删除 v 破坏所有 s-t 路径。BFS 用时 O(m+n)，逐层检查即可定位 v。

习题 10

题目： 当然，这类网络中节点 v 与 w 之间单独一条偶然的短路径可能只是巧合；大量 v-w 短路径则更有说服力。因此除计算图中单条最短 v-w 路径外，社会网络研究者还研究确定**最短 v-w 路径条数**的问题。这个问题可以高效求解。设给定无向图 G=(V,E)，并指定其中两个节点 v 和 w。给出一个算法计算 G 中最短 v-w 路径的条数（算法不必列出所有路径，只需给出数目）。对 n 个节点、m 条边的图，运行时间应为 O(m+n)。

答案： 从 v 做 BFS 得各层 L_0={v}, L_1, L_2, …；由 BFS 性质，v 到 x 的路径为最短路当且仅当沿路各点层号每步恰增 1。令 S(x) 为最短 v-x 路径条数，则 L_1 中 S(x)=1；对 j>1 及 y ∈ L_j，S(y) = ∑{x∈L{j−1}, (x,y)∈E} S(x)。按层序计算，计算 S(y) 的代价为 deg(y)，总和为 O(m)，故总时间 O(m+n)，答案即 S(w)。

习题 11

题目： 你正在协助安全分析员监控一组联网计算机，追踪一种网络病毒的传播。系统中有 n 台计算机，标号为 $C_1,C_2,\dots,C_n$；输入给你一组**通信记录**（trace data），指明各对计算机在哪些时刻通信过。数据是一个三元组序列 (C_i,C_j,t_k)，表示 C_i 与 C_j 在时刻 t_k 交换过数据，共有 m 个三元组。假设三元组按时间顺序给

出；为简化，假设每对计算机在观察期内至多通信一次。安全分析员希望回答如下形式的问题：若病毒在时刻 x 被植入计算机 C_a，到时刻 y 它是否可能已感染计算机 C_b？感染机制很简单：若已感染的计算机 C_i 在时刻 t_k 与未感染的 C_j 通信（即记录中出现三元组 (C_i,C_j,t_k) 或 (C_j,C_i,t_k)），则 C_j 从时刻 t_k 起也被感染。感染可沿**一串**通信从一台机器传到另一台，只要该序列中没有任何一步向时间倒退。例如若 C_i 在时刻 t_k 前已被感染，且记录含 (C_i,C_j,t_k) 与 (C_j,C_q,t_r)，其中 t_k ≤ t_r，则 C_q 会经 C_j 被感染（t_k = t_r 是允许的：这表示 C_j 在同一时刻既与 C_i 又与 C_q 有连接，故病毒可由 C_i 传到 C_q）。例如设 n=4，记录为

$(C_1,C_2,4)$, $(C_2,C_4,8)$, $(C_3,C_4,8)$, $(C_1,C_4,12)$,

病毒在时刻 2 植入 C_1，则 C_3 会在时刻 8 被感染，共三步：C_1 在时刻 4 被感染，C_2 在时刻 8 从 C_1 获得病毒，C_4 在时刻 8 从 C_2 获得病毒，C_3 在时刻 8 从 C_4 获得病毒。另一方面，若记录为

$(C_2,C_3,8)$, $(C_1,C_4,12)$, $(C_1,C_2,14)$,

同样在时刻 2 把病毒植入 C_1，则观察期内 C_3 不会被感染：虽然 C_2 在时刻 14 被感染，但 C_3 与 C_2 的通信发生在 C_2 被感染**之前**，不存在时间上向前推进的、可使病毒从 C_1 传到 C_3 的通信序列。设计一个回答此类问题的算法：给定一组通信记录，判定在时刻 x 植入计算机 C_a 的病毒能否在时刻 y 之前感染计算机 C_b。算法运行时间应为 O(m+n)。

答案： 建图把时间信息化为可达性。顶点为 (C_i,t)（时刻 t 涉及 C_i 的事件）；对每个三元组 (C_i,C_j,t_k) 建顶点 (C_i,t_k)、(C_j,t_k)，在二者间加双向有向边，并按时间顺序为每台计算机维护其事件链表；若 (C_i,t_k) 不是 C_i 的第一个事件，则从上一次事件 (C_i,t) 向 (C_i,t_k) 加一条有向边（C_j 同理）。查询时在 C_a 的链表中找到时刻 ≤ x 的最后一个事件 (C_a,x')，从它做 BFS；若可到达形如 (C_b,y') 且 y' ≤ y 的节点，则可能感染，否则不可能。顶点与边均为 O(m)，故总时间 O(m+n)。

习题 12

题目： 你正在协助一群民族志学者分析他们通过访谈村民收集的口述史料，以了解过去两百年间生活于此的人们的生活。通过这些访谈，他们了解到 n 个人（均已去世），记为 $P_1,P_2,\ldots,P_n$，并收集到关于这些人生活年代先后关系的事实。每条事实具有以下两种形式之一：

- 对某些 i 和 j，P_i 死于 P_j 出生之前；或
- 对某些 i 和 j，P_i 与 P_j 的生平至少有部分重叠。

自然，他们并不确定所有这些事实都正确：记忆并不那么可靠，许多内容是口耳相传的。因此他们希望你判定所收集的数据是否至少**内部一致**（internally consistent），即是否存在一组人的生活年代使得他们了解到的所有事实同时成立。请给出一个高效算法：要么给出 n 个人各自的出生日期与死亡日期使命题全部成立，要么（正确地）报告不存在这样的日期——即民族志学者收集的事实内部不一致。

答案： 为每人建两个节点 b_i（出生）与 d_i（死亡），并加边 (b_i,d_i) 表示出生早于死亡。对"P_i 死于 P_j 出生之前"加边 (d_i,b_j)；对"P_i 与 P_j 生平至少部分重叠"加边 (b_i,d_j) 与 (b_j,d_i)。数据一致当且仅当该有向图无圈：若有圈，圈上各事件互相要求严格先后，矛盾；若无圈，取其拓扑序作为全部出生/死亡事件的先后次序，按序赋以递增日期，即得一组满足所有事实的日期。用拓扑排序判定并构造，图有 2n 个节点、n+2m 条边，时间 O(n+m)。

第4章 贪心算法

习题 1

题目： 判断下列陈述的真假。若为真，给出简要解释；若为假，给出反例。

设 G 是任意连通无向图，每条边 e 上有互不相同的代价 c(e)。设 e *是 G 中最便宜的边，即对每条满足 e ≠ e* 的边 e 都有 c(e*) < c(e)。那么存在 G 的一棵最小生成树（minimum spanning tree）T，它包含边 e*。

答案： 真。e* 是 Kruskal 算法最先考虑的一条边，故必被选入其生成树；而 Kruskal 算法输出的即为最小生成树。

习题 2

题目： 对以下两个陈述分别判断真假。若为真，给出简要解释；若为假，给出反例。

(a) 给定图 G 上最小生成树问题的一个实例，各边代价均为正且互不相同。设 T 是该实例的一棵最小生成树。现把每条边的代价 c_e 换成它的平方 c_e²，得到同一张图、不同代价的新实例。问：T 是否仍必须是这个新实例的最小生成树？

(b) 给定有向图 G 上最短 s-t 路问题的一个实例。假设所有边代价为正且互不相同。设 P 是该实例的一条最小代价 s-t 路。现把每条边的代价 c_e 换成它的平方 c_e²，得到同一张图、不同代价的新实例。问：P 是否仍必须是这个新实例的最小代价 s-t 路？

答案： (a) 真。平方不改变代价之间的大小次序，Kruskal 算法在新代价下按完全相同的顺序选边，得到同一棵生成树，故 T 仍是唯一的 MST（只需比较相对顺序，而非具体数值）。

(b) 假。取边 (s,v)、(v,t) 代价均为 3，(s,t) 代价为 5：原最短路是单边 (s,t)（代价 5 < 6），平方后经 v 的路代价 9 + 9 = 18 < 25，最短路变成了 s-v-t。

习题 3

题目： 你为一家在纽约与波士顿之间大量运送包裹的卡车公司做咨询。运量很大，公司每天必须在这两地之间派出若干辆卡车。卡车有固定的载重上限 W。包裹逐个到达纽约站，第 i 个包裹重量为 w_i。卡车车站很小，任一时刻至多只能停一辆卡车。公司规定包裹必须按到达顺序装运；否则某位顾客若发现自己之后才到的包裹反而先到波士顿，就会不满。目前公司采用一个简单的贪心算法：按到达顺序装箱，一旦下一个箱子装不下，就让这辆卡车出发。

但他们怀疑自己可能用了过多的卡车，想听听你的意见：这种情形能否改进？他们的想法是：也许可以先让一辆装得不太满的卡车出发，从而使接下来的几辆车装得更满，最终减少所需卡车数。

请证明：对给定的一组重量已知的箱子，公司现用的贪心算法确实使所需卡车数最少。你的证明应仿照区间调度问题（Interval Scheduling Problem）所用的分析方式：找出一个使贪心解始终"领先"（stays ahead）其他所有解的度量，以此证明该贪心装车算法的最优性。

答案： 设贪心解前 k 辆车共装入前 j 个箱子，任一其他可行解前 k 辆车装入前 i 个箱子；归纳证明 i ≤ j（且 k = 1 时贪心把第一个箱子之后的箱子装到再也装不下为止，显然领先）。由第 k 辆车的归纳步：其他解从第

i'+1 个箱子开始装，而贪心从第 j'+1 ≤ i'+1 个箱子开始装且一直装到装不下，故仍至少领先。取 k 为贪心所用卡车数即得最优性。

习题 4

题目： 你的几位朋友进入了方兴未艾的时间序列数据挖掘（time-series data mining）领域：在按时间先后发生的事件序列中寻找模式。股票交易所的购买记录（即"买了什么"）就是一类天然按时间排序的数据。给定这样一个很长的序列 S，朋友们希望能高效地检测其中某些"模式"，例如判断四个事件

buy Yahoo, buy eBay, buy Yahoo, buy Oracle

是否在 S 中按此顺序出现（不必连续）。

他们从一组可能的事件（如可能的交易）和一个由 n 个此类事件构成的序列 S 出发。同一事件可能在 S 中多次出现（例如同一支雅虎股票在一个序列里可能被多次买入）。若可以通过删去 S 中的某些事件，使剩下的事件按顺序恰等于序列 S'，则称 S' 是 S 的子序列（subsequence）。例如上面四个事件的序列就是序列

buy Amazon, buy Yahoo, buy eBay, buy Yahoo, buy Yahoo, buy Oracle

的子序列。

他们的目标是能够随时想出一些短序列，并快速判断它是否为 S 的子序列。于是他们向你提出这个问题：设计一个算法，输入为两个事件序列——长度为 m 的 S' 与长度为 n 的 S（二者都可能含重复事件）——在 O(m+n) 时间内判定 S' 是否为 S 的子序列。

答案： 贪心双指针：i 指向 S 中当前位置，j 指向 S' 中待匹配位置，初始均为 1；若 s_i 与 s'_j 相同则记录 k_j = i 并令 i、j 同时后移，否则只令 i 后移；当 j = m+1 时成功，否则失败。每个字符比较 O(1)，i 最多前进 n 次，故 O(n)（≤ O(m+n)）。正确性可归纳证明：若存在匹配，算法第 j 次匹配到的位置 k_j 不大于任何合法匹配的第 j 个位置 l_j，故不会错过解。

习题 5

题目： 设想一条漫长而安静的乡间公路，房屋非常稀疏地散布在路旁（可把公路看成一个线段，有东端和西端）。再假设尽管环境幽静，这些房屋的居民都是手机的重度用户。你要在路旁若干位置建手机基站，使每座房屋都位于某个基站四英里以内。请给出一个高效算法完成此事，并使基站数量尽可能少。

答案： 从最西端开始向东走，一旦遇到其西侧恰有四英里处存在房屋，就在该点设一个基站（再往东就会漏掉那座最靠西的未覆盖房屋），然后删除被该基站覆盖的所有房屋，对剩余房屋重复。形式上：设已有基站 s_1,…,s_i，则把第 i+1 个基站放在使 [s_i, s_{i+1}] 之间所有房屋均被覆盖的最大位置。设贪心解为 s_1,…,s_k、最优解为 t_1,…,t_m（自西向东），归纳可证 s_i ≥ t_i，于是 k ≤ m；若 k > m 则最优解无法覆盖全部房屋，矛盾，故 k = m。

习题 6

题目： 你的朋友在一处夏令营做辅导员，负责为初中生营员组织活动。他的一个计划是如下小型铁人三项：每位参赛者先游 20 个泳池长度，再骑 10 英里自行车，最后跑 3 英里。计划用如下规则错开出发：参赛者必须轮流使用泳池——第一位参赛者游完 20 个来回、出水并开始骑车后，第二位参赛者立即开始游泳；他一出水开始骑车，第三位就下水……依此类推。

每位参赛者有一个预计的游泳时间（完成 20 个来回所需时间）、预计的骑车时间（骑完 10 英里所需时间）和预计的跑步时间（跑完 3 英里所需时间）。你的朋友想为这场铁人三项确定一个日程（schedule）：参赛者出发的先后顺序。称某个日程的完成时间（completion time）为所有参赛者都完成三项比赛的最早时刻，假定每人在三个阶段都精确花用其预计时间（注意：参赛者可以同时骑车和跑步，但任一时刻泳池中至多一人）。若希望整场比赛尽早结束，应以什么顺序让人出发？更确切地说，请给出一个高效算法，产生完成时间尽可能小的日程。

答案： 按 b_i + r_i（骑车时间加跑步时间）从大到小排序，依次出发，复杂度 O(n log n)。交换论证：若相邻两人 i 先于 j 但 b_i + r_i < b_j + r_j（一个"逆序"），交换二者后 j 更早完成；i 在 j 原先出水的时刻出水，且因 b_i + r_i < b_j + r_j 而比原来 j 的完成时刻更早完成，故完成时间不增。反复消除逆序即得贪心序，故其完成时间不大于任何最优解。

习题 7

题目： 风靡一时的西班牙语搜索引擎 El Goog 每次重建索引都要做大量计算。所幸公司拥有一台大型超级计算机，以及几乎无限多的高端 PC。他们把整个计算分解为 n 个相互独立的任务，标为 J_1,…,J_n。每个任务分两个阶段：先在超级计算机上预处理（preprocess），再在某一台 PC 上完成（finish）。设任务 J_i 在超级计算机上需 p_i 秒，随后在 PC 上需 f_i 秒。

由于现场至少有 n 台 PC，各任务的最后阶段可完全并行——所有任务能同时进行。但超级计算机一次只能处理一个任务，因此管理员需要确定把任务送入超级计算机的顺序。顺序中的第一个任务在超算上一完成，即可交给 PC 收尾，同时第二个任务可送入超算；第二个任务在超算上完成后即可转给 PC，无论第一个任务是否已完成（因为 PC 并行工作）；其余依此类推。

称一个日程（schedule）为各任务在超级计算机上的一个排序，其完成时间（completion time）为所有任务在 PC 上全部完成的最早时刻。这是要最小化的重要指标，因为它决定 El Goog 多快能生成新索引。请给出一个多项式时间算法，找出完成时间尽可能小的日程。

答案： 按 f_i 从大到小（最后阶段耗时最长者先上超算）排序，O(n log n)。因为超算的总工作时间与顺序无关，最后一台被交接的机器决定结果，故直觉上最后收尾的应是 f 最小者。交换论证：若相邻两任务 J_k 在前、J_l 在后但 f_k < f_l，交换后其余任务完成时间不变；J_l 提前开始并提前完成，J_k 的上机时刻不变故与其原完成时刻相衔接，所有相关完成时间均不增。反复消去逆序即得贪心序，故最优。

习题 8

题目： 设给定一个连通图 G，各边代价互不相同。证明 G 有唯一的最小生成树。

答案： 反证。设 T ≠ T' 是两棵不同的最小生成树，取 e' ∈ T' \ T，把 e' 加入 T 得到环 C，设 e 为 C 上最贵的边。由环性质（Cycle Property），e 不属于任何最小生成树；但 e 至少在 T 或 T' 之一中，矛盾。

习题 9

题目： 最小生成树问题的一个基本动机是为节点集设计总成本最小的连通网络。这里考察另一类目标：设计一个使最贵的那条边尽可能便宜的连通网络。

具体地，设 G = (V, E) 是含 n 个顶点、m 条边的连通图，边代价均为正，且可假定互不相同。设 T = (V, E') 是 G 的一棵生成树；定义 T 的瓶颈边（bottleneck edge）为 T 中代价最大的边。

若不存在 G 的生成树 T' 其瓶颈边更便宜，则称生成树 T 是 G 的一棵最小瓶颈生成树（minimum-bottleneck spanning tree）。

(a) G 的每棵最小瓶颈树都是 G 的最小生成树吗？证明或给出反例。

(b) G 的每棵最小生成树都是 G 的最小瓶颈树吗？证明或给出反例。

答案： (a) 假。取顶点 {v_1,v_2,v_3,v_4} 的完全图，边 (v_i,v_j) 的权为 i+j。所有树的瓶颈边代价至少为 5，路径 v_3–v_2–v_1–v_4 即为一棵最小瓶颈树（瓶颈为 5）；但它不是最小生成树，因为以 v_1 为中心、连向其余各点的星形树总权更小。

(b) 真。反证：设 T 是 MST，T' 是瓶颈边更便宜的生成树。则 T 中必有某边 e 比 T' 的所有边都重。把 e 加入 T' 得到环 C，e 是 C 上最重的边（C 的其余边都在 T' 中）。由割性质（Cut Property），e 不属于任何最小生成树，与 $e \in T$ 矛盾。

习题 10

题目： 设 G = (V, E) 是（无向）图，每条边 $e \in E$ 的代价 $c_e \geq 0$。假定已给定 G 的一棵最小代价生成树 T。现在向 G 中加入一条新边，它连接两个节点 $v, w \in V$，代价为 c。

(a) 给出一个高效算法，判断在把新边加入 G（但不加入树 T）后，T 是否仍是最小代价生成树。使算法运行时间为 O(|E|)。能否做到 O(|V|)？请说明你对表示树 T 与图 G 所用数据结构所作的假设。

(b) 假设 T 不再是最小代价生成树。给出一个线性时间算法（时间 O(|E|)），把 T 更新为新的最小代价生成树。

答案： (a) 用邻接表表示 T，在 T 中找 v 到 w 的唯一路径 P。若 P 上所有边的代价都小于 c（代价互异的情形下），由环性质新边不属于任何 MST，T 仍最优；只要 P 上有一条边代价大于 c，该最重边 f 就不属于任何 MST，T 不再最优。找路径需 O(|V|)（BFS/DFS），故可在 O(|V|) 内完成，优于 O(|E|)；代价有相等时需少量特殊处理（可用微小扰动使互异）。

(b) 若 P 上最重边 f 的代价大于 c，则在 T 中用 e = (v,w) 替换 f 得 T'，T' 即新的 MST。正确性是环/割性质：对任意不在 T' 中的边 e'，加入 T' 成环 C'，可证 e' 是 C' 上最重边，故不属于任何 MST。运行时间 O(|E|)（找路径并比较）。

习题 11

题目： 设给定连通图 G = (V, E)，各边 e 有代价 c_e。此前我们已知：当所有边代价互不相同时，G 有唯一的最小生成树。但当边代价并非全不相同时，G 可能有许多棵最小生成树。现在的问题是：能否让 Kruskal 算法找出 G 的全部最小生成树？

回忆 Kruskal 算法把边按代价递增排序，然后依次贪心地处理每条边，只要加入 e 不形成环就加入。当有些边代价相同时，"按代价递增"这一说法需要更细致地界定：称边的一个排序是合法的（valid），若相应的代价序列非降；称 Kruskal 算法的一次合法执行（valid execution）为以 G 的某个合法边序开始的执行。

对任意图 G 及其任意一棵最小生成树 T，是否存在 Kruskal 算法在 G 上的一次合法执行，其输出恰为 T？请证明或给出反例。

答案： 真。把各边任意标号为 e_1,…,e_m，并使 T 的边恰为 e_{m-n+1},…,e_m。设 δ 为任意两条不等代价之差的最小值，从第 i 条边的权中减去 $\delta i/n^3$。则所有权互不相同，且新权的一个排序与原权重的某个合法排

序一致；而 T 是总权减少最多的生成树，故成为新权下唯一的 MST，会被 Kruskal 在这组合法顺序下输出。

习题 12

题目： 设有 n 个视频流需依次通过一条通信链路发送。第 i 个流共含 b_i 比特，需在 t_i 秒内以恒定速率发送。同一时刻不能发送两个流，因此需要为这些流确定一个日程：它们的发送顺序。无论采用何种顺序，一个流结束与下一个流开始之间都不能有延迟。设日程从时刻 0 开始（因此总在时刻 ∑_{i=1}^n t_i 结束，与顺序无关）。假定所有 b_i 与 t_i 都是正整数。

由于你只是一名用户，链路不希望你占用过多带宽，于是用固定参数 r 施加如下限制：

(*) 对每个自然数 t > 0，你在时间区间 [0, t] 内发送的比特总数不得超过 rt。

注意该限制只对从 0 开始的时间区间施加，而不是对从其他时刻开始的区间。

称一个日程是合法的（valid），若它满足链路施加的限制 (*)。

问题：给定 n 个流（由比特数 b_i 与持续时间 t_i 描述）以及链路参数 r，判定是否存在合法日程。

例：设 n = 3，各流为 (b_1,t_1) = (2000,1)，(b_2,t_2) = (6000,2)，(b_3,t_3) = (2000,1)，链路参数 r = 5000。则按 1,2,3 的顺序发送是合法的，因为限制 (*) 得到满足：t = 1 时整个第一个流已发完，2000 < 5000·1；t = 2 时第二个流也已发送一半，2000 + 3000 < 5000·2；t = 3 与 t = 4 可类似计算。

(a) 考虑如下断言：存在合法日程当且仅当每个流都满足 b_i ≤ r t_i。判断该断言真假，并给出证明或其否证。

(b) 给出一个算法，输入为 n 个流的比特数 b_i 与持续时间 t_i 以及链路参数 r，判定是否存在合法日程。算法运行时间应为 n 的多项式。

答案： (a) 假。b_i > r t_i 只说明该流不能排在第一个。反例：t_1 = t_2 = 1，b_1 = r + 1，b_2 = r − 1；则流 2 可先发（速率 r−1 ≤ r），随后流 1 在其后 1 秒内以 r+1 发送，任意 [0,t] 内累计比特仍不超过 rt，故存在合法日程，尽管 b_1 > r t_1。

(b) 按"流速率" b_i / t_i 从小到大排序发送，并检验 $\sum_{i=1}^{n} b_i \le r \sum_{i=1}^{n} t_i$。若该不等式不成立则任何顺序都不可行（总量必然超限）；若成立，则上述按速率递增的顺序可行：任意时刻 t，前 t 秒恰为速率最低的 t 秒，发送量最小，若此顺序在某时刻超限则所有顺序都会超限。排序需 O(n log n)，判定本身 O(n)。

习题 13

题目： 一家小企业——比如只有一台大型机器的复印服务社——面临如下调度问题。每天早上他们收到一批客户任务，希望以让客户最满意的顺序在自己的单台机器上处理。客户 i 的任务需 t_i 时间完成。给定一个日程（即任务的一个排序），令 C_i 表示任务 i 的完成时刻。例如，若任务 j 第一个做则 C_j = t_j；若任务 j 紧接在任务 i 之后做则 C_j = C_i + t_j。每位客户 i 还带有一个权重 w_i，表示其对该企业的重要程度；客户 i 的满意度被认为取决于任务 i 的完成时刻。因此公司希望安排任务顺序，使完成时间的加权和 ∑_{i=1}^n w_i C_i 最小。

设计一个高效算法求解此问题：给定 n 个任务的处理时间 t_i 与权重 w_i，确定任务顺序使 ∑_{i=1}^n w_i C_i 最小。

例：设有两个任务，第一个用时 t_1 = 1、权重 w_1 = 10，第二个用时 t_2 = 3、权重 w_2 = 2。则先做任务 1 的加权完成时间为 10·1 + 2·4 = 18，而先做任务 2 的加权完成时间为 10·4 + 2·3 = 46。

答案： 按比值 w_i / t_i 从大到小排序（Smith 规则），O(n log n)。交换论证：设相邻的 i 在 j 之前但 w_j/t_j ≥ w_i/t_i，交换二者前后其余任务完成时间不变；设二者之前总完成时刻为 C，交换前后两项贡献之差为 w_j t_i − w_i t_j ≤ 0，故交换不增大目标值。反复消除逆序即得最优贪心序。

习题 14

题目： 你与一组安全顾问合作，帮助监控一个大型计算机系统。他们特别关心追踪被标记为"敏感"的进程。每个这样的进程有指定的开始与结束时刻，并在这段时间内持续运行；顾问们掌握当天将运行的所有敏感进程的计划起止时刻。

作为简单的第一步，他们写了一个名为 status_check 的程序：每次调用时运行数秒，记录当时系统上所有敏感进程的若干日志信息（我们把每次调用建模为一个时间点）。现在他们希望一天内尽可能少地调用 status_check，但又足以使每个敏感进程 P 在其执行期间至少被调用一次。

(a) 给出一个高效算法：输入所有敏感进程的起止时刻，找出尽可能小的时刻集合，在其上调用 status_check，并满足每个敏感进程 P 在其执行期间至少被调用一次。

(b) 在你设计算法的同时，安全顾问们做了一点粗略估算："假设我们能找到 k 个敏感进程，它们两两从不同时运行。那么你的算法显然至少要调用 status_check k 次：一次调用不可能同时处理其中多于一个进程。"

这当然是对的，经过进一步讨论，大家开始怀疑是否有更强的结论——某种上述论证的逆命题。设 k *为"两两从不同时运行的敏感进程集合"的最大规模。是否存在 k* 个时刻，在这些时刻调用 status_check 能使每个敏感进程执行期间都有一次调用发生？（换言之，前面那段论证是否真的是迫使我们做很多次调用的唯一原因？）判断该断言真假，并给出证明或反例。

答案： (a) 把全部进程按结束时刻非降序排列；反复取尚未被覆盖的进程中结束最早者，恰在其结束时刻插入一次 status_check，并标记所有与它相交（已被覆盖）的进程，直到全部覆盖。每步至少覆盖一个新进程，且这样得到的检查点集合最小。用时 O(n log n)（排序）加 O(n)（用队列/数组扫描标记被覆盖进程）。

(b) 真。记算法解为 C。显然 k ≤ *|C|（k* 个互不重叠的进程各需一次不同调用）。另一方面，算法每次插入检查点是由某个进程的结束时刻触发的，这些触发进程两两不相交（每个都未被此前所有检查点覆盖，故不与此前的触发进程重叠），构成 k *个互不重叠的进程，故 |C| ≤ k*。于是 |C| = k*。

习题 15

题目： 校园里一个大型学生会的主席带着如下问题来找你。她负责 n 名学生，每人在本周内被安排值一个班（shift）。这些班对应不同的工作（守主服务台、帮忙送包裹、重启出故障的信息亭等），但可把每个班看作一个连续的时间区间，且同一时刻可能有多个班在进行。

她想从这 n 名学生中选出一个监督委员会（supervising committee），每周与他们会面一次。若对每个不在委员会中的学生，其值班时间与委员会中某名学生的时间至少部分重叠，则称该委员会是完整的（complete）。这样，每个学生的表现都能被至少一名委员会成员观察到。

请给出一个高效算法，输入 n 个班的安排，输出一个规模尽可能小的完整监督委员会。

例：设 n = 3，各班时间为

周一 下午 4 点—晚上 8 点， 周二 下午 6 点—晚上 10 点， 周二 晚上 9 点—晚上 11 点。

则最小的完整监督委员会只含第二名学生，因为第二个班与第一个、第三个班都重叠。

答案： 迭代进行：在尚未被覆盖的区间中取结束最早者 x_t，在所有与之相交的区间中选结束最晚者 l 加入委员会，并标记所有与 l 相交的区间。最优性论证：设算法选出 i_1,...,i_k、最优解选出 j_1,...,j_m（均可按起止时刻排序，且不含嵌套区间）；可归纳证明 j_1,...,j_{t−1} 都不与 x_t 相交，从而 m ≥ k（若 m ≤ k−1 则 j_m 甚至覆盖不到 x_k），故贪心最优。用时 O(n log n)。

习题 16

题目： 金融领域的一些安全顾问正在为客户调查一桩可能的洗钱案。调查表明近日发生了 n 笔可疑交易，每笔都涉及向某个账户转钱。但证据尚不完整：他们不知道账户的身份、交易金额，也不知道交易发生的准确时刻。他们掌握的是每笔交易的近似时间戳：证据表明交易 i 发生在时刻 t_i ± e_i，即带有"误差范围" e_i（也就是说，它发生在 t_i − e_i 与 t_i + e_i 之间的某一时刻）。不同交易的误差范围可以不同。

最近一两天，他们发现了一个银行账户（出于某些无需细究的原因）怀疑它就是涉案账户。该账户上有 n 笔近期事件，发生在时刻 x_1, x_2, ..., x_n。为了判断"该账户正是要找的账户"这一假设是否合理，他们想知道能否把账户的 n 笔事件与 n 笔可疑交易一一配对，使得若时刻 x_i 的账户事件对应时间约为 t_j 的可疑交易，则 |t_j − x_i| ≤ e_j。换言之，账户上的活动能否在误差范围内与可疑交易对齐；麻烦之处在于不知道该把哪笔账户事件与哪笔交易配对。

请给出一个高效算法，输入上述数据，判定这样的配对是否存在。若存在应能给出；运行时间至多 $O(n^2)$。

答案： 这是点与区间的完美匹配问题：点为 x_i，区间为 [t_i − e_i, t_i + e_i]。不妨设 x_1 ≤ x_2 ≤ ... ≤ x_n，贪心扫描：对 i = 1..n，若存在尚未匹配且包含 x_i 的区间，就把 x_i 与其中右端点最早者匹配；否则判定不存在完美匹配。正确性用反证/交换：设存在完美匹配 M 且与算法在前 i 个点上一致（i 最大），第 i+1 个点在 M 中配给以 t_l 为中心的区间、算法配给以 t_j 为中心者，由算法选择 t_j + e_j ≤ t_l + e_l，可把 M 中 x_i+1 与 x_k 的配对互换得到与算法更一致的完美匹配，矛盾。每次扫描 O(n)，共 n 次，总 $O(n^2)$。

习题 17

题目： 考虑区间调度问题（Interval Scheduling Problem）的如下变体。你有一台每天 24 小时、全年无休运行的处理器。人们提交在处理器上运行日常任务（daily jobs）的请求。每个任务带有开始时刻与结束时刻；若被接受，它必须每天在起止时刻之间连续运行。（注意某些任务可在午夜前开始、午夜后结束；这使得此处情形与课上所见的区间调度问题不同。）

给定 n 个任务的列表，目标是接受尽可能多的任务（不论其长短），并满足任一时刻处理器至多运行一个任务。请给出一个运行时间为 n 的多项式的算法。为简单起见可假设没有两个任务的起止时刻完全相同。

例：考虑以下四个任务，由（开始时刻，结束时刻）对给出：

(下午 6 点, 上午 6 点), (晚上 9 点, 凌晨 4 点), (凌晨 3 点, 下午 2 点), (下午 1 点, 晚上 7 点)。

最优解是选 (晚上 9 点, 凌晨 4 点) 与 (下午 1 点, 晚上 7 点) 这两个任务，它们可以互不重叠地排开。

答案： 称包含区间 I_j 的解为 I_j-受限解。对固定 j：取 I_j 内一点 x，先删去 I_j 及所有与它相交的区间；剩余区间都不含 x，故可在 x 处把时间轴"剪开"，化为课上普通的区间调度问题（按结束时刻贪心），O(n) 求解。对所有 j = 1..n 各求一次 I_j-受限解，共 $O(n^2)$，取其中最大者。正确性：任一最优解 S 非空，必含某个 I_j，故它是 I_j-受限解，其规模不超过该 j 下求得的最大受限解，故算法结果至少与最优解一样大。

习题 18

题目： 你的朋友们计划明年寒假去加拿大北部深处的一个小镇探险。他们调研了各种出行方式，画出一张有向图：节点表示中途目的地，边表示它们之间的公路。

在此过程中他们还了解到，这一带的公路在冬季会因极端天气变得很慢，可能造成严重延误。他们找到一个很好的旅行网站，能准确预测沿各条公路的行驶速度；但行驶速度取决于一年中的时段。更准确地说，网站可回答如下形式的查询：给定连接两地 v 与 w 的边 e = (v, w)，以及从 v 出发的拟定出发时刻 t，网站返回在 w 的预计到达时刻 f_e(t)。网站保证对所有边 e 与所有时刻 t 有 f_e(t) ≥ t（不能时光倒流），且 f_e(t) 是 t 的单调递增函数（即出发越晚不会到得越早）。除此之外，各函数 f_e(t) 可以任意。例如在行驶时间不随季节变化的地区，就有 f_e(t) = t + ℓ_e，其中 ℓ_e 是从边 e 起点到终点所需的行驶时间。

朋友们想用该网站确定从起点到目的地在有向图中最快的一条路线。（假定他们从时刻 0 出发，且网站的所有预测完全正确。）请给出一个多项式时间算法，其中对网站的一次单查询（针对特定边 e 与时刻 t）算作一步计算。

答案： 推广 Dijkstra 算法（可类比"水管"模型：水从起点以与旅行相同的速度按 f_e 传播）。维护已确定集合 S，对每个 u ∈ S 记录最早到达时刻 d(u) 及最快路径上 u 的前驱 r(u)；初始 S = {s}、d(s) = 0。每步从 S 外选取使 d'(v) = min_{e=(u,v), u∈S} f_e(d(u)) 最小的 v，置 d(v) = d'(v)、r(v) = 相应 u 并加入 S，直至到达终点，再沿前驱回溯得到路径。用优先队列实现，每条边松弛至多一次，总时间 O(m log n)。正确性归纳证明：加入 v 时 d(v) 已是全局最早到达时刻——任一其他 s-v 路必先离开 S，设 y 为路上第一个 S 外节点、x 为其前驱，则经该路到 y 不早于经 x 直接到 y（f 单调且 f_e(t) ≥ t），故不早于 d(v)。

习题 19

题目： CluNet 通信公司的网络设计师们面临如下问题。他们有一个连通图 G = (V, E)，节点表示希望互相通信的站点。每条边 e 是一条通信链路，具有给定的可用带宽 b_e。

对每一对节点 u, v ∈ V，他们想选择一条 u-v 路径 P，供这一对节点通信。路径 P 的**瓶颈速率**（bottleneck rate）b(P) 是 P 所含各边带宽的最小值，即 b(P) = min_{e∈P} b_e。节点对 u, v 在 G 中的**最佳可达瓶颈速率**则是对 G 中所有 u-v 路径取 b(P) 的最大值。

为每一对节点分别记录一条路径变得非常复杂，于是其中一位设计师大胆提议：也许可以找到 G 的一棵生成树 T，使得对每一对节点 u, v，树中唯一的那条 u-v 路径的瓶颈速率恰好达到 u, v 在 G 中的最佳可达瓶颈速率。（换句话说，即使你可以在整个图中任意选择 u-v 路径，也不可能比 T 中的 u-v 路径更好。）

这个想法在 CluNet 办公室被嘲笑了好几天，怀疑是有道理的：每一对节点可能想要一条形状很不一样的路径来最大化其瓶颈速率；凭什么能有一棵单独的树同时让所有人都满意？但在几次试图推翻该想法失败之后，人们开始怀疑它可能是对的。

证明这样的树存在，并给出一个高效算法来找到它。也就是说，给出一个构造生成树 T 的算法，使得对每个 u, v ∈ V，T 中 u-v 路径的瓶颈速率等于 G 中该对节点的最佳可达瓶颈速率。

答案： 取每条边的权为 −b_e，求最小生成树（等价于按带宽从大到小做 Kruskal，即最大瓶颈生成树），复杂度 O(m log m)。正确性：若 MST 中 u-v 路径 P 的瓶颈低于某条路径 P′，取 P 上瓶颈边 e（P∪P′ 中带宽最小的边），则 e ∉ P′，用 P∪P′−{e} 可构造一个含 e 且 e 为最大权边的环，违反环性质。带宽有相等时对 b_e 作极小扰动使互异，再令扰动量趋于 0 即可。

习题 20

题目： 每年九月，在世界某个遥远山区的县里，公路队会聚在一起决定冬天清理哪些道路。该县有 n 个镇，道路系统可看作镇集上的连通图 G = (V, E)，每条边表示连接两镇的一条道路。冬天人们身处高山，不再担心道路的**长度**，而开始担心其**海拔**——这才真正决定旅途的难易。

于是每条道路（即图中每条边 e）都标有一个数 a_e，表示该路上最高点的海拔。我们假设没有两条边的海拔 a_e 恰好相同。路径 P 的**高度**（height）是 P 上所有边的 a_e 的最大值。最后，若一条从 i 到 j 的路径在所有 i-j 路径中达到最小可能高度，则称它是**冬季最优的**（winter-optimal）。

公路队要选出一个边集 E′ ⊆ E 在冬天保持畅通，其余道路不维护并对旅行者封闭。他们一致同意，无论选哪个 E′，都应满足 (V, E′) 是连通子图；并且更强地，对每一对镇 i, j，(V, E′) 中冬季最优路径的高度不大于整个图 G = (V, E) 中的高度。若 (V, E′) 具有这一性质，我们就称它是**最小海拔连通子图**（minimum-altitude connected subgraph）。

既然要维持这一关键性质，他们又希望尽量少清路。某一年，他们提出了如下猜想：

G 关于边权 a_e 的最小生成树是一个最小海拔连通子图。

（在先前某道题中我们曾断言：边权互异时最小生成树唯一。因此，由于假设所有 a_e 互异，我们可以说“该”最小生成树。）

起初这个猜想有些反直觉，因为最小生成树是尽量最小化 a_e 的**和**，而最小化海拔的目标看起来相当不同。但由于找不到反驳的论据，他们开始考虑一个更大胆的第二个猜想：

子图 (V, E′) 是最小海拔连通子图，当且仅当它包含最小生成树的全部边。

注意第二个猜想会立即蕴含第一个，因为最小生成树包含它自己的边。

于是问题如下。

(a) 第一个猜想对任意 G 和互异的海拔 a_e 都成立吗？给出证明或带解释的反例。

(b) 第二个猜想对任意 G 和互异的海拔 a_e 都成立吗？给出证明或带解释的反例。

答案： (a) 成立；(b) 成立。

(a) 设 T 为最小生成树。若 T 不是最小海拔连通子图，则存在 u, v，T 中 u-v 路径 P 与另一条 u-v 路径 P *使 P 高度更小。取 P 上 a_e 最大的边 e′，它也是 P∪P 中最重的边；由 P∪P−{e′} 可得 u′-v′ 的简单路径，加上 e′ 构成一个环且 e′ 为环上最重边，违反环性质。*

(b) 设 H = (V, E′) 不含 T 的某条边 e = (u, v)。删去 e 把 T 分成 A、B 两部分，e 是横跨 A、B 的最低海拔边。H 中任一条 u-v 路径必须从 A 跨到 B 且不能用 e，故其高度大于 a_e，因此 H 不是最小海拔连通子图。

习题 21

题目： 若图 G = (V, E) 连通且至多有 n + 8 条边（其中 n = |V|），则称它为**近树**（near-tree）。给出一个运行时间为 O(n) 的算法，输入是一棵带边代价的近树 G，返回 G 的最小生成树。你可以假设所有边代价互不相同。

答案： 因 m ≤ n + 8，图中独立环的个数 m − n + 1 ≤ 9。重复至多 9 次：用 BFS/DFS 找出 G 中一个环，删去该环上最重的边（由环性质，最小生成树不变，图仍连通）。9 次后剩下 n − 1 条边的连通图，即最小生成树。每轮 BFS 与找环为 O(m + n) = O(n)，总时间 O(n)。

习题 22

题目： 考虑无向图 G = (V, E) 上的最小生成树问题，每条边有代价 c_e ≥ 0，代价未必互不相同。若代价不全互异，一般可能有多个不同的最小代价解。假设给定一棵生成树 T ⊆ E，并保证对每个 e ∈ T，e 属于**某个**最小代价生成树。能否断言 T 本身必定是 G 的最小代价生成树？给出证明或带解释的反例。

答案： 不能。反例：4 个节点 v_1, v_2, v_3, v_4，边 $(v_1,v_2), (v_2,v_3), (v_3,v_4), (v_4,v_1)$ 代价均为 2，另有边 (v_1,v_3) 代价为 1。每条边都属于某个最小生成树（含 (v_1,v_3) 的树代价为 5），但由三条代价为 2 的边组成的生成树代价为 6，并非最小。

习题 23

题目： 回忆有向图 G = (V, E) 中计算最小代价树形图（arborescence）的问题，每条边有代价 c_e ≥ 0。这里我们只考虑 G 是**有向无环图**的情形，即它不含有向环。

与一般有向图一样，可能有多个不同的最小代价解。假设给定有向无环图 G = (V, E) 和一个树形图 A ⊆ E，并保证对每个 e ∈ A，e 属于**某个**最小代价树形图。能否断言 A 本身必定是 G 的最小代价树形图？给出证明或带解释的反例。

答案： 能。以 r 为根，边集 A 是树形图当且仅当 (i) 除 r 外每个节点入度恰为 1，(ii) r 到所有节点有路径。在 DAG 中 (i) 自动蕴含 (ii)：若某节点 v 从 r 不可达，沿入边反向追溯必重复访问节点而成环。故任取每个非 r 节点的一条入边即得树形图。于是 A 代价最小当且仅当对每个 v ≠ r，A 中进入 v 的边是所有进入 v 的边中代价最小者。题设保证 A 中每条边 e = (u, v) 在某个最小树形图中、因而是进入 v 的最小代价边，故 A 是最小代价树形图。

习题 24

题目： 时序电路是 VLSI 芯片的关键部件。下面是这种时序电路的一个简单模型。考虑一棵完全平衡二叉树，有 n 个叶节点，n 是 2 的幂。树的每条边 e 有一个关联长度 $\ell_e > 0$。从根到某个叶的**距离**是根到该叶路径上所有边长之和。

根产生一个**时钟信号**，沿边传播到各叶。我们假设信号到达某叶所需时间与根到该叶的距离成正比。

现在，若所有叶到根的距离不全相同，信号就不会同时到达各叶，这是个大问题。我们希望各叶完全同步、同时收到信号。为此只能**增加**某些边的长度（不能缩短），使所有根-叶路径长度相同。若做到这一点，则称该树（用新的边长）具有**零偏斜**（zero skew）。目标是在使所有边长之和尽量小的前提下实现零偏斜。

给出一个算法，增加某些边的长度，使所得树零偏斜且总边长最小。

例子： 考虑图 4.20 中的树，字母表示节点名，数字表示边长。

该实例的唯一最优解是把三条长度为 1 的边各增加到 2。所得树零偏斜，总边长为 12，是最小的。

答案： 采用 VLSI 中的**延迟合并嵌入**（Deferred Merge Embedding, DME），自底向上贪心。设 v 有两个孩子 v′, v″，d′、d″ 分别是 v′、v″ 子树中根到叶距离的最大值：若 d′ > d″，把 v-v″ 边加长 d′ − d″；若 d″ > d′，把 v-v′ 边加长 d″ − d′；相等则都不加。然后对 v′、v″ 为根的子树递归。该实例中即把 v-v″、v′-b、v″-d 三条长为 1 的边各加到 2，总长 12。正确性用交换论证：任一解若在某节点下方两条边都增加了正长度，可同时减去较小的增量，仍保持零偏斜且总长更小；若某节点 w 下方每条 w-叶路径都被加长，则可把子

树内各边的最小增量 δ 从这些边移到 w 的上方边，根-叶路径不变而总长减少 $(|F| - 1)\delta > 0$。故 DME 最优。

习题 25

题目： 设给定点集 $P = \{p_1, p_2, \ldots, p_n\}$ 以及 P 上的距离函数 d；d 只是 P 中点对上的函数，满足 $i \neq j$ 时 $d(p_i, p_j) = d(p_j, p_i) > 0$，且对每个 i 有 $d(p_i, p_i) = 0$。

我们定义 P 上的**层次度量**（hierarchical metric）为按如下方式构造出的任意距离函数 τ。我们建一棵有 n 个叶子的有根树 T，并给 T 的每个节点 v（叶与内部节点都算）赋一个**高度** h_v。这些高度须满足：对每个叶 v_i 有 $h(v_i) = 0$，且若 u 是 T 中 v 的父节点，则 $h(u) \geq h(v)$。我们把 P 中每个点放在 T 的一个不同叶上。现在，对任意两点 p_i 和 p_j，它们的距离 $\tau(p_i, p_j)$ 定义如下：找出 T 中包含 p_i 与 p_j 的两个叶的最小共同祖先 v，令 $\tau(p_i, p_j) = h_v$。

若对所有 i, j 都有 $\tau(p_i, p_j) \leq d(p_i, p_j)$，则称层次度量 τ 与距离函数 d **一致**（consistent）。

给出一个多项式时间算法，输入距离函数 d，输出一个层次度量 τ，具有以下性质：

(i) τ 与 d 一致；且

(ii) 若 τ' 是任何其他与 d 一致的层次度量，则对每对点 p_i, p_j 有 $\tau'(p_i, p_j) \leq \tau(p_i, p_j)$。

答案： 按 Kruskal 思路做单链接聚类：初始每个点自成一个分量，按 d 从小到大考察点对，若两点在不同分量则合并，新建节点 v 作为两个分量子树之父，令 h_v 等于该点对的距离。所得树定义的 τ 即为答案，时间 $O(n^2 \log n)$。一致性：合并发生在当前距离值 ℓ 处，而两点间的直接距离 $\geq \ell$。最优性：若某一致 τ' 在某对 (p_i, p_j) 上更大，设其在 T' 中的最小共同祖先高度为 $h' > \tau(p_i, p_j)$，则按 Kruskal 合并 p_i、p_j 时，最小生成树中 p_i-p_j 路径上跨出 T'_i 的那条边长度 $\geq h' > \tau(p_i, p_j)$，与该边当时已存在矛盾。

习题 26

题目： 微积分里最先学到的事情之一，是如何最小化像 $y = ax^2 + bx + c$（$a > 0$）这样的可微函数。另一方面，最小生成树问题是风格很不同的最小化问题：最小值只有有限多种可能的实现方式，而不是一个连续统；我们关心的是如何在不枚举这（巨大的）有限多种可能的情况下完成计算。

可以问：把这两类最小化问题放在一起会怎样？下面就是一个例子。设连通图 $G = (V, E)$，每条边 e 有一个**随时间变化的边代价**，由函数 $f_e: R \to R$ 给出，即在时刻 t 其代价为 $f_e(t)$。我们假设这些函数在整个定义域上取正值。注意构成 G 的最小生成树的边集可能随时间变化；当然，G 的最小生成树的代价也成为时间的函数，记为 $c_G(t)$。一个自然的问题于是是：求使 $c_G(t)$ 最小的时刻 t。

假设每个 f_e 是二次多项式：$f_e(t) = a_e t^2 + b_e t + c_e$，其中 $a_e > 0$。给出一个算法，输入图 G 及数值 $\{(a_e, b_e, c_e) : e \in E\}$，返回最小生成树代价最小的那个时刻 t。算法运行时间应为图节点数与边数的多项式。可假设对 $\{(a_e, b_e, c_e)\}$ 中数的算术运算每步耗常数时间。

答案： 关键观察：只要各边代价的整体排序不变，Kruskal 输出的最小生成树不变。两条抛物线至多交于 2 点，故排序只在至多 $2 \cdot C(m,2) \leq m^2$ 个交点处改变。求出所有交点，把时间轴分成 $\leq m^2$ 个区间；每个区间内排序固定，用 Kruskal 求出该区间的生成树 T_I，其代价是 $n - 1$ 个二次函数之和，仍是二次函数，可在该区间上 $O(1)$ 求最小值。在所有区间的最优解中取全局最优。总时间为 m 与 n 的多项式。

习题 27

题目： 为了理解生成树的组合结构，我们可以考虑给定图的所有可能生成树构成的空间，并研究这个空间的性质。这一策略也被用于许多类似问题。

一种做法如下。设 G 是连通图，T 与 T′ 是 G 的两棵不同生成树。若 T 恰有一条边不在 T′ 中，而 T′ 恰有一条边不在 T 中，则称 T 与 T′ 互为**邻居**（neighbors）。

现在，对任意图 G，我们可以构造一个（大）图 $\mathcal{H}$ 如下。$\mathcal{H}$ 的节点是 G 的所有生成树，若两棵对应生成树互为邻居，则这两个节点之间有一条边。

问：对任意连通图 G，所得的图 $\mathcal{H}$ 是否总是连通的？给出 $\mathcal{H}$ 恒连通的证明，或者给出一个使 $\mathcal{H}$ 不连通的连通图 G 的例子（并解释）。

答案： 总是连通。更强地可证：若生成树 T、T′ 满足 $|F - F'| = |F' - F| = k$，则 $\mathcal{H}$ 中存在长度为 k 的 T-T′ 路径。对 k 归纳：$k = 1$ 时即邻居；$k > 1$ 时取 $f' \in F' - F$，$T \cup \{f'\}$ 含唯一的环 C，C 中必有边 $f \notin F'$（否则 F′ 含环），令 $T'' = T \cup \{f'\} - \{f\}$，则 $|F'' - F'| = k - 1$，由归纳有长 $k - 1$ 的 T″-T′ 路径，而 T 与 T″ 相邻。故 $\mathcal{H}$ 连通。

习题 28

题目： 假设你是 CluNet 网络公司的顾问，他们有以下问题。他们目前正在处理的网络建模为 n 个节点的连通图 $G = (V, E)$。每条边 e 是一根光纤电缆，由两家公司之一所有——两家公司姑且称为 X 和 Y——并租给 CluNet。

他们的计划是选择 G 的一棵生成树 T，并升级 T 中各边对应的链路。商务人员已经与公司 X 和 Y 达成协议，规定某个整数 k，使得在所选的树 T 中，恰有 k 条边归 X 所有，$n - k - 1$ 条边归 Y 所有。

CluNet 管理层现在面临如下问题：他们完全不清楚是否存在满足这些条件的生成树 T，也不知道如何找到一棵。因此他们把问题交给了你：给出一个多项式时间算法，输入 G（每条边标有 X 或 Y），或者 (i) 返回一棵恰有 k 条边标为 X 的生成树，或者 (ii) 正确地报告不存在这样的树。

答案： 用最小生成树算法求 X 边最少的生成树 T（X 边权 0、Y 边权 1）和 X 边最多的生成树 T′（X 边权 1、Y 边权 0），设其中 X 边数分别为 a、b。若 $k < a$ 或 $k > b$，则不存在。若 $k = a$ 或 $k = b$，直接返回相应树。否则利用习题 27 的图 $\mathcal{H}$：沿 T 到 T′ 的路径逐次换边，相邻两树的 X 边数至多相差 1，从 a 变到 b 必经过 k，返回该树。多项式时间。

习题 29

题目： 给定 n 个自然数 $d_1, d_2, \ldots, d_n$ 的列表，说明如何在多项式时间内判定是否存在无向图 $G = (V, E)$，其各节点度数恰为这些数 $d_1, d_2, \ldots, d_n$。（即若 $V = \{v_1, v_2, \ldots, v_n\}$，则 v_i 的度应恰为 d_i。）G 不应含同一对节点间的多重边，也不应含两端为同一节点的“自环”边。

答案： 用 Havel–Hakimi 递归。若某个 $d_i = 0$，该点必为孤立点，删去它后对更小的实例递归。否则所有 $d_i > 0$，排序使 $d_1 \geq d_2 \geq \ldots \geq d_n > 0$，构造新序列 $L = \{d_1 - 1, d_2 - 1, \ldots, d_{d_n} - 1, d_{d_n+1}, \ldots, d_{n-2}, d_{n-1}\}$（前 d_n 个数减 1，去掉最后一个数），断言原序列可图当且仅当 L 可图，递归判定。正确性用交换论证：若某实现中度数最大的 v_n 未与 $v_1, \ldots, v_{d_n}$ 全部相连，则存在 $i < j$ 使 v_n 连 v_j 而不连 v_i，且存在 v_k

同时连 v_i 而不连 v_j；把边 (v_i, v_k)、(v_j, v_n) 换成 (v_i, v_n)、(v_j, v_k)，所有度数不变，反复变换即得所需图。复杂度多项式（朴素实现 $O(n^2)$）。

习题 30

题目： 设 G = (V, E) 是有 n 个节点的图，其中每一对节点之间都有一条边。每条边 (i, j) 有正权 w_ij，并假设这些权满足**三角不等式** w_ik ≤ w_ij + w_jk。对子集 V′ ⊆ V，我们用 G[V′] 表示在 V′ 上诱导出的子图（带权）。

给定 V 的一个子集 X ⊆ V，含 k 个**终端**（terminals），它们必须通过边被连通。我们称 X 上的一个**Steiner 树**是一个集合 Z，满足 X ⊆ Z ⊆ V，连同 G[Z] 的一棵生成子树 T。Steiner 树的**权**是树 T 的权。

证明：求 X 上最小权 Steiner 树的问题可在 O(n^{O(k)}) 时间内求解。

答案： 在 Steiner 树 T（点集 X ∪ Z）中，每个额外节点（Z 中节点）度数至少为 3：否则由三角不等式，可用连接其两个邻居的一条边替换它的两条关联边，不增大权且减少节点。t 个节点的树度数和为 2t − 2，故树叶数不少于度数大于 2 的节点数，于是 |Z| ≤ k。因此只需枚举所有形如 X ∪ Z 且 |Z| ≤ k 的点集（至多 ∑_{i=0}^{k} C(n,i) = O(n^k) 个），对每个点集求最小生成树并取最小者，总时间 n^{O(k)}。

习题 31

题目： 回到最小生成树问题最初的动机。给定连通无向图 G = (V, E)，边长为正数 {ℓ_e}，我们想求它的一个生成子图。现在假设我们愿意接受一个比树“更稠密”的子图 H = (V, F)，并希望保证：对每对顶点 u, v ∈ V，H 中最短 u-v 路径的长度不比 G 中最短 u-v 路径的长度长太多。路径 P 的**长度**指 P 上所有边 e 的 ℓ_e 之和。

下面是 Kruskal 算法的一个变体，用来产生这样的子图。

- 首先按长度递增排序所有边。（可假设所有边长互不相同。）
- 然后按顺序考察每条边，构造子图 H = (V, F)。
- 当考察边 e = (u, v) 时，若 H 中当前不存在 u-v 路径，则把 e 加入 H（标准 Kruskal 也会这样做）。否则，若 H 中已存在 u-v 路径，令 d_uv 表示 H 中最短这样的路径的长度（长度仍按 {ℓ_e} 计）；若 3ℓ_e < d_uv，也把 e 加入 H。

换言之，即使 u 与 v 已在同一连通分量中，只要加入 e 能把它们的最短路径距离减少足够多，我们也加入 e。

设 H = (V, F) 是该算法返回的 G 的子图。

(a) 证明：对每对节点 u, v ∈ V，H 中最短 u-v 路径的长度至多是 G 中最短 u-v 路径长度的 3 倍。

(b) 尽管 H 能近似保持最短路径距离，该算法产生的 H 不可能太稠密。设 f(n) 表示：在所有 n 个节点的输入图（带边长）上，该算法输出可能达到的最大边数。证明

lim_{n→∞} f(n)/n² = 0。

答案： (a) 对每条 e = (u, v) ∈ F，H 中都有长度 ≤ 3ℓ_e 的 u-v 路径（加入时成立；若 e 被拒，说明当时已有这样的路径，而 H 之后只增边）。对 G 中最短 s-t 路径 Q 的每条边 (u, v)，用相应的 u-v 路径替换后再短路掉产生的环，得到 H 中长度至多 3·len(Q) 的 s-t 路径。

(b) 先证 H 不含长度 ≤ 4 的环：若存在这样的环 C，设 e = (u, v) 是最晚加入 C 的边，加入时 H 中已有由至多 3 条边（每条长 ≤ ℓ_e）组成的 u-v 路径，即 d_uv ≤ 3ℓ_e，与加入条件矛盾。再用引理（若 $0 \le a_i \le n$ 且 $\sum a_i \ge \varepsilon n^2$，则至少 εn/2 个 $a_i \ge \varepsilon n/2$）可得 H 的边数为 $o(n^2)$；另一证法：无 ≤ 4 环的 n 点图必有度 ≤ √n 的节点，故 g(n) ≤ g(n−1) + √n，解得 $g(n) = O(n^{3/2}) = o(n^2)$。

习题 32

题目： 考虑有向图 G = (V, E)，根 r ∈ V，边代价非负。本题考虑最小代价树形图算法的若干变体。

(a) 4.9 节讨论的算法如下：修改代价，考虑零代价边构成的子图，在其中找一个有向环，若存在则收缩。请简要论证：不必寻找环，我们可以改为识别并收缩该子图的强连通分量。

(b) 算法过程中，我们定义 y_v 为进入 v 的边的最小代价，并把所有进入 v 的边 e 的代价改为 c′_e = c_e − y_v。假设我们改用如下修改后的代价：c″_e = max(0, c_e − 2y_v)。这一改动很可能把更多边变成 0 代价。假设此时我们找到一棵 0 代价的树形图 T。证明 T 的代价至多是原图中最小代价树形图代价的两倍。

(c) 假设你没有找到 0 代价的树形图。收缩所有 0 代价强连通分量，在所得图上递归地应用同一过程，直到找到树形图为止。证明这样得到的 T 的代价至多是原图最小代价树形图代价的两倍。

答案： (a) 在零代价边子图中，每个非平凡强连通分量内都存在有向环，且各强连通分量互不相交、可独立收缩；收缩后若需展开，从该分量中进入边最接近根的端点出发做 DFS，保留一条 DFS 树边并删去其余内部边，所得仍是同代价树形图，故可整块收缩强连通分量（而不只是某个环）。

(b) *c″_e = max(0, c_e − 2y_v) ≥ c_e − 2y_v，故 c_e ≤ c″_e + 2y_v。T 的 c″ 代价为 0，于是* $c(T) = \sum_{e\in T} c_e \le \sum_{e\in T} c''_e + 2\sum_{v\ne r} y_v = 2\sum_{v\ne r} y_v \le 2c(T_{opt})$*，因为每个进入 v 的边代价至少为 y_v，* $\sum_{v\ne r} y_v$ 是任何树形图代价的下界。

(c) 对递归层数归纳。设第 i 层图为 G^i、代价为 c^i、算法所得树形图为 T^i、最优为 T^i_{opt}。关键不等式为 $y_v \le c^{i-1}_e - c^i_e \le 2y_v$ 及 $c^i(T^i) \le 2c^i(T^i_{opt})$；把上一层的最优树形图按 (a) 展开成 G^{i-1} 上的树形图 A（代价不增），再逐层放大可得 $c^{i-1}(T^{i-1}) \le 2c^{i-1}(T^{i-1}_{opt})$，最终 $c(T) = c^0(T^0) \le 2c^0(T^0_{opt}) = 2c(T_{opt})$。

习题 33

题目： 给定有向图 G = (V, E)，其中每条边的代价为 0 或 1。还假设 G 有一个节点 r，从 r 到其他每个节点都有路径。再给定一个整数 k。给出一个多项式时间算法，它要么构造出以 r 为根、代价恰为 k 的树形图，要么（正确地）报告不存在这样的树形图。

答案： 先求最小代价树形图 (V, F)（代价 a），再求最大代价树形图 (V, F′)（代价 b；把代价取负后用同一多项式算法即可）。若 k < a 或 k > b，则不存在。否则做交换：对两个不同树形图 F、F′，取二者对称差中在各自树形图中到 r 距离最小的边 e = (u, v) ∈ F′，则存在 (w, v) ∈ F，令 F″ = F − {(w, v)} + {e} 仍是树形图（因 e 是最靠 r 的差异边，F 中到 v 的路径不含 (w, v) 也不含 e）。逐步把 F 变换为 F′，每步恰使与 F′ 公共边数加一，而代价每次变化至多 1（代价为 0/1），故从 a 到 b 之间每个整数都可取到，变换过程中必出现代价恰为 k 的树形图并返回之。

第5章 分治

习题 1

题目： 你正在分析来自两个独立数据库的某些难以获取的数据。每个数据库包含 n 个数值——因此总共有 2n 个值——并且可以假设没有两个值相同。你希望确定这 2n 个值的中位数，这里把中位数定义为第 n 小的值。

然而，你访问这些值的唯一方式是对数据库进行查询（query）。在一次查询中，你可以向两个数据库之一指定一个值 k，被查询的数据库会返回它包含的第 k 小的值。由于查询代价高昂，你希望用尽可能少的查询计算出中位数。

请给出一个最多使用 O(log n) 次查询求出中位数值的算法。

答案： 设 A、B 为两数据库，k = ⌊n/2⌋。查询 A(k) 与 B(k)：若 A(k) < B(k)，则 B(k) 至少是合并后的第 2k 小（≥ 第 n 小），故 B 的后半部分可全部舍弃；同时 A 的前 k 个元素都小于中位数，可舍弃。于是保留 A 的后半段（k 个）与 B 的前半段（k 个），递归求其中位数。递归式 $Q(n) = Q(\lfloor n/2 \rfloor) + 2$，解得 $Q(n) \le 2\lceil \log_2 n \rceil = O(\log n)$ 次查询。（实现上只能通过"取第 i 小"的查询来定位子数组元素。）

习题 2

题目： 回顾求逆序对（inversion）数目的问题。如正文所述，给定由 n 个数 $a_1, \ldots, a_n$ 构成的序列，假设它们互不相同，定义逆序对为满足 $i < j$ 且 $a_i > a_j$ 的数对。

我们把计数逆序对作为衡量两种次序差异程度的一个好指标。然而，人们可能觉得这个度量过于敏感。若 $i < j$ 且 $a_i > 2a_j$，我们称数对 $i < j$ 为一个**显著逆序对**（significant inversion）。请给出一个 O(n log n) 算法，计数两种次序之间的显著逆序对数目。

答案： 仿照计数逆序的分治算法（归并排序）：递归处理左右两半，返回各自的显著逆序数及已排序序列；合并时用双指针统计跨越中线的显著逆序对——对右半每个 b_j，统计左半中满足 $b_i > 2b_j$ 的元素个数（因 b_i 已排序，指针单调移动即可 O(n) 完成）；最后归并两半得到有序序列。由于计数条件带因子 2，统计与排序需分两趟合并（先排序合并，再用原两半的有序序列计数）。递归式 $T(n) = 2T(n/2) + O(n)$，解得 $T(n) = O(n \log n)$。

习题 3

题目： 假设你为一家担心欺诈检测的银行做咨询，他们带着如下问题来找你。他们有一批没收来的 n 张银行卡，怀疑它们被用于欺诈。每张银行卡是一个带磁条和加密数据的小塑料片，对应银行中一个唯一的账户。每个账户可以对应许多张银行卡，若两张卡对应同一账户，则称它们是**等价的**（equivalent）。

很难直接从卡上读出账号，但银行有一台高科技"等价测试器"（equivalence tester），它取两张卡，经过一些计算后判定二者是否等价。

他们的问题是：在这 n 张卡中，是否存在多于 n/2 张卡彼此全部等价？假设你对卡唯一能做的操作是任取两张插入等价测试器。请说明如何仅用 O(n log n) 次等价测试器调用判定该问题的答案。

答案： 分治：把卡集分成两半（各 $\lfloor n/2 \rfloor$、$\lceil n/2 \rceil$ 张），递归求每一半中是否存在占多数的等价类，若有则返回该类的一张代表卡。整体若有超过 $n/2$ 张同类的卡，则其中至少有一半含有超过其自身半数的该类卡，故某个递归调用必返回该类代表卡；把返回的卡与其余所有卡逐一测试（$O(n)$ 次）即可确认是否真的超过 $n/2$ 张（若一侧只有局部多数则会被否定）。递归式 $T(n) \le 2T(n/2) + 2n$，解得 $T(n) = O(n \log n)$。（另有基于"两两抵消"的线性时间做法。）

习题 4

题目： 你一直在与一些物理学家合作，作为实验设计的一部分，他们需要研究大量非常小的带电粒子之间的相互作用。他们的装置大致如下：他们有一个惰性晶格结构，用它把带电粒子以规则间隔沿一条直线放置。于是可把他们的结构建模为实直线上的点集 $\{1, 2, 3, \ldots, n\}$；在每个点 j 处有一个电荷为 q_j 的粒子（电荷可正可负）。

他们想通过测量每个粒子所受的总力，并与计算预测比较来研究它。这个计算部分正需要你的帮助。由库仑定律，作用在粒子 j 上的总净力为

$$F_j = \sum_{ij} C\, q_i q_j / (j - i)^2$$

他们编写了如下简单程序来计算所有 j 的 F_j：

对 $j = 1, 2, \ldots, n$：　将 F_j 初始化为 0　对 $i = 1, 2, \ldots, n$：　若 $i < j$，则把 $C\, q_i q_j / (j - i)^2$ 加到 F_j　否则若 $i > j$，则把 $-C\, q_i q_j / (j - i)^2$ 加到 F_j　输出 F_j

分析该程序的运行时间并不难：内层循环的每次调用（对 i）耗时 $O(n)$，而内层循环总共被调用 $O(n)$ 次，故总运行时间为 $O(n^2)$。

问题在于，对他们所用的大 n，程序要运行好几分钟。另一方面，他们的实验装置经过优化，可以在几秒内放下 n 个粒子、完成测量并准备好处理接下来的 n 个粒子。因此他们非常希望能有办法更快地算出所有 F_j，以跟上实验的节奏。

请帮他们设计一个在 $O(n \log n)$ 时间内计算所有 F_j 的算法。

答案： 用卷积（FFT）。取向量 $a = (q_1, q_2, \ldots, q_n)$；取 b 使其第 t 项为 $1/t^2$（$t > 0$）、0（$t = 0$）、$-1/t^2$（$t < 0$），即 $b = (n^{-2}, (n-1)^{-2}, \ldots, 1/4, 1, 0, -1, -1/4, \ldots, -n^{-2})$。则卷积 $(a * b)$ 中对应 j 的项恰为 $\sum_{ij} q_i/(j-i)^2$，乘以 $C\, q_j$ 即得 F_j。FFT 计算卷积需 $O(n \log n)$ 时间，再由各卷积项重构 F_j 另需 $O(n)$。总时间 $O(n \log n)$。

习题 5

题目： **隐藏面消除**（hidden surface removal）是计算机图形学中一个几乎无需介绍的问题：当 Woody 站在 Buzz 前面时，你应能看到 Woody 而看不到 Buzz；当 Buzz 站在 Woody 前面时……好吧，你明白意思。

隐藏面消除的神奇之处在于，你往往能比直觉所想的更快地完成计算。下面是一个干净的几何例子，用以说明所能实现的基本加速。给定平面中 n 条非竖直直线，标为 $L_1, \ldots, L_n$，第 i 条直线由方程 $y = a_i x + b_i$ 给出。假设没有三条直线交于同一点。若直线 L_i 在 x 坐标 x_0 处的 y 坐标大于其它所有直线在该处的 y 坐标，即对所有 $j \ne i$ 有 $a_i x_0 + b_i > a_j x_0 + b_j$，则称 L_i 在 x_0 处是**最上方的**（uppermost）。若存在某个 x 坐标使 L_i 是最上方的，则称 L_i 是**可见的**（visible）——直观地说，若从 "$y = \infty$" 向下看，它总有一部分能被看到。

请给出一个算法，以 n 条直线为输入，在 $O(n \log n)$ 时间内返回所有可见直线。图 5.10 给出一个例子。

答案： 先按斜率递增排序。分治：对前一半、后一半分别递归求出各自的可见直线序列（按斜率递增）及各相邻可见直线的交点 x 坐标（这些 x 坐标递增）。合并时把两组的交点列表按 x 坐标归并（O(n)），线性扫描找出左组最上方直线被右组最上方直线超过的位置 ℓ，将两组序列在对应直线处拼接，并删去右侧不再最上方的直线。单层合并 O(n)，递归式 T(n) = 2T(n/2) + O(n)，故总时间 O(n log n)。

习题 6

题目： 考虑一棵 n 个结点的完全二叉树 T，其中 n = 2^d − 1 对某个 d 成立。T 的每个结点 v 被标上一个实数 x_v。可假设各结点上的实数互不相同。若标号 x_v 小于所有与 v 有边相连的结点 w 的标号 x_w，则称结点 v 是 T 的一个**局部最小值**（local minimum）。

给你这样一棵完全二叉树 T，但标号只以如下**隐式**（implicit）方式给出：对每个结点 v，你可以通过**探测**（probe）结点 v 来确定 x_v 的值。请说明如何仅用 O(log n) 次对 T 中结点的探测找出 T 的一个局部最小值。

答案： 从根 r 出发：探测 r 及其两个孩子；若 r 小于两个孩子，则 r 为局部最小值并返回。否则走向较小的那个孩子并重复。过程终止于某个小于两个孩子的结点 v（返回 v），或到达叶子 w（返回 w，因为它小于其父结点，而父结点是上一步选中的较小者）。每层只探测常数个结点，深度 $d = \log_2(n+1)$，共用 O(d) = O(log n) 次探测。

习题 7

题目： 现在假设给你一个 n × n 网格图 G。（n × n 网格图就是 n × n 棋盘的邻接图。更确切地说，它是一个图，其结点集为所有有序自然数对 (i, j)，其中 $1 \le i \le n$、$1 \le j \le n$；结点 (i, j) 与 (k, ℓ) 之间有边当且仅当 $|i - k| + |j - \ell| = 1$。）

我们沿用上一题的部分术语。同样，每个结点 v 标有一个实数 x_v，可假设所有标号互不相同。请说明如何仅用 O(n) 次对 G 中结点的探测找出 G 的一个局部最小值。（注意 G 有 n^2 个结点。）

答案： 分治。在当前 n × n 子网格中取中间一行与中间一列的并集 S，令 T 为 S 的邻居集（$|S \cup T| = O(n)$），探测 S ∪ T 中最小结点 u。 - 若 u ∈ S，则 u 的所有邻居都在 S ∪ T 中且都比它大，u 即局部最小值，返回； - 若 u ∈ T，则 u 小于其全部邻居（含相邻子网格边界上的结点），故可向 u 所在的那一侧递归到一块 ⌊n/2⌋ × ⌊n/2⌋ 子网格，其中的局部最小值也是整个网格的局部最小值。

每层探测 O(n) 个结点、规模减半，递归式 T(n) = T(n/2) + O(n)，解得 T(n) = O(n) 次探测（网格共 n^2 个结点）。

第6章 动态规划

习题 1

题目： 设 G = (V, E) 是含 n 个节点的无向图。回忆：若节点子集中任意两点之间都没有边，则称该子集为**独立集**（independent set）。一般图中寻找大独立集是困难的，但这里我们会看到，如果图"足够简单"，就可以高效地做到。

称图 G = (V, E) 为一条**路径**（path），若它的节点可写成 $v_1, v_2, \ldots, v_n$，且 v_i 与 v_j 之间有边当且仅当下标 i 与 j 恰好相差 1。给每个节点 v_i 关联一个正整数**权** w_i。例如图 6.28 中的五节点路径，权就是画在节点内的数字。本题的目标是求解下述问题：

在路径 G 中找一个总权尽可能大的独立集。

(a) 举例说明下述算法**并不**总能找到总权最大的独立集： 「最重优先」贪心算法——令 S 为空集；当 G 中还有节点时，选取权最大的节点 v_i，把 v_i 加入 S，并从 G 中删除 v_i 及其所有邻居；循环结束后返回 S。

(b) 举例说明下述算法同样**并不**总能找到总权最大的独立集： 令 S_1 为所有下标 i 为奇数的 v_i 构成的集合，S_2 为所有下标 i 为偶数的 v_i 构成的集合（注意 S_1、S_2 都是独立集）；比较 S_1 与 S_2 哪个总权更大，返回较大的那个。

(c) 给出一个算法，输入为带权的 n 节点路径 G，返回总权最大的独立集，运行时间应是 n 的多项式，且与权值的大小无关。

答案： (a) 权序列 2, 3, 2：贪心先选中间权为 3 的节点，而最优解是首尾两个节点（总权 4）。(b) 权序列 3, 1, 2, 3：算法取 $\{v_1, v_3\}$（总权 5）或 $\{v_2, v_4\}$（总权 4），而最优解是 $\{v_1, v_4\}$（总权 6）。(c) 令 X_i 为前 i 个节点上的最优值，$X_0 = 0$、$X_1 = w_1$，转移为 $X_i = \max(X_{i-1}, w_i + X_{i-2})$。按 i 递增计算，时间 O(n)，再由 max 的选择回溯出独立集。

习题 2

题目： 假设你管理一支由顶尖计算机高手组成的咨询团队，每周要为他们挑一项工作。可选的差事分为**低压力**（low-stress，如给本地小学的班级搭个网站）与**高压力**（high-stress，如保护国家最珍贵的机密，或帮一群绝望的康奈尔学生完成某个与编译器有关的项目）两类。每周的基本问题就是：接低压力还是高压力。

若在第 i 周为团队选低压力工作，可得收入 $\ell_i > 0$ 美元；选高压力工作则得 $h_i > 0$ 美元。但有个条件：要接第 i 周的高压力工作，团队在第 i−1 周必须不接任何工作（无论哪类），因为需要整整一周的准备时间来进入状态。另一方面，即使在第 i−1 周做过工作（无论哪类），第 i 周接低压力工作也没问题。

于是给定 n 周的一个序列，一个**计划**就是为这 n 周各选「低压力」「高压力」或「不做」，并满足：若第 i 周（i > 1）选「高压力」，则第 i−1 周必须选「不做」（第 1 周可以直接选高压力）。计划的**价值**按自然方式确定：对每个 i，若第 i 周选「低压力」就把 ℓ_i 累加进去，若选「高压力」就把 h_i 累加进去（选「不做」则加 0）。

问题。 给定 $\ell_1, \ell_2, \ldots, \ell_n$ 与 $h_1, h_2, \ldots, h_n$，求一个价值最大的计划（这样的计划称为**最优**的）。

例子。 设 n = 4，ℓ_i 与 h_i 如下表。则价值最大的计划是：第 1 周「不做」，第 2 周高压力，第 3、4 周低压力，价值为 0 + 50 + 10 + 10 = 70。

	第 1 周	第 2 周	第 3 周	第 4 周
ℓ	10	1	10	10
h	5	50	5	1

(a) 举例说明下述算法不能正确求解本题（给出它返回错误答案的实例）：对 i = 1 到 n，若 $h_{i+1} > \ell_i + \ell_{i+1}$，则输出「第 i 周不做」及「第 i+1 周做高压力」，转去处理 i+2；否则输出「第 i 周做低压力」，转去处理 i+1。（为避免数组越界，定义当 i > n 时 $h_i = \ell_i = 0$。）在你的例子中要说明正确答案是什么、上述算法得到什么。

(b) 给出一个高效算法，输入 $\ell_1, \ldots, \ell_n$ 与 $h_1, \ldots, h_n$，返回最优计划的**价值**。

答案： (a) 取 3 周，$\ell = (2, 2, 2)$，$h = (1, 5, 10)$：算法在第 1 周因 $h_2 = 5 > \ell_1 + \ell_2 = 4$ 而选「不做、第 2 周高压力」，再加第 3 周低压力，得 0 + 5 + 2 = 7；最优为低压力、不做、高压力，价值 2 + 0 + 10 = 12。(b) 令 OPT(i) 为前 i 周的最大价值，则 $\mathrm{OPT}(i) = \max(\ell_i + \mathrm{OPT}(i-1), h_i + \mathrm{OPT}(i-2))$，初值 OPT(0) = 0、$\mathrm{OPT}(1) = \max(\ell_1, h_1)$；时间 O(n)，回溯可得计划。

习题 3

题目： 设 G = (V, E) 是节点为 $v_1, \ldots, v_n$ 的有向图。称 G 为**有序图**（ordered graph），若它具有以下性质：

(i) 每条边都从下标较小的节点指向下标较大的节点，即每条有向边形如 (v_i, v_j) 且 i < j。(ii) 除 v_n 外每个节点至少有一条出边，即对每个 v_i（i = 1, 2, …, n−1），至少存在一条形如 (v_i, v_j) 的边。

路径的**长度**指其中边的数目。本题的目标是求解下述问题（例子见图 6.29）：

给定有序图 G，求从 v_1 开始、到 v_n 结束的最长路径的长度。

(a) 举例说明下述算法不能正确求解本题（给出一个它返回错误答案的有序图）：置 $w = v_1$、L = 0；当节点 w 还有出边时，选出边 (w, v_j) 中 j 最小的一条，置 $w = v_j$，把 L 加 1；循环结束后把 L 作为最长路径长度返回。在你的例子中要说明正确答案以及上述算法得到的结果。

(b) 给出一个高效算法，输入有序图 G，返回从 v_1 开始、到 v_n 结束的最长路径的**长度**（同样，路径长度指边数）。

答案： (a) 取节点 $v_1, \ldots, v_5$，边为 (v_1, v_2), (v_1, v_3), (v_2, v_5), (v_3, v_4), (v_4, v_5)。算法沿 (v_1, v_2), (v_2, v_5) 返回 2，而最优为 (v_1, v_3), (v_3, v_4), (v_4, v_5)，长度为 3。(b) 令 OPT[i] 为 v_1 到 v_i 的最长路径长度，OPT[1] = 0，$\mathrm{OPT}[i] = \max\{\mathrm{OPT}[j] + 1 : (v_j, v_i) \in E\}$（不可达时取 $-\infty$）。按 i 递增计算，时间 $O(n^2)$（按边计为 O(m)），回溯得路径。

习题 4

题目： 假设你经营一家轻量级咨询公司——只有你、两位合伙人和一些租来的设备。客户分布在东海岸与西海岸，于是有了下面的问题。

每个月你可以在纽约（NY）的办公室或旧金山（SF）的办公室开展业务。第 i 个月若在 NY 办公，会产生运营成本 N_i；若在 SF 办公，会产生运营成本 S_i（取决于该月客户需求的分布）。

然而，若第 i 个月在某城市办公、第 i+1 个月换到另一城市，则要支付固定的**搬迁成本** M。

给定 n 个月的序列，一个**计划**是一个城市序列——每项为 NY 或 SF——其中第 i 项表示第 i 个月所在的城市。计划的**成本**等于 n 个月的运营成本之和，再加上每次换城市时的搬迁成本 M。计划可以从任一城市开始。

问题。 给定搬迁成本 M 的值以及运营成本序列 $N_1, \dots, N_n$ 与 $S_1, \dots, S_n$，求成本最小的计划（这样的计划称为**最优**的）。

例子。 设 n = 4，M = 10，运营成本如下表。则成本最小的计划是 [NY, NY, SF, SF]，总成本 1 + 3 + 2 + 4 + 10 = 20，最后的 10 来自一次换址。

	第 1 月	第 2 月	第 3 月	第 4 月
NY	1	3	20	30
SF	50	20	2	4

(a) 举例说明下述算法不能正确求解本题：对 i = 1 到 n，若 $N_i < S_i$ 则输出「第 i 月在 NY」，否则输出「第 i 月在 SF」。在你的例子中说明正确答案与算法得到的结果。

(b) 给出一个实例，使得**每一个**最优计划都必须搬迁（即改变所在地）至少三次，并简要说明该实例为何具有此性质。

(c) 给出一个高效算法，输入 n、M 以及运营成本序列 $N_1, \dots, N_n$ 与 $S_1, \dots, S_n$，返回最优计划的成本。

答案： (a) 取 M = 10，N = (1, 4, 1)，S = (20, 1, 20)：贪心得 [NY, SF, NY]，成本 1 + 1 + 1 + 2·10 = 23，而最优 [NY, NY, NY] 成本为 6。(b) 取 M = 10，N = (1, 100, 1, 100)，S = (100, 1, 100, 1)：计划 [NY, SF, NY, SF] 成本为 1+1+1+3·10 = 34，而任何少搬一次的方案在某月至少付 100，故不最优。(c) 令 $\mathrm{OPT_N}(i)$、$\mathrm{OPT_S}(i)$ 分别为前 i 个月且第 i 月在 NY / SF 的最小成本，则 $\mathrm{OPT_N}(i) = N_i + \min(\mathrm{OPT_N}(i-1), M + \mathrm{OPT_S}(i-1))$，$\mathrm{OPT_S}(i) = S_i + \min(\mathrm{OPT_S}(i-1), M + \mathrm{OPT_N}(i-1))$，$\mathrm{OPT_N}(0) = \mathrm{OPT_S}(0) = 0$；答案为 $\min(\mathrm{OPT_N}(n), \mathrm{OPT_S}(n))$，时间 O(n)。

习题 5

题目： 有些语言（包括中文和日文）书写时词与词之间不留空格。因此处理这类文本的软件必须解决**词分割**（word segmentation）问题——推断文本中相邻词之间的可能边界。若英语也不写空格，对应的问题就是：给定串 "meetateight"，判定最好的分割是 "meet at eight"（而非 "me et at eight"、"meet ate ight"，或许多更不合理的备选）。如何自动完成这一过程？

一种简单而较为有效的做法是：寻找使各个组成词的累计「质量」最大的分割。于是假设你有一个黑箱：对任意串 $x = x_1x_2\cdots x_k$ 返回一个数 quality(x)。该数可正可负，数值越大表示越像合理的英文词（例如 quality("me") 为正，quality("ght") 为负）。

给定长串 $y = y_1y_2\cdots y_n$，y 的一个**分割**就是把它的字母划分成若干连续块，每块对应分割中的一个词。分割的**总质量**等于各块质量之和。因此只要 quality("meet") + quality("at") + quality("eight") 大于该串任何其他分割的总质量，我们就会得到上面那个正确答案。

给出一个高效算法，输入串 y，计算总质量最大的分割（可把对 quality(x) 的一次调用视为一步计算）。

（一个与解题无关的附注：实际系统会使用更复杂的表述以提高效果，例如还要求分割在短语和句子层面连贯。但即便加上这些额外准则与约束，动态规划仍是许多成功分词系统的核心。）

答案： 令 Opt(i) 为 y 前 i 个字符的最优分割得分，Opt(0) = 0，则 Opt(i) = max_{1 ≤ j ≤ i}{Opt(j−1) + quality(y_j⋯y_i)}。答案为 Opt(n)，时间 O(n²)（即 O(n²) 次黑箱调用）。

习题 6

题目： 在文字处理软件中，「漂亮打印」（pretty printing）的目标是把右边界参差不齐的文本（如下）

```
Call me Ishmael.
Some years ago,
never mind how long precisely,
having little or no money in my purse,
and nothing particular to interest me on shore,
I thought I would sail about a little
and see the watery part of the world.
```

排成右边界尽可能「齐」的文本。

为把这一点说得足够精确，设文本由词序列 W = {w_1, w_2, …, w_n} 组成，w_i 含 c_i 个字符。最大行宽为 L。假定使用定宽字体，并忽略标点与连字符问题。

W 的一个**排版**（formatting）是把 W 中的词划分成若干**行**。分到同一行的词之间，除最后一个词外每个词后要有一个空格；因此若 w_i, w_{i+1}, …, w_k 被分到一行，则应有

|∑_{t=i}^{k−1}(c_t + 1) + c_k| ≤ L。

若某行满足该不等式，则称把词分配到该行是**有效的**（valid）。不等式左端与右端之差称为该行的 **slack**——即右边界处剩下的空格数。

给出一个高效算法，把词集 W 划分成有效行，使所有行（**包括最后一行**）的 slack 的**平方和**最小。

答案： 令 S_{i,j} 为把 w_i, …, w_j 放在一行时的 slack（超过 L 时记为 ∞）。令 OPT[k] 为前 k 个词的最小代价，OPT[0] = 0，则 OPT[k] = min_{1 ≤ j ≤ k}(S_{j,k}² + OPT[j−1])。先用 O(n²) 算出所有 S_{i,j}，再 O(n²) 求 DP，回溯可得换行方案；总时间 O(n²)，空间 O(n²)（S 可省为 O(n²) 或边算边用）。

习题 7

题目： 在第 5 章的一个已解习题中，我们给出了下述问题的 O(n log n) 算法。考察连续 n 天某股票的收盘价，记为 i = 1, 2, …, n。第 i 天每股价格为 p(i)（为简单起见假定当天价格固定）。我们想知道：若目标是最大化每股利润 p(j) − p(i)，应选哪一天 i 买入、并在其后的某天 j > i 卖出？（若这 n 天内无论如何都赚不到钱，则应得出这一结论。）

在那个已解习题中，我们说明了如何在 O(n log n) 时间内找出最优的 i 与 j。但事实上可以做得更好：请给出在 O(n) 时间内找出最优 i 与 j 的方法。

答案： 令 X_j 为「第 j 天卖出」可获得的最大收益，X_1 = 0，则 X_j = max(0, X_{j−1} + (p(j) − p(j−1)))。答案为 max_{j=1..n} X_j（若最大值为 0 则说明无利可图），时间 O(n)。

习题 8

题目： 地下城 Zion 的居民靠功夫、重炮与高效算法的结合抵御外敌。近来他们开始关注能帮助击退机器人蜂群的自动化方法。

一次机器人进攻的情形如下：

- 一 swarm 机器人在 n 秒内陆续到达；第 i 秒到达 x_i 个机器人。根据遥感数据，你事先已知序列 $x_1, x_2, \ldots, x_n$。
- 你掌握一种**电磁脉冲**（EMP），可在机器人到达时摧毁其中一部分；其威力取决于已充电多久。确切地说，存在函数 f(·)：若距上次使用 EMP 已过 j 秒，则它最多能摧毁 f(j) 个机器人。
- 因此具体地，若在第 k 秒使用、且距上次使用已 j 秒，则它将摧毁 min(x_k, f(j)) 个机器人（用后即完全放电）。
- 还假定 EMP 初始完全放电，因此若首次在第 j 秒使用，则最多摧毁 f(j) 个机器人。

问题。 给定机器人到达数据 $x_1, x_2, \ldots, x_n$ 与充电函数 f(·)，选择激活 EMP 的时刻，使摧毁的机器人总数尽可能多。

例子。 设 n = 4，x_i 与 f(i) 如下表。最优方案是在第 3 秒和第 4 秒激活 EMP：第 3 秒时 EMP 已充电 3 秒，摧毁 min(10, 4) = 4 个；第 4 秒时距上次使用只过了 1 秒，摧毁 min(1, 1) = 1 个，共 5 个。

i	1	2	3	4
x_i	1	10	10	1
f(i)	1	2	4	8

(a) 举例说明下述算法不能正确求解本题：令 j 为使 f(j) ≥ x_n 的最小者（若不存在则取 j = n）；在第 n 秒激活 EMP；若 n − j ≥ 1，则对输入 $x_1, \ldots, x_{n-j}$ 递归调用 Schedule-EMP。在你的例子中说明正确答案与算法得到的结果。

(b) 给出一个高效算法，输入机器人到达数据 $x_1, \ldots, x_n$ 与充电函数 f(·)，返回一串 EMP 激活所能摧毁的最大机器人数。

答案： (a) 把上例中的 x_4 改为 2：算法先在第 4 秒激活（j = 2，因 f(2) = 2 ≥ x_4），再对 x_1, x_2 递归并在第 2 秒激活，共摧毁 2 + 2 = 4；而仍在第 3、4 秒激活可得 min(10,4) + min(2,1) = 5。(b) 令 OPT(j) 为只考虑 $x_1, \ldots, x_j$ 时的最大摧毁数，最后一次激活取在 i（0 ≤ i < j），则 $OPT(j) = \max_{0 \le i < j}[OPT(i) + \min(x_j, f(j-i))]$，OPT(0) = 0；答案为 OPT(n)，$O(n^2)$。

习题 9

题目： 你在帮助运营一套每天可处理数 TB 数据的高性能计算系统。在 n 天中，每天都会送来一批数据；第 i 天送来 x_i TB。每处理 1 TB 获得固定收入，但未处理的数据在当天结束时即不可用（之后再也无法处理）。

你无法每天都处理全部数据，因为系统能力有限：某天只能处理固定数量的 TB。事实上系统运行着一套精密却不太可靠的软件，因此自最近一次重启以来，经过的天数越多，能处理的数据量越少。重启后的第 1 天可处理 s_1 TB，第 2 天可处理 s_2 TB，依此类推直到 s_n；假定 $s_1 > s_2 > \cdots > s_n > 0$。（当然，第 i 天无论系统多快，也只能处理至多 x_i TB。）为让系统恢复峰值性能可以选择重启；但选择重启的那一天无法处理任何数据。

问题。 给定未来 n 天的可用数据量 $x_1, x_2, \ldots, x_n$ 以及系统性能曲线 $s_1, s_2, \ldots, s_n$（第 1 天为刚重启状态），选择在哪些天重启，使处理的数据总量最大。

例子。 设 n = 4，x_i 与 s_i 如下表。最优方案是只在第 2 天重启：第 1 天处理 8 TB，第 2 天为 0，第 3 天处理 7 TB，第 4 天处理 4 TB，共 19。（若不重启则处理 8 + 1 + 2 + 1 = 12；其他重启策略也都少于 19。）

	第 1 天	第 2 天	第 3 天	第 4 天
x	10	1	7	7
s	8	4	2	1

(a) 给出一个满足以下性质的实例：存在数据「过剩」，即对每个 i 都有 $x_i > s_i$；且最优解至少重启两次。除实例外还要说明最优解是什么（无需证明其最优性）。

(b) 给出一个高效算法，输入 $x_1, \ldots, x_n$ 与 $s_1, \ldots, s_n$，返回最优方案所处理的 TB 总数。

答案： (a) 取 $s_1 = 10$、$s_i = 1$（$i > 1$），$x_i = 11$（所有 i）：最优是隔天重启，每两天处理 10 TB，共约 5n TB（重启次数约为 $n/2 \geq 2$）。(b) 令 Opt(i, j) 为从第 i 天到第 n 天、距上次重启已 j 天时的最大处理量，则 Opt(i, j) = max(Opt(i+1, 1), min(x_i, s_j) + Opt(i+1, j+1))，边界 Opt(n, j) = min(x_n, s_j)；答案为 Opt(1, 1)，$O(n^2)$ 时间与空间。

习题 10

题目： 你要运行一个大型计算作业，用它尽可能多地模拟一个物理系统的离散**步骤**。所在实验室有两台大型超级计算机（记为 A 和 B）可处理该作业。但你不是这两台机器的高优先级用户，因此任一时刻只能使用其空闲周期。

你面临的问题如下。作业在任一给定分钟内只能跑在**一台**机器上。在接下来的 n 分钟里，你有两台机器各自可用处理能力的「轮廓」：第 i 分钟若作业在机器 A 上，可跑 $a_i > 0$ 步；若在机器 B 上，可跑 $b_i > 0$ 步。你也可以把作业从一台机器搬到另一台，但这要花一分钟，且这一分钟内作业不做任何处理。

于是给定 n 分钟的序列，一个**计划**就是每分钟选 A、B 或 **move**，并满足：A 与 B 不能出现在相邻的分钟里。例如，若第 i 分钟作业在 A 上而想换到 B，则第 i+1 分钟的选择必须是 move，第 i+2 分钟才能选 B。计划的**价值**是 n 分钟内执行的总步数：即作业在 A 上的所有分钟的 a_i 之和，加上在 B 上的所有分钟的 b_i 之和。

问题。 给定 $a_1, a_2, \ldots, a_n$ 与 $b_1, b_2, \ldots, b_n$，求价值最大的计划（这样的方案称为**最优**的）。注意计划在第 1 分钟可以从 A 或 B 开始。

例子。 设 n = 4，a_i 与 b_i 如下表。则价值最大的计划是：第 1 分钟选 A，第 2 分钟 move，第 3、4 分钟选 B，价值为 10 + 0 + 20 + 20 = 50。

	第 1 分钟	第 2 分钟	第 3 分钟	第 4 分钟
A	10	1	1	10
B	5	1	20	20

(a) 举例说明下述算法不能正确求解本题：第 1 分钟选 a_1、b_1 中较大者对应的机器，置 i = 2；当 $i \leq n$ 时，考察第 i−1 分钟的选择。若为 A：如果 b_{i+1} > a_i + a_{i+1}，则第 i 分钟选 move、第 i+1 分钟选 B，转

到 i+2；否则第 i 分钟选 A，转到 i+1。若为 B：把上述规则中 A 与 B 的角色互换后同样处理。在你的例子中说明正确答案与算法得到的结果。

(b) 给出一个高效算法，输入 $a_1, \ldots, a_n$ 与 $b_1, \ldots, b_n$，返回最优计划的价值。

答案： (a) 两个反例：n = 2 时 A = (2, 10)、B = (1, 20)，贪心第 1 分钟选 A，第 2 分钟继续 A，得 12；最优为 B、B，得 21。n = 4 时 A = (2, 1, 1, 200)、B = (1, 1, 20, 100)，贪心得 A、move、B、B = 2+0+20+100 = 122；最优为全程 A = 2+1+1+200 = 204。(b) 令 Opt(i, A)、Opt(i, B) 为前 i 分钟且第 i 分钟在 A / B 上的最大价值，则 $\mathrm{Opt}(i, A) = a_i + \max(\mathrm{Opt}(i-1, A), \mathrm{Opt}(i-2, B))$，$\mathrm{Opt}(i, B) = b_i + \max(\mathrm{Opt}(i-1, B), \mathrm{Opt}(i-2, A))$，初值 $\mathrm{Opt}(1, A) = a_1$、$\mathrm{Opt}(1, B) = b_1$；时间 $O(n)$。

习题 11

题目： 假设你为一家生产 PC 设备并运往各地经销商的公司做咨询。未来 n 周中，每周他们有预计的**供应量** s_i（以磅计），需由空运承运商运走。

每周的供应可由两家空运公司 A 或 B 之一承运：

- 公司 A 按每磅固定费率 r 收费（故运一周的供应 s_i 花费 $r \cdot s_i$）。
- 公司 B 每周收取固定金额 c 的合同费，与重量无关。但与 B 的合同必须以**连续四周**为一个块签订。

PC 公司的一个**日程**（schedule）是为 n 周各选一家空运公司（A 或 B），并满足：公司 B 一旦被选用，就必须按连续四周的块选用。日程的**成本**是按上述规定付给 A 与 B 的总额。

给出一个多项式时间算法，输入供应量序列 $s_1, s_2, \ldots, s_n$，返回成本最小的**日程**。

例子。 设 r = 1，c = 10，供应量为 11, 9, 9, 12, 12, 12, 12, 9, 9, 11。则最优日程是：前 3 周选 A，接着连续 4 周选 B，最后 3 周选 A。

答案： 令 OPT(i) 为前 i 周的最小成本，$\mathrm{OPT}(i) = \min(r \cdot s_i + \mathrm{OPT}(i-1), 4c + \mathrm{OPT}(i-4))$，并令 $i \le 0$ 时 OPT(i) = 0；答案为 OPT(n)，时间 $O(n)$，回溯即得日程。

习题 12

题目： 假设要在 n 台服务器 $S_1, S_2, \ldots, S_n$ 上复制一个文件。在服务器 S_i 放一份副本的**放置成本**为整数 $c_i > 0$。

若用户向 S_i 请求该文件而 S_i 上没有副本，则依次查找 S_{i+1}, S_{i+2}, S_{i+3}, … 直到找到一份副本；设副本在 S_j（j > i），则产生**访问成本** j − i。（注意下标更小的服务器 S_{i−1}, S_{i−2}, … 不在本次查找范围内。）若 S_i 上有副本则访问成本为 0。我们要求必须在 S_n 上放一份副本，从而所有查找最迟在 S_n 处终止。

我们希望放置副本使放置成本与访问成本之和最小。形式化地，一个**配置**（configuration）是对每台服务器 S_i（i = 1, 2, …, n−1）决定是否在 S_i 放副本（注意 S_n 上总是放一份）。配置的**总成本**等于所有放有副本的服务器的放置成本之和，加上与全部 n 台服务器相关的访问成本之和。

给出一个多项式时间算法，求总成本最小的配置。

答案： 令 OPT(j) 为「在 S_j 放副本」时前 j 台服务器的最小成本。若上一个副本放在 S_i（$0 \le i < j$，S_0 视为无副本），则 S_i 之后到 S_j 之间的访问成本为 $0 + 1 + \cdots + (j-i-1) = C(j-i, 2)$。故 $\mathrm{OPT}(j) = c_j + \min_{0 \le i < j}(\mathrm{OPT}(i) + C(j-i, 2))$，其中 OPT(0) = 0、C(1, 2) = 0；答案为 OPT(n)，时间 $O(n^2)$。

习题 13

题目： 图中找环的问题在金融交易应用中自然出现。考虑一家交易 n 家不同公司股票的公司。对每一对 $i \neq j$，他们维护一个交易比率 r_{ij}，表示一股 i 可换 r_{ij} 股 j。这里比率允许为分数，即 $r_{ij} = 3/2$ 表示可用 3 股 i 换 2 股 j。

对股票序列 $i_1, i_2, \ldots, i_k$，一个**交易环**（trading cycle）是指依次把公司 i_1 的股票换成 i_2 的、再把 i_2 的换成 i_3 的……最后把 i_k 的换回 i_1 的股票。绕这样一圈后，你最终持有与开始时相同公司 i_1 的股票。绕环交易通常是坏主意，因为往往股数会变少。但偶尔在短时间内存在增加股数的机会：若沿环交易使股数增加，则称该环为**机会环**（opportunity cycle）。这恰好发生在环上各比率之积大于 1 时。在分析市场状态时，从事交易的公司希望知道是否存在机会环。

给出一个多项式时间算法，若存在这样的机会环则找出它。

答案： 取对数把乘积化为和：建图，每个公司一个节点，对每对股票加有向边 (i, j)，边权为 $-\log r_{ij}$。交易环 C 是机会环 $\Leftrightarrow \prod_{(i,j)\in C} r_{ij} > 1 \Leftrightarrow \sum \log r_{ij} > 0 \Leftrightarrow \sum(-\log r_{ij}) < 0$，即 C 是**负环**。故用 Bellman-Ford 做负环检测即可（时间 $O(nm)$，稠密图 $O(n^3)$），检测到的负环即所需机会环。

习题 14

题目： 大量移动无线设备会自然构成一个网络：设备是节点，两台设备 x 与 y 若能直接通信（例如通过短距无线电链路）则之间有一条边。这种无线设备网络是高度动态的对象，随着设备移动，边会随时间出现和消失。例如当 x 与 y 彼此远离、失去直接通信能力时，边 (x, y) 就会消失。

在随时间变化的网络中，自然地要寻找高效方式来**维护**某些指定节点之间的路径。这里有两个相互冲突的关切：我们希望路径短，但也不希望在网络结构每次变化时都频繁改动路径（也就是说，如果可能，希望同一条路径即使在网络增删边的情况下也继续可用）。下面给出该问题的一种建模方式。

设有移动节点集合 V，在某一时刻节点之间存在边集 E_0。随着节点移动，边集依次变为 $E_1, E_2, \ldots$，直到 E_b。对 $t = 0, 1, 2, \ldots, b$，令 G_t 表示图 (V, E_t)。于是若把节点 V 上的网络结构看作一段「延时摄影」，它恰好就是图序列 $G_0, G_1, G_2, \ldots, G_{b-1}, G_b$。假定每个图 G_t 都是连通的。

现考虑两个特定节点 $s, t \in V$。对某个图 G_i 中的 s-t 路径 P，定义 P 的**长度**为 P 中边的数目，记为 $\ell(P)$。目标是给出一列路径 $P_0, P_1, \ldots, P_b$，使对每个 i，P_i 是 G_i 中的 s-t 路径。我们希望路径相对较短，也不希望**改变**（changes）太多——改变指路径身份发生切换的那些点。形式化地，定义 $\mathrm{changes}(P_0, P_1, \ldots, P_b)$ 为使 $P_i \neq P_{i+1}$ 的下标 i（$0 \le i \le b-1$）的个数。

固定常数 $K > 0$。定义路径序列 $P_0, P_1, \ldots, P_b$ 的**成本**为

$$\mathrm{cost}(P_0, P_1, \ldots, P_b) = \sum_{i=0}^{b} \ell(P_i) + K \cdot \mathrm{changes}(P_0, P_1, \ldots, P_b)。$$

(a) 假设可以选一条单一路径 P，它在每个图 $G_0, G_1, \ldots, G_b$ 中都是 s-t 路径。给出多项式时间算法找出最短的这样的路径。

(b) 给出多项式时间算法，求成本最小的路径序列 $P_0, P_1, \ldots, P_b$，其中 P_i 是 G_i 中的 s-t 路径（$i = 0, 1, \ldots, b$）。

答案： (a) 取所有图 $G_0, \ldots, G_b$ 的公共边构成图 H，在 H 上用 BFS 求 s-t 最短路（不存在则无这样的路径）。(b) 令 $G(i, j)$ 为 $G_i, \ldots, G_j$ 的公共边构成的图，$\ell(i, j)$ 为其上 s-t 最短路长度（不存在为 ∞）。按「最后一次换路」分解：$\mathrm{Opt}(b) = \min((b+1)\cdot\ell(0, b), \min_{1 \le i \le b}[\mathrm{Opt}(i-1) + (b-i+1)\cdot\ell(i, b) + K])$，即换路

后从 G_i 到 G_b 共用同一条路径。先对所有 $0 \le i \le j \le b$ 用 BFS 预计算 $\ell(i, j)$（$O(b^2)$ 对、总 $O(n^2b^3)$，可优化到 $O(b^2n^2)$），再 $O(b^2)$ 求 DP，整体多项式时间。

习题 15

题目： 在大多数晴朗的夜晚，你在天文系的一群朋友会聚在一起规划当晚要观测的天文事件。对事件作如下假设。

- 共有 n 个事件，为简单起见假定它们依次相隔恰好一分钟发生；事件 j 在第 j 分钟发生；若不在第 j 分钟准时观测，他们就会错过它。
- 天空按一维坐标系统（以距某中心基线的度数计）标定；事件 j 位于坐标 d_j，其中 d_j 为某个整数值。望远镜在第 0 分钟位于坐标 0。
- 最后一个事件 n 比其他事件重要得多，因此要求他们必须观测事件 n。

天文系运营着一台大型望远镜，可用于观测这些事件。由于仪器复杂，它每分钟至多只能移动一度的距离。因此他们不指望能观测全部 n 个事件，只希望在望远镜运行限制与「必须观测事件 n」的约束下，观测尽可能多的事件。

称事件子集 S 是**可观测的**（viewable），若能在各自时刻 j 观测 S 中每个事件，且望远镜（以每分钟至多一度的速度）有足够时间在 S 中相邻事件之间移动。

问题。 给定 n 个事件的坐标，求一个规模最大的可观测子集，并要求它包含事件 n。这样的解称为**最优**的。

例子。 设事件 1~9 的一维坐标依次为 1, −4, −1, 4, 5, −4, 6, 7, −2。则最优解是观测事件 1, 3, 6, 9（可以验证，即使以每分钟一度的速度，望远镜也有时间在该子集的相邻事件间移动）。

(a) 举例说明下述算法不能正确求解本题：把所有满足 $|d_n - d_j| > n - j$ 的事件 j 标为非法（观测它会妨碍观测事件 n），其余标为合法；把当前位置初始化为第 0 分钟时的坐标 0；当事件序列尚未结束时，找出能到达的最早的合法事件 j（不超出望远镜最大移动速率），把 j 加入 S；把当前位置更新为第 j 分钟时的坐标 d_j。最后输出 S。在你的例子中说明正确答案与算法得到的结果。

(b) 给出一个高效算法，输入事件坐标 $d_1, d_2, \ldots, d_n$，返回最优解的规模。

答案： (a) 取 n = 6，坐标 3, 2, −3, −2, −1, 0：贪心依次观测事件 2, 5, 6（共 3 个）；最优为 3, 4, 5, 6（共 4 个）。(b) 令 OPT(j) 为「观测事件 j」前提下最多能观测的事件数，则 $\mathrm{OPT}(j) = 1 + \max\{\mathrm{OPT}(i) : i < j \text{ 且 } |d_j - d_i| \le j - i\}$（若 $|d_j| > j$ 则事件 j 从起点不可达，记为 $-\infty$）；答案为 OPT(n)，时间 $O(n^2)$，回溯得子集。

习题 16

题目： 纽约州伊萨卡（Ithaca）的晴天很多，但今年康奈尔大学春季 ROTC 野餐偏赶上雨天。负责的军官决定推迟野餐，并且必须打电话通知所有人，她采用的机制如下。

校园里每个 ROTC 人员只向唯一的一位上级军官（superior officer）汇报。因此汇报层级可描述为一棵以负责军官为根的树 T，其中每个节点 v 有一个父节点 u，即他/她的上级；反过来称 v 是 u 的直接下属（direct subordinate）。图 6.30 中 A 是负责军官，B、D 是 A 的直接下属，C 是 B 的直接下属。

为通知推迟，负责军官先一次一个地给她每个直接下属打电话。某个直接下属一接到电话，就应当尽快一次一个地给他/她自己的直接下属打电话。如此继续，直到所有人都被通知。注意在此过程中每个人只能给直接下

属打电话；例如图 6.30 中 A 不允许直接打给 C。

我们可以把这一过程划分为若干轮（rounds）。在一轮中，每个已经知道推迟消息的人可以给他/她的一个直接下属打电话。所有人都被通知所需的轮数取决于每个人给其直接下属打电话的顺序。例如图 6.30 中，若 A 先打给 B 只需两轮；若 A 先打给 D 则需要三轮。

给出一个高效算法，确定通知所有人所需的最少轮数，并输出一个达到该最少轮数的电话呼叫序列。

答案： 与贪心章节中三项赛排程（triathlon scheduling）同理：子树必须逐个"启动"，启动后各自递归并行。设 x(T′) 为根已知消息后通知完子树 T′ 所有人所需轮数；把 T′ 的子节点按 x 降序排为 T_1,…,T_k，则 x(T′) = min_j { j + x(T_j) }（j 从 1 开始，即第 j 个被叫），叶子节点 x = 0。 自底向上计算，每个子树按其子树的 x 值排序，时间 O(d′ log d′)，总时间 O(n log n)；记录排序顺序回溯即可得呼叫序列。

习题 17

题目： 你的朋友们一直在研究科技股的收盘价，寻找有趣的模式。他们定义了所谓上升趋势（rising trend）如下。

他们记录了某只股票连续 n 天的收盘价，记为 P[1], P[2], …, P[n]。这些价格中的一个上升趋势是价格的一个子序列 P[i_1], P[i_2], …, P[i_k]，对应天数 i_1 < i_2 < ⋯ < i_k，满足

- i_1 = 1，且
- 对每个 j = 1, 2, …, k − 1 有 P[i_j] < P[i_{j+1}]。

因此上升趋势是始于第一天的子序列（不必连续），且价格在该子序列上严格递增。

他们想找出给定价格序列中最长的上升趋势。

例：设 n = 7，价格序列为

10, 1, 2, 11, 3, 4, 12。

则最长上升趋势由第 1, 4, 7 天的价格给出。注意第 2, 3, 5, 6 天的价格虽递增，但该子序列不始于第 1 天，不符合上升趋势的定义。

(a) 证明下列算法不能正确返回最长上升趋势的长度，给出一个使它得不到正确答案的实例。

```
令 i = 1
  L = 1
For j = 2 to n
  若 P[j] > P[i] 则
    令 i = j
    把 L 加 1
  结束若
结束For
```

在你的例子中，给出最长上升趋势的真实长度，并说明该算法返回什么。

(b) 给出一个高效算法，输入价格序列 P[1], P[2], …, P[n]，返回最长上升趋势的长度。

答案： (a) 取序列 1, 4, 2, 3：算法只在 P[j] 大于当前 P[i] 时才推进，输出 2（趋势 1, 4），而真实最长长度为 3（1, 2, 3）。

(b) 令 OPT(j) 为以 P[j] 开头的（在 P[j..n] 中的）最长递增子序列长度。则 OPT(n) = 1， OPT(j) = 1 + max_{k>j, P[k]>P[j]} OPT(k)， 答案为 OPT(1)。按 j 递减填表，每步 O(n − j)，总时间 O(n²)，可回溯给出该子序列。

习题 18

题目： 考虑四字母表 {z_1, z_2, z_3, z_4} 上的序列比对（sequence alignment）问题，给定空缺代价（gap cost）与失配代价（mismatch cost）。假设这些参数都是正整数。

设给定两个字符串 A = a_1a_2⋯a_m 与 B = b_1b_2⋯b_n，以及它们之间的一个候选比对。给出 O(mn) 时间的算法，判定该比对是否是 A 与 B 之间唯一的（unique）最小代价比对。

答案： 用课上构造的有向无环图 G（s 在左上、t 在右下，s-t 路径 ↔ 全局比对），设 Q 为给定比对对应的路径，Δ(Q) 为其中对角边集合。取 ε = 1/(2n)、ε′ = 1/(4n²)，构造 G′：把不在 Δ(Q) 中的对角边代价减 ε、把 Δ(Q) 中的对角边代价加 ε′（其余边不变）。可证 Q 是 G′ 的最小代价 s-t 路 ⇔ Q 是 G 中唯一的最小代价 s-t 路。故只需在 G′ 上跑一次比对 DP，检查最小代价是否等于 c′(Q)；时间 O(mn)、空间 O(m + n)。

习题 19

题目： 你正为一群人做咨询（他们不愿在此具名），这些人的工作是监测和分析来自大西洋沿岸船舶的电子信号。他们需要一个快速算法来实现一个频繁出现的基本原语：把两个已知信号的叠加"解缠"（untangling）。具体来说，他们设想两艘船各自反复发送一个短的 0/1 序列，希望确认所听到的信号恰好是这两个发送序列的交错（interleaving），没有掺入任何额外成分。

这即是完整问题，可更明确地表述如下。给定只含 0 和 1 的字符串 x，记 x^k 为 k 份 x 连接起来。若字符串 x′ 是某个 k 的 x^k 的前缀，则称 x′ 是 x 的一个重复（repetition）。例如当 x = 101 时，101101010 是 x 的一个重复。

称字符串 s 是 x 与 y 的交错（interleaving），如果 s 的符号可划分成两个（不一定连续的）子序列 s′ 与 s″，使 s′ 是 x 的重复、s″ 是 y 的重复。（于是 s 的每个符号恰属于 s′ 或 s″ 之一。）例如 x = 101、y = 00 时，s = 100010101 是 x 与 y 的交错，因为字符 1, 2, 5, 7, 8, 9 构成 101101——x 的一个重复——而其余字符 3, 4, 6 构成 000——y 的一个重复。

对应到应用：x 与 y 是两艘船重复发送的序列，s 是听到的信号；要确认 s 能"解开"为 x 与 y 的简单重复。给出一个高效算法，输入字符串 s, x, y，判定 s 是否是 x 与 y 的交错。

答案： 设 s 共 n 个字符，把 x、y 各自延长为其重复的前 n 个字符，记为 x′、y′（均长 n），问题化为「s 是否为 x′ 与 y′ 的交错」。令 M[i, j] = 「s 的前 i + j 个字符能由 x′[1..i] 与 y′[1..j] 交错而成」。则 M[0,0] = yes； M[i,j] = (M[i−1,j] ∧ s[i+j] = x′[i]) ∨ (M[i,j−1] ∧ s[i+j] = y′[j])。 按 i + j 递增填表，O(n²) 个状态、每个 O(1)；若存在 i + j = n 且 M[i,j] = yes 则返回是。时间、空间均 O(n²)。

习题 20

题目： 假设学期接近尾声，你有 n 门课，每门都还有一个期末项目未完成。每个项目按如下标准评分：得到一个 1 到 g（g > 1）之间的整数分，分数越高越好。你的目标自然是最大化 n 个项目的平均分。

你总共可以花 H > n 小时在这 n 个项目上，需要决定如何分配时间。为简单起见，设 H 是正整数，且你会给每个项目花整数小时。为了确定如何分配时间，你已经得到一组函数 {f_i : i = 1, 2, …, n}（当然只是粗略估计）：若你在课程 i 的项目上花 h ≤ H 小时，就能得到分数 f_i(h)。（可以假设 f_i 非降：h < h′ 时 f_i(h) ≤ f_i(h′)。）

因此问题是：给定这些函数 {f_i}，决定每个项目花多少小时（整数值），使按 f_i 计算的平均分尽可能大。为保证效率，算法运行时间应是 n、g、H 的多项式；这些量都不应出现在运行时间的指数上。

答案： 平均分 = 总分 / n，故只需最大化总分。令 A[i, h] 为前 i 门课、总共至多 h 小时能取得的最大总分：A[0, h] = 0，A[i, 0] = $\sum_{j=1}^{i} f_j(0)$，$A[i, h] = \max_{0 \le k \le h} \{ f_i(k) + A[i-1, h-k] \}$，同时记录取到最大值的 k。答案为 A[n, H]，回溯即得时间分配。填每个格 O(H)，共 nH 格，时间 O(nH²)，空间 O(nH)。

习题 21

题目： 前些时候你帮助过一群为某计算密集型投资公司做模拟的朋友，如今他们带着新问题回来找你。他们在研究某只股票过去连续 n 天的数据，天数编号为 i = 1, 2, …, n；第 i 天的每股价格为 p(i)。

对某些（可能很大的）k，他们想研究所谓的 k-shot 策略（k-shot strategy）。一个 k-shot 策略是 m 对天数 (b_1, s_1), …, (b_m, s_m)，其中 0 ≤ m ≤ k 且

1 ≤ b_1 < s_1 < b_2 < s_2 < ⋯ < b_m < s_m ≤ n。

我们把它视为至多 k 个互不重叠的区间：在每个区间内，投资者在第 b_i 天买入 1000 股，在第 s_i 天卖出。给定 k-shot 策略的回报（return）就是这 m 笔买卖所得利润，即

1,000 $\sum_{i=1}^{m} (p(s_i) - p(b_i))$。

投资者希望通过在 n 天价格数据上模拟来评估 k-shot 策略的价值。你的目标是设计一个高效算法，给定价格序列，确定回报最大的 k-shot 策略。由于这些模拟中 k 可能相当大，运行时间应是 n 与 k 的多项式；不应含 k 的指数。

答案： 先预处理 Q[i, j] = 在 [i, j] 区间内做至多一笔交易（可以不做）的最大收益：令 P[i,j] = 1000(p(j) − p(i))，则 Q[i,j] = max{ P[i,j], Q[i,j−1], Q[i+1,j] }，按 i + j 递增填表，O(n²) 全部算出。再令 M[m, d] = 前 d 天恰好做 m 笔不重叠交易的最大收益：M[0, d] = 0，M[m, 0] = −∞（m ≥ 1），$M[m, d] = \max_{1 \le i < j \le d} \{ Q[i,j] + M[m-1, i-1] \}$，每格 O(n)，共 O(kn) 格，总时间 O(kn²)；答案 $\max_{0 \le m \le k} M[m, n]$，用指针回溯即得策略。

习题 22

题目： 要评估有向图中两个节点"连接得有多紧密"，不能只看它们之间最短路的长度，还可以数最短路的条数。

事实证明，在一些对边代价的限制下，这个问题可以高效求解。设有向图 G = (V, E)，边上带代价；代价可正可负，但图中每个环的代价严格为正。又给定两个节点 v, w ∈ V。给出一个高效算法，计算 G 中 v 到 w 的最短路条数。（算法不必列出所有路径，只需给出数目。）

答案： 因每个环代价严格为正，最短路必是简单路，至多 n − 1 条边。令 Opt(i, s) = 用恰好 i 条边到 s 的最小代价，N(i, s) = 达到该最优值的路径条数。初值 Opt(0, v) = 0、Opt(0, s) = ∞（s ≠ v），N(0, v) = 1、N(0, s) = 0。则 $Opt(i, s) = \min_{(t,s) \in E} \{ Opt(i-1, t) + c_{ts} \}$，$N(i, s) = \sum_{t : Opt(i,s) = Opt(i-1,t) +$

c_ts} N(i−1, t)。 随后 Opt(w) = min_i Opt(i, w)，N(w) = ∑{i : Opt(i,w) = Opt(w)} N(i, w)。两层循环（i 与外层、所有边），时间 O(nm)，空间 O(n)（滚动）。

习题 23

题目： 设有向图 G = (V, E)，每条边 e ∈ E 有代价 c_e，并给定一个汇点（sink）t（代价可以为负）。又给定每个 v ∈ V 的一个有限值 d(v)。有人断言：对每个节点 v ∈ V，d(v) 就是从节点 v 到汇点 t 的最小代价路径的代价。

(a) 给出一个线性时间算法（图有 m 条边时为 O(m)），检验该断言是否正确。

(b) 假设这些距离是正确的，且对所有 v ∈ V，d(v) 有限。现在你要计算到另一个汇点 t′ 的距离。给出一个 O(m log n) 的算法，计算所有 v ∈ V 到新汇点 t′ 的距离 d′(v)。（提示：考虑如下定义的新代价函数：对边 e = (v, w)，令 c′_e = c_e − d(v) + d(w)。两种不同代价下路径的代价之间有什么关系？）

答案： (a) 先对每条边 (v, w) 检查：若 d(v) > d(w) + c_e 则立即拒绝（否则经 w 到 t 有更便宜的路径）。再构造 G′，只保留满足 d(v) = d(w) + c_e 的"紧边"，检查每个节点在 G′ 中都能到达 t（把边反向做 BFS/DFS），否则拒绝，否则接受。两个阶段均 O(m)。正确性：若接受而某个真实距离 d′(v) < d(v)，沿真实最短路取最后一个 d′ < d 的节点 x 及其后继 y（d′(y) = d(y)），则 (x, y) 的紧性条件被违反，与接受矛盾；同时这些紧边恰好包含真实最短路上的边。

(b) 由 (a) 的紧性，c′_e = c_e − d(v) + d(w) ≥ 0。对任一 x 到 t′ 的路径 P，有 c′(P) = c(P) − d(x) + d(t′)，故 d′(x) = d(x) − d(t′) + dist′(x, t′)。在反图中以 c′ 为权从 t′ 运行 Dijkstra，O(m log n)。

习题 24

题目： Gerrymandering（不公正划分选区）是指以极其精心设计的方式划分选区，使结果有利于某个特定政党。近来法院对这一做法的质疑主张：通过这种精心设计的重新划分，大量选民实际上被（有意地）剥夺了投票权。

事实证明，计算机在有关此事的新闻报道中被指为"元凶"之一：借助强大的软件，gerrymandering 已从一群人拿着地图、铅笔和纸进行的活动，变为如今这种工业规模的流程。为什么 gerrymandering 是一个计算问题？把选民人口统计追踪到个别街道和住宅的层级涉及数据库问题；而把选民归入各选区则涉及算法问题。我们来看看后一类问题是什么样子。

设有 n 个选区（precincts）P_1, P_2, …, P_n，每个选区含 m 名登记选民。我们要把这些选区划分成两个区（districts），每个区由 n/2 个选区组成。现在对每个选区，我们都有登记为两个政党之一的人数。（为简单起见，假设每个选民只登记在这两个政党之一。）称这组选区易受 gerrymandering 影响（susceptible），如果能够将它们划分成两个区，使得同一政党在两个区中都占多数。

给出算法判定给定选区集合是否易受 gerrymandering 影响；运行时间应是 n 与 m 的多项式。

例：设 n = 4 个选区，登记选民信息如下。

选区	1	2	3	4
登记为 A 党的人数	55	43	60	47
登记为 B 党的人数	45	57	40	53

该集合是易受影响的：若把选区 1 和 4 划为一个区、2 和 3 划为另一个区，则 A 党在两个区都占多数。（做划分的"我们"大概是 A 党成员。）这个例子简要说明了 gerrymandering 的基本不公：A 党在总人口中仅有微弱多数（205 对 195），最终却在不止一个区中占多数。

答案： 令 M[j, p, x, y] = true 表示能把前 j 个选区中的 p 个分给第 1 区，使第 1 区有 x 张 A 票、第 2 区有 y 张 A 票。设选区 j + 1 有 z 张 A 票，则 M[j+1, p, x, y] = M[j, p−1, x−z, y]（放入第 1 区）∨ M[j, p, x, y−z]（放入第 2 区）。 按 j 递增填表，最后扫描是否存在 x, y > mn/4 且 M[n, n/2, x, y] = true（每区共 mn/2 票，过半即 A 票 > mn/4）。状态数 O(n²m²)（x, y ≤ mn），每个 O(1)，时间 O(n²m²)。

习题 25

题目： 考虑股票经纪人要卖出某公司大量股票、而该公司股价持续下跌时所面临的问题。预测正确的卖出时机总是很难，但持有单一公司的大量股份还有一个额外复杂性：一天之内卖出大量股票这一行为本身就会对价格产生不利影响。

由于未来市场价格以及大额卖出对这些价格的影响都难以预测，券商用市场模型来辅助决策。本题考虑如下简单模型。设我们要卖出某公司的 x 股股票，并设我们有准确的市场模型：它预测未来 n 天的股价为 p_1, p_2, …, p_n。此外，有一个函数 f(·) 预测大额卖出的影响：若某天卖出 y 股，则从那天起价格将永久下降 f(y)。于是若第 1 天卖出 y_1 股，我们得到每股 p_1 − f(y_1) 的价格，总收入为 y_1·(p_1 − f(y_1))。第 1 天卖出 y_1 股后，第 2 天可再卖 y_2 股，每股价格 p_2 − f(y_1) − f(y_2)；这带来额外收入 y_2·(p_2 − f(y_1) − f(y_2))。此过程持续 n 天。（注意，正如第 2 天的计算所示，较早天数造成的降幅已并入其后所有天数的价格中。）

设计一个高效算法，输入价格 p_1, …, p_n 与函数 f（以值 f(1), f(2), …, f(x) 的列表给出），确定在第 n 天之前卖出 x 股的最佳方式。换言之，求自然数 y_1, y_2, …, y_n 使得 x = y_1 + ⋯ + y_n，且第 i 天卖出 y_i 股（i = 1, 2, …, n）使可获得的收入最大。你可以假设股价 p_i 单调递减、f(·) 单调递增；即卖出越多，价格下跌越多。算法运行时间可对 n（天数）、x（股数）与 p_1（股价峰值）呈多项式依赖。

例：设 n = 3，三天价格为 90, 80, 40；f(y) = 1（y ≤ 40,000），f(y) = 20（y > 40,000）。设开始有 x = 100,000 股。第 1 天全部卖出，每股 70，总收入 7,000,000。另一方面，第 1 天卖 40,000 股得每股 89，第 2 天卖剩余 60,000 股得每股 59，总收入 7,100,000。

答案： 令 Opt(i, k) 表示从第 i 天起卖出剩余 k 股的最大收入（此前各天造成的降价是常数，对后续所有剩余股份影响相同，故不影响最优选择）。设第 i 天卖 y 股：当天收入 y·p_i；这一决定使当时仍持有的 k 股每股多跌 f(y)，总损失 k·f(y)。于是 Opt(i, k) = max_{0 ≤ y ≤ k} { y·p_i + Opt(i+1, k−y) − k·f(y) }， 边界 Opt(n+1, 0) = 0，Opt(n+1, k) = −∞（k > 0）。按 i 从 n 递减、k 递增填表，答案 Opt(1, x)，记录 y 即可回溯。时间 O(nx²)，空间 O(nx)，均对 n、x、p_1 多项式。

习题 26

题目： 考虑如下库存问题。你经营一家销售大型产品（不妨说卡车）的公司，预测告诉你未来 n 个月的销售量。令 d_i 表示第 i 个月的预期销售量。我们假设所有销售都发生在月初，未卖出的卡车会被存放到下月初。你最多可存放 S 辆卡车，每辆卡车每月的存储费为 C。你通过下订单来获得卡车，每次下单需支付固定订购费 K（无论订多少辆卡车）。开始时你没有卡车。问题是设计算法决定如何下单，以满足所有需求 {d_i} 并使成本最小。总结如下：

- 成本由两部分组成：(1) 存储——每辆当月不需要的卡车花 C；(2) 订购费——每次下单花 K。

- 每个月你都需要足够的卡车满足需求 d_i，但满足当月需求后剩余的卡车数不应超过库存上限 S。

给出一个在 n 与 S 的多项式时间内求解该问题的算法。

答案： 令 OPT(i, s) = 满足前 i 个月需求、且第 i 月末剩下 s 辆卡车时的最小总成本（0 ≤ s ≤ S），答案为 OPT(n, 0)。设上月末剩 z 辆（0 ≤ z ≤ S），则本月共有 z + d_i 辆、须有 s + d_i ≤ z 才能留下 s 辆，存储费 zC；若 z < s + d_i 则本月必须下单并付 K。由于"下单"情形的 z 越小越好（不必为额外卡车付存储费），最优只在 z = 0 处取，故 - 若 s + d_i > S：OPT(i, s) = OPT(i−1, 0) + K； - 否则 OPT(i, s) = min{ OPT(i−1, s+d_i) + C(s+d_i), OPT(i−1, 0) + K }。 共 n(S+1) 个子问题，每个 O(1)，时间与空间 O(nS)。

习题 27

题目： 一家独立经营的加油站的业主面临如下情况。他们有一个大型地下储油罐，一次最多可装 L 加仑。订购汽油相当昂贵，因此他们希望尽量少订。但每次订购除了汽油本身的费用外，还要支付固定的配送费 P。然而，把一加仑汽油多存放一天要花 c，所以过早订得太多会增加存储成本。

他们计划冬季停业一周，希望届时油罐是空的。幸运的是，凭多年经验，他们对到那时为止每天需要多少汽油有准确预测。假设距停业还有 n 天，第 i 天需要 g_i 加仑，i = 1, …, n。假设第 0 天结束时油罐为空。给出一个算法，决定应在哪些天下单、每次订多少，以最小化他们的总成本。

答案： 各方案买油总量相同，故汽油本身费用可忽略，只计配送费与存储费。记 S(a, b) = *∑{j=a}^{b} (j − a)·c·g_j 为第 a 天到货的油在第 a 至 b 天间的存储费。令 OPT(a) = 第 a 天开始时罐空、且当天有油到达时，覆盖第 a..n 天的最小成本。若下一次到货在第 b 天，则本次须订* ∑{j=a}^{b−1} g_j 加仑（要求 ≤ L），付配送费 P 与存储费 S(a, b−1)，之后从第 b 天起最优： OPT(a) = P + min_{b > a, ∑_{j=a}^{b−1} g_j ≤ L} { S(a, b−1) + OPT(b) }， 其中约定 b = n + 1 表示此后不再订货、OPT(n+1) = 0。只需考虑 O(n) 个后续到货日。按 a 递减填表，O(n²) 时间；答案 OPT(1)，回溯得每次下单的日期与数量。

习题 28

题目： 回忆 4.2 节中的排程问题，我们要最小化最大延迟（maximum lateness）。有 n 个作业，每个有截止时间 d_i 与所需处理时间 t_i，所有作业从时刻 s 起都可以被排程。作业 i 需被分配一个从 s_i ≥ s 到 f_i = s_i + t_i 的区间，不同作业应分到互不重叠的区间。如常，这样的时间分配称为一个排程（schedule）。

本题沿用相同设置，但要优化一个不同的目标。特别地，考虑每个作业要么必须在截止时间前完成、要么干脆不做的情况。称作业子集 J 是可排程的（schedulable），如果存在 J 中作业的排程使其中每个作业都在自己的截止时间前完成。你的问题是选出尽可能大的可排程子集，并给出使该子集中每个作业都按时完成的排程。

(a) 证明存在一个最优解 J（即规模最大的可排程集合），其中 J 的作业按截止时间递增的顺序排程。

(b) 假设所有截止时间 d_i 与所需时间 t_i 均为整数。给出求最优解的算法，运行时间应是作业数 n 与最大截止时间 D = max_i d_i 的多项式。

答案： (a) 取最大可排程集 J，只对 J 中作业考虑"最小化最大延迟"。由 J 的定义其最小最大延迟为 0；而课上已证按截止时间递增排序（EDF）的贪心算法对该目标最优，故该顺序给出 J 的一个可行排程。

(b) 把作业按截止时间递增重新编号，使 d_1 ≤ ⋯ ≤ d_n = D。令 OPT(d, m) = 在时刻 d 之前能从 {1, …, m} 中完成的最大作业数。要么不做 m：OPT(d, m) = OPT(d, m−1)；要么做 m（由 (a) 可设它最后做，需 t_m ≤ d）：OPT(d, m) = OPT(d − t_m, m−1) + 1。取二者较大者，自 d = 0 与 m = 0 的边界开始填 n × D 的表，O(nD) 时间与空间，回溯即得排程。

习题 29

题目： 设 $G = (V, E)$ 是 n 个节点的图，其中每一对节点之间都有一条边。每条边 (i, j) 上有正权 w_{ij}；并假设这些权满足三角不等式 $w_{ik} \le w_{ij} + w_{jk}$。对子集 $V' \subseteq V$，用 $G[V']$ 表示由 V' 诱导的子图（带边权）。

给定一组 k 个必须被边连接起来的终端（terminals）$X \subseteq V$。称 X 上的 Steiner 树是一个集合 Z，满足 $X \subseteq Z \subseteq V$，连同一个 $G[Z]$ 的生成子树 T。Steiner 树的权（weight）就是树 T 的权。

证明：存在函数 $f(\cdot)$ 与多项式函数 $p(\cdot)$，使得求 X 上最小权 Steiner 树的问题可以在 $O(f(k) \cdot p(n))$ 时间内求解。

答案： 称 $Y \subseteq V$ 为"丰富集"（enriched），若 $|Y \setminus X| \le 1$；因 X 只有 k 个终端，丰富集共 $O(2^k n)$ 个。按 $|Y|$ 递增用 DP 计算两个量：$f(Y) = Y$ 上最小生成树的权（可在 $O(n^2)$ 内算出），$g(Y) = Y$ 上最小 Steiner 树的权。对 $Y = X' \cup \{i\}$（$X' \subseteq X$，$i \notin X$）：最优 Steiner 树要么不含额外节点，此时 $g(Y) = f(Y)$；要么含一个额外节点 j（可设度为 3）。删去 j 得若干子树，把 Y 划分为 $Y_1 \ni i$、Y_2、Y_3，取 $p \in Y_1$ 为与 j 相邻者，则 $g(Y) = \min\{\, g(Y_1) + g(Y_2 \cup \{j\}) + g(Y_3 \cup \{j\}) + w_{jp} : \text{划分与 } j, p \text{ 的枚举} \,\}$，右端各项都是更小丰富集上已算好的值（若都大于 $f(Y)$ 则取 $f(Y)$，即最小生成树）。每个丰富集需枚举 $O(3^k)$ 种划分及 j、p，总时间 $O(3^k \cdot \mathrm{poly}(n))$，故取 $f(k) = 3^k$、$p(n)$ 为适当多项式即可。

第8章 NP 与计算难解性

习题 1

题目： 对下面两个问题，判断答案属于哪一种：(i)「是」；(ii)「否」；(iii)「未知，因为回答它就会解决 $\mathcal{P} = \mathcal{NP}$ 问题」。并给出简要解释。 (a) 把第 4 章中的区间调度问题（Interval Scheduling）定义为如下判定形式：给定时间轴上的一组区间，以及一个界 k，这组区间中是否包含一个大小至少为 k 的两两不重叠的区间子集？问：是否有 Interval Scheduling ≤_P Vertex Cover（区间调度多项式时间归约到顶点覆盖）？ (b) 问：是否有 Independent Set ≤_P Interval Scheduling（独立集多项式时间归约到区间调度）？

答案： (a)（i）是。区间调度可在多项式时间内求解，因此借助 Vertex Cover 的黑箱也能在多项式时间内求解（甚至根本不必调用黑箱）；也可由「区间调度 ∈ NP，而 NP 中任何问题都多项式归约到 NP 完全的 Vertex Cover」或由传递性（Interval Scheduling ≤_P Independent Set ≤_P Vertex Cover）得到。 (b)（iii）未知，因为这等价于 P = NP：若 P = NP，则 Independent Set 多项式可解，故可归约到 Interval Scheduling；反之若 Independent Set ≤_P Interval Scheduling，则由 Interval Scheduling ∈ P 得 Independent Set ∈ P，而 Independent Set 是 NP 完全的，从而 P = NP。

习题 2

题目： 一家商店为了分析顾客行为，常维护一个二维数组 A，行对应顾客，列对应商店出售的商品。元素 A[i, j] 表示顾客 i 购买商品 j 的数量。 下面是这样一个数组 A 的小例子：

	liquid detergent	beer	diapers	cat litter
Raj	0	6	0	3
Alanis	2	3	0	0
Chelsea	0	0	0	7

商店可能想做的一件事如下。称顾客的子集 S 是 diverse（多样）的，如果 S 中任意两位顾客从未买过同一件商品（即对每件商品，S 中至多有一位顾客买过它）。多样的顾客集合很有用，例如可作为市场研究的目标人群。 于是定义 Diverse Subset Problem（多样子集问题）：给定如上定义的 m × n 数组 A，以及一个数 k ≤ m，是否存在至少 k 位顾客组成的子集是 diverse 的？证明 Diverse Subset 是 NP 完全的。

答案： 属于 NP：给定 k 位顾客，可在多项式时间内检验没有两人买过同一商品。从 Independent Set 归约：为图 G 的每个顶点建一位顾客，为每条边建一件商品，顾客 v 购买商品 e 当且仅当 e 与 v 关联；则大小 k 的多样子集与大小 k 的独立集一一对应。

习题 3

题目： 假设你在帮忙组织一个夏令营，遇到如下问题。夏令营要覆盖 n 个运动项目（棒球、排球等），每个项目至少需要一名擅长的辅导员。他们收到了 m 位潜在辅导员的申请。对每个运动项目，都有 m 位申请人中具备该项目资格的一个子集。问题是：给定 k < m，能否雇用至多 k 位辅导员，使得 n 个运动项目中每个都至少有一位具备资格的辅导员？我们称之为 Efficient Recruiting Problem（高效招聘问题）。证明 Efficient Recruiting 是 NP 完全的。

答案： 属于 NP：给定至多 k 位辅导员，可检验他们覆盖所有项目。从 Vertex Cover 归约：每条边 e 对应一个运动项目 S_e，每个顶点 v 对应一位辅导员 C_v，C_v 具备 S_e 的资格当且仅当 v 是 e 的端点；则 k 位辅导员覆盖全部项目当且仅当对应 k 个顶点构成顶点覆盖。

习题 4

题目： 假设你为一个管理高性能实时系统的团队做咨询，系统中异步进程使用共享资源。系统有 n 个进程（process）和 m 个资源（resource）。任一时刻，每个进程指定它请求使用的一组资源。每个资源可能同时被多个进程请求，但同一时刻只能被一个进程使用。你的工作是为提出请求的进程分配资源：若一个进程被分配了它请求的全部资源，则它是 active（活跃）的，否则是 blocked（阻塞）的。你希望分配使尽可能多的进程活跃。于是 Resource Reservation Problem（资源预约问题）表述为：给定一组进程和资源、每个进程请求的资源集合，以及一个数 k，能否为进程分配资源使得至少 k 个进程活跃？ 考虑下面几个问题，对每个问题要么给出多项式时间算法，要么证明它是 NP 完全的。 (a) 上面定义的一般 Resource Reservation 问题。 (b) k = 2 的特殊情形。 (c) 只有两类资源——比如人员和设备——且每个进程对每类资源至多请求一个（即每个进程需要一个特定的人和一个特定的设备）的特殊情形。 (d) 每个资源至多被两个进程请求的特殊情形。

答案： (a) NP 完全。问题等价于：是否存在 k 个进程，其请求的资源集合两两不交。属于 NP；从 Independent Set 归约：资源对应边，进程对应顶点，顶点 v 请求所有与 v 关联的边。 (b) 多项式时间：暴力枚举所有 $O(n^2)$ 对进程，检查是否有某一对的请求集合不交。 (c) 化为二部匹配：为每个人和设备各设一个结点，每个进程是连接其所需要的人与设备的一条边；请求不交的进程集合就是二部图中的匹配，故只需检验是否存在大小至少 k 的匹配。 (d) NP 完全：(a) 的归约中每个资源（边）恰好只被它的两个端点（进程）请求，正属于该特殊形式，故该特殊情形仍是 NP 完全的。

习题 5

题目： 考虑集合 $A = \{a_1, \ldots, a_n\}$ 以及 A 的一族子集 $B_1, B_2, \ldots, B_m$（即对每个 j，$B_j \subseteq A$）。 称集合 $H \subseteq A$ 是 $B_1, B_2, \ldots, B_m$ 的 hitting set（命中集），如果 H 至少包含每个 B_j 中的一个元素——即对每个 j，$H \cap B_j \neq \emptyset$（于是 H「命中」所有 B_j）。 现定义 Hitting Set Problem（命中集问题）：给定集合 $A = \{a_1, \ldots, a_n\}$、A 的一族子集 $B_1, B_2, \ldots, B_m$，以及一个数 k。问：是否存在 A 的命中集 $H \subseteq A$，其大小至多为 k？证明 Hitting Set 是 NP 完全的。

答案： 属于 NP：给定 H，可在多项式时间内检验 $|H| \le k$ 且每个 B_j 都被命中。从 Vertex Cover 归约：令 A = V，对每条边 $e_j = (u_j, v_j)$ 令 $S_j = \{u_j, v_j\}$；则大小至多 k 的命中集与大小至多 k 的顶点覆盖一一对应。

习题 6

题目： 考虑由子句 $C_1, \ldots, C_k$ 在一组布尔变量 $x_1, \ldots, x_n$ 上指定的 Satisfiability 问题实例。若每个子句中的每一项都由未取反的变量构成，即每一项都形如 x_i（而非 $\neg x_i$），则称该实例是 monotone（单调）的。单调的 Satisfiability 实例很容易求解：它们总是可满足的，把所有变量置为 1 即可。 例如，假设有三个子句 $(x_1 \vee x_2)$, $(x_1 \vee x_3)$, $(x_2 \vee x_3)$。这是单调的，把三个变量都置 1 确实满足所有子句。但可以注意到这不是唯一的满足赋值；也可以令 x_1 与 x_2 为 1、x_3 为 0。事实上，对任何单调实例，自然会问：需要把多少个变量置为 1 才能满足它。 给定一个单调 Satisfiability 实例以及一个数 k，Monotone Satisfiability with Few True

Variables（少真变量的单调可满足性）问题问：是否存在该实例的满足赋值，其中至多 k 个变量被置为 1？证明该问题是 NP 完全的。

答案： 从 Vertex Cover 归约：为每个顶点 v_i 设变量 x_i，为每条边 e_j = (v_a, v_b) 设子句 C_j = (x_a ∨ x_b)；至多 k 个变量置 1 即满足全部子句，当且仅当对应至多 k 个顶点构成顶点覆盖。

习题 7

题目： 既然 3-Dimensional Matching（三维匹配）是 NP 完全的，自然期望相应的 4-Dimensional Matching 至少同样难。定义 4-Dimensional Matching 如下：给定四个大小均为 n 的集合 W、X、Y、Z，以及一族由形如 (w_i, x_j, y_k, z_l) 的有序四元组组成的集合 C，是否存在 C 中的 n 个四元组，使得其中任意两个都没有公共元素？证明 4-Dimensional Matching 是 NP 完全的。

答案： 属于 NP：用 n × n 数组标记已用元素，可在多项式时间内检验 n 个四元组两两不交。从 3-Dimensional Matching 归约：取 W 含 n 个新元素，对每个三元组 (x_j, y_k, z_l) ∈ C 及每个 i = 1, …, n 加入四元组 (w_i, x_j, y_k, z_l)；则 n 个不交四元组与 n 个不交三元组一一对应。

习题 8

题目： 你朋友家学龄前的女儿 Madison 最近学会拼一些简单单词。为鼓励她，父母给她买了一套彩色字母冰箱贴（若干个字母 A、若干个字母 B，等等）。上次你看到她时，她花了很久排列这些磁贴来拼出她认识的单词。 和你在一起时，事情总会变得比原计划更复杂：你们俩试着拼出一些单词，以便把整套磁贴全部用上——即挑选她会拼的单词，使得这些单词全部拼完后，每块磁贴恰好参与拼写其中一个单词。（单词重复出现没关系；例如若磁贴中有两块 C、两块 A、两块 T，拼两次 CAT 也可以。） 这其实相当困难；后来你意识到一个可能的原因。考虑该问题的一个一般版本，称为 Using Up All the Refrigerator Magnets（用完所有冰箱贴）：把英文字母表换成任意符号集合，并把 Madison 的词汇表建模为这些符号上的任意字符串集合，目标同上。证明 Using Up All the Refrigerator Magnets 是 NP 完全的。

答案： 属于 NP：证据是一组单词的多重集，统计其中各类磁贴的用量并与给定数量比较即可。从 3-Dimensional Matching 归约：X、Y、Z 中每个元素变成一个带唯一字母的磁贴（字母表共 3n 个字母），每个三元组 (x_i, y_j, z_k) 变成一个 Madison 会拼的单词；完美三维匹配存在当且仅当全部磁贴能恰好用完。

习题 9

题目： 考虑如下问题。你管理一个用有向图 G = (V, E) 建模的通信网络。有 c 位用户想使用该网络；第 i 位用户（i = 1, 2, …, c）提出一个请求，要求预留 G 中一条特定的路径 P_i 来传输数据。 你希望接受尽可能多的路径请求，但受如下限制：若同时接受 P_i 和 P_j，则 P_i 与 P_j 不能有任何公共结点。 于是 Path Selection Problem（路径选择问题）问：给定有向图 G = (V, E)、一组路径 P_1, P_2, …, P_c（每条都必须是 G 中的路径）以及一个数 k，能否选出至少 k 条路径，使得所选路径中任意两条不共享结点？证明 Path Selection 是 NP 完全的。

答案： 属于 NP：给定 k 条路径，可检验它们两两无公共结点。从 3-Dimensional Matching 归约：结点集取 X ∪ Y ∪ Z，对每个三元组 (x_i, y_j, z_k) 加入边 (x_i, y_j) 与 (y_j, z_k)，并令 P 经过 x_i, y_j, z_k；则 n 条结点不交路径与 n 个不交三元组一一对应。

习题 10

题目： 你在 WebExodus 的朋友最近为一些维护大型、可公开访问的网站的公司做咨询——合同问题使他们不能说出是哪些公司——他们遇到了下面的 Strategic Advertising Problem（战略广告问题）。 一家公司带着网站地图来找他们，我们把它建模为有向图 G = (V, E)。公司还提供了用户通常遵循的 t 条 trail（浏览路径）；我们把每条路径建模为 G 中的有向路径 P_1, P_2, ..., P_t（即每条 P_i 是 G 中的一条路径）。 公司希望 WebExodus 回答：给定 G、路径 {P_i} 以及一个数 k，能否在 G 的至多 k 个结点上放置广告，使得每条路径 P_i 都至少包含一个带广告的结点？我们称之为 Strategic Advertising Problem，输入为 G、{P_i : i = 1, ..., t} 和 k。 你的朋友认为一个好算法会让他们发财；不幸的是，事情从没这么简单。 (a) 证明 Strategic Advertising 是 NP 完全的。 (b) 你的朋友还是硬着头皮写出了一个相当快的算法 $\mathcal{S}$，能对任意 Strategic Advertising 实例输出 yes/no 答案。可假设 $\mathcal{S}$ 总是正确的。把 $\mathcal{S}$ 当作黑箱，设计一个算法，输入如 (a) 中的 G、{P_i} 和 k，完成下列两件事之一： - 输出 G 中至多 k 个结点组成的集合，使每条路径 P_i 至少包含其中一个结点；或 - （正确地）输出不存在这样的至多 k 个结点的集合。 算法至多使用多项式步，并至多调用 $\mathcal{S}$ 多项式次。

答案： (a) 属于 NP：给定 k 个结点，可在 O(knt) 时间内检验每条路径都被覆盖。从 Vertex Cover 归约：把无向图 G 的每条边任意定向得 G′，为 G′ 的每条边 e = (u, v) 定义路径 u → v；则至多 k 个结点覆盖所有路径当且仅当存在大小至多 k 的顶点覆盖。 (b) 对 k 归纳。k = 1 时直接找是否有一个结点位于所有路径上。否则询问 $\mathcal{S}$ 是否存在大小至多 k 的有效集合；若答 no 则如实报告；若答 yes，则对每个结点 v 删除 v 及所有经过 v 的路径后再问 $\mathcal{S}$ 是否存在大小至多 k − 1 的有效集合，必有某个 v 使答案为 yes（取任一最优集合 U 中任一点即可）。把该 v 放入解集 T 并递归，总时间为 $O(n^2 + nt)$ 次操作与调用。

习题 11

题目： 正如一些人记得、许多人被告知的那样，超文本（hypertext）的思想比万维网早了几十年。甚至超文本小说也是相对古老的想法：它不受印刷页面线性顺序的限制，可以讲述由相互交织的虚拟「场所」通过虚拟「通道」连接而成的故事。因此一部超文本小说实际上是在一张底层有向图上展开的；为具体起见（尽管缩小了该领域的全部可能性），我们这样建模。 把一部超文本小说的结构视为有向图 G = (V, E)。每个结点 u ∈ V 包含一些文本；当读者位于 u 时，他/她可以选择沿 u 的任一条出边前进；若选择边 e = (u, v) 就到达下一个结点 v。有一个开始结点 s ∈ V 供读者开始，一个结束结点 t ∈ V；读者第一次到达 t 时故事结束。因此任何从 s 到 t 的路径都是故事的一个有效情节（plot）。注意，与使用网页浏览器不同，这里不一定能后退；一旦从 u 走到 v，你可能再也回不到 u。 这样，超文本结构在同一底层内容上定义了极多不同的情节，所有这些可能性之间的关系可能非常复杂。考虑如下类型的问题：一部超文本小说建立在图 G = (V, E) 上，其中有一些关键的 thematic elements（主题元素）：爱、死亡、战争、对主修计算机科学的强烈渴望，等等。每个主题元素 i 用集合 T_i ⊆ V 表示，由 G 中出现该主题的结点组成。现在给定一组特定的主题元素，可以问：是否存在一个有效情节，其中这些元素每一个都会遇到？更具体地，给定有向图 G、开始结点 s 和结束结点 t，以及由集合 T_1, T_2, ..., T_k 表示的主题元素，Plot Fulfillment Problem（情节实现问题）问：是否存在一条从 s 到 t 的路径，它包含每个 T_i 中至少一个结点？证明 Plot Fulfillment 是 NP 完全的。

答案： 属于 NP：给定 s-t 路径，可检验它是 G 中有效路径且与每个 T_i 相交。从 Hitting Set 归约：构造分层图，结点为 s、t 及 v_{ij}（1 ≤ i ≤ k，1 ≤ j ≤ n），边为 s → v_{1j}、v_{ij} → v_{i+1,l}、v_{kj} → t；令 T_i = {v_{ij} : a_j ∈ S_i}；则大小至多 k 的命中集与命中所有 T_i 的 s-t 路径一一对应。

习题 12

题目： 你有朋友维护一个流行的新闻与讨论网站，流量已大到他们要开始区分付费与非付费访客。标准做法是：向每月付费订阅的顾客提供网站全部内容；同时未订阅的访客仍可查看一部分页面（同时不断被广告轰炸，劝其订阅）。 控制非订阅者访问的两种简单方法是：可以 (1) 指定一个固定的页面子集供非订阅者查看；或 (2) 原则上允许查看任何页面，但规定非订阅者在单次会话中最多查看的页面数。（我们假设网站能跟踪访客的浏览路径。） 你的朋友在试验一种不同且比这两种都更微妙的方式。他们希望非订阅者能浏览网站的不同部分，于是把某些页面子集指定为特定的 zone（区域）——例如可以有政治页面区、音乐页面区等等。一个页面可以属于多个区域。当非订阅者浏览网站时，访问策略允许其从每个区域访问一个页面，但在同一次浏览会话中稍后试图访问同一区域的第二个页面时会被拒绝（而是被引导到建议其订阅的广告）。 更形式化地，把网站建模为有向图 G = (V, E)，结点表示网页，边表示有向超链接。有一个特别的 entry node（入口结点）s ∈ V，以及区域 $Z_1, \ldots, Z_k \subseteq V$。非订阅者所走的路径 P 被限制为：从每个区域 Z_i 中至多包含一个结点。 这种更复杂的访问策略有一个问题：即使回答基本的可达性问题也会变难，包括：非订阅者是否可能访问某个给定结点 t？更精确地，定义 Evasive Path Problem（规避路径问题）如下：给定 G、$Z_1, \ldots, Z_k$、s ∈ V 以及目标结点 t ∈ V，是否存在 G 中一条 s-t 路径，它从每个区域 Z_i 中至多包含一个结点？证明 Evasive Path 是 NP 完全的。

答案： 属于 NP：给定 s-t 路径 P，可对每个 i 检验 $|P \cap Z_i| \le 1$。从 3-SAT 归约：构造分层图，第 0 层只有 s；第 i 层（$1 \le i \le n$）有两个结点，分别标为文字 x_i 与 $\neg x_i$；第 n+i 层（$1 \le i \le t$）有三个结点，标为第 i 个子句的三个文字；第 n+t+1 层只有 t，各层间全连接。对每一对「变量及其否定」对应的结点设一个专属区域。则满足赋值对应一条只经过真文字结点的规避路径，反之规避路径在每个变量层选出的文字给出满足赋值。

习题 13

题目： combinatorial auction（组合拍卖）是经济学家为把一组物品卖给一群潜在买家而设计的一种特殊机制。（美国联邦通信委员会研究过用这种拍卖把无线电频谱上的电台分配给广播公司。） 下面是组合拍卖的一个简单类型。有 n 件待售物品，标为 $I_1, \ldots, I_n$。每件物品不可分割，只能卖给一个人。现在有 m 个不同的买家出价（bid）：第 i 个出价指定物品的一个子集 S_i，以及报价 x_i，即买家愿意为集合 S_i 中的物品作为一个整体支付的价格。（我们把该出价表示为 (S_i, x_i)。） 拍卖人查看全部 m 个出价，选择接受（accept）其中一些、拒绝（reject）另一些。第 i 个出价被接受的人获得对应集合 S_i 中的全部物品。因此规则是：任意两个被接受的出价不能含有公共物品，否则同一件物品会卖给两个人。 拍卖人获得所有被接受出价的价格之和。（注意这是「一次性」拍卖，没有进一步出价的机会。）拍卖人的目标是收取尽可能多的钱。 于是 Winner Determination for Combinatorial Auctions（组合拍卖的胜者确定问题）问：给定物品 $I_1, \ldots, I_n$、出价 $(S_1, x_1), \ldots, (S_m, x_m)$ 以及一个界 B，是否存在拍卖人可以接受的一组出价，使其收取的钱数至少为 B？ 例：假设拍卖人用这种方法出售一些二手计算机设备。有四件物品，标为「PC」「monitor」「printer」「scanner」；三个人出价。定义 S_1 = {PC, monitor}, S_2 = {PC, printer}, S_3 = {monitor, printer, scanner}，且 $x_1 = x_2 = x_3 = 1$。 出价为 $(S_1, x_1), (S_2, x_2), (S_3, x_3)$，界 B = 2。则该实例的答案为否：拍卖人至多接受一个出价（任意两个出价都有共同想要的物品），总收入仅为 1。 证明组合拍卖的胜者确定问题是 NP 完全的。

答案： 属于 NP：给定被接受的出价集合，可多项式时间验证两两不含公共物品且总价 ≥ B。从 Independent Set 归约：为 G 的每个顶点 v 建一个出价 S_v（恰为与 v 关联的边集），价格均为 1，物品对应 G 的边，取 B = k。被接受的出价两两不交 ⇔ 对应顶点两两不相邻 ⇔ 构成大小为 k 的独立集，故该实例

答案为「是」当且仅当 G 有独立集。于是胜者确定问题是 NP 完全的。（原解答页把问题名误写作“Diverse Subset”，归约内容与本题一致。）

习题 14

题目： 我们在第 1 章和第 4 章见过 Interval Scheduling Problem。这里考虑它的一个计算上难得多的版本，称为 Multiple Interval Scheduling（多区间调度）。和之前一样，你有一台在一段时间内可用于运行作业的处理器（例如上午 9 点到下午 5 点）。 人们提交作业到处理器上运行；处理器在任一时刻只能处理一个作业。但本模型中的作业比以往复杂：每个作业需要一组时间区间来使用处理器。例如，一个作业可能需要在上午 10 点到 11 点、以及下午 2 点到 3 点使用处理器。若接受该作业，它会在这些时段占用你的处理器（两小时），但你仍可接受需要其他时段的作业（包括上午 11 点到下午 2 点）。 现给定 n 个作业，每个由一组时间区间指定，你要回答：对给定数 k，能否接受至少 k 个作业，使得任意两个被接受的作业在时间上都没有重叠？证明 Multiple Interval Scheduling 是 NP 完全的。

答案： 属于 NP：给定 k 个作业，可检验它们两两不重叠。从 3-Dimensional Matching 归约：把时间划分为 3n 个互不相交的时片 $s_1, \ldots, s_{3n}$，每个三元组 (x_i, y_j, z_k) 对应一个需要时片 s_i、s_{n+j}、s_{2n+k} 的作业；则存在完美三部分匹配当且仅当可接受 n 个时片完全不交的作业。

习题 15

题目： 你桌上来了一个快递包裹，里面是一部开始响铃的手机，来电的是久未联系的朋友 Neo。Neo 需要你帮忙解决一个问题：他和几位同伴要监测电磁频谱上若干点的无线电信号，具体地有 n 个不同频率需要监测，为此有一批传感器可用。监测问题有两个组成部分：(1) 一个由 m 个地理位置组成的集合 L，传感器可放置在这些位置；(2) 一个由 b 个干扰源（interference source）组成的集合 S，每个干扰源在特定位置阻塞特定频率。具体地，每个干扰源 i 由二元组 (F_i, L_i) 给出，其中 F_i 是频率的子集，L_i 是位置的子集；其含义是：由于无线电干扰，放在 L_i 中任何位置的传感器都无法接收 F_i 中任何频率的信号。

称位置子集 $L' \subseteq L$ 是**充分的**（sufficient），如果对每个频率 j（$j = 1, \ldots, n$），都存在 L' 中的某个位置，使频率 j 在该处不被任何干扰源阻塞。于是，在充分集的每个位置各放一个传感器，就能监测全部 n 个频率。

他们有 k 个传感器，因此想知道是否存在大小至多 k 的充分位置集。称之为 **Nearby Electromagnetic Observation 问题**的一个实例：给定频率、位置、干扰源以及参数 k，是否存在大小至多 k 的充分集？

例子：设有四个频率 $\{f_1, f_2, f_3, f_4\}$ 和四个位置 $\{\ell_1, \ell_2, \ell_3, \ell_4\}$，三个干扰源：$(F_1, L_1) = (\{f_1, f_2\}, \{\ell_1, \ell_2, \ell_3\})$ $(F_2, L_2) = (\{f_3, f_4\}, \{\ell_3, \ell_4\})$ $(F_3, L_3) = (\{f_2, f_3\}, \{\ell_1\})$ 此时存在大小为 2 的充分集：取位置 ℓ_2 和 ℓ_4（因为 f_1、f_2 在 ℓ_4 处未被阻塞，f_3、f_4 在 ℓ_2 处未被阻塞）。

证明 Nearby Electromagnetic Observation 是 NP 完全的（NP-complete）。

答案： 属于 NP：证书是这 k 个位置，可多项式时间检验每个频率至少在其中一个位置未被阻塞。归约：从顶点覆盖（vertex cover）归约到本问题。每个节点 v_i 对应一个位置 ℓ_i，每条边 e_s 对应一个频率 f_s；对每条边 $e_s = (v_i, v_j)$ 构造一个干扰源，它在除 ℓ_i、ℓ_j 之外的所有位置阻塞频率 f_s。则大小 $\le k$ 的充分集恰对应 G 中大小 $\le k$ 的顶点覆盖，双向成立。

习题 16

题目： 考虑由集合与其它集合交集的大小来推断该集合的问题。给定一个大小为 n 的有限集 U，以及 U 的子集族 $A_1, \ldots, A_m$；还给定数 $c_1, \ldots, c_m$。问题是：是否存在集合 $X \subseteq U$，使得对每个 $i = 1, 2, \ldots, m$，$X \cap A_i$ 的基数都等于 c_i？称此为 **Intersection Inference 问题**的实例，输入为 U、$\{A_i\}$ 与 $\{c_i\}$。

证明 Intersection Inference 是 NP 完全的。

答案： 属于 NP：证书是集合 X，逐项检验 $|X \cap A_i| = c_i$ 即可。归约：从三维匹配（3-Dimensional Matching）归约。设三元组集合为 T，取 U = T；对每个元素 $j \in X \cup Y \cup Z$ 构造 $A_j = \{$含 j 的三元组$\}$，并令所有 $c_j = 1$。则满足所有 $|X \cap A_j| = 1$ 的 $X \subseteq T$ 恰为完美三维匹配。

习题 17

题目： 给定有向图 $G = (V, E)$，每条边 $e \in E$ 带权 w_e，权可以为负也可以为正。**Zero-Weight-Cycle 问题**是判定 G 中是否存在一个简单环，使环上各边权之和恰为 0。证明该问题是 NP 完全的。

答案： 属于 NP：证书是该环，检验其权和为 0。归约：从子集和（Subset Sum）归约。给定 $w_1, \ldots, w_n$ 与目标 W，构造节点 $0, 1, \ldots, n$，对所有 $i < j$ 连边 (i, j) 并赋权 w_j，另加边 $(n, 0)$ 赋权 $-W$。则存在和为 W 的子集当且仅当存在零权环（该环必用边 $(n, 0)$，其余边权须恰好抵消 $-W$）。

习题 18

题目： 你受托帮一些组织理论学者分析群体决策的数据，数据来自某个政府政策委员会的决策记录，他们想判断能否找出委员会中一小撮有影响力的成员。

委员会有 n 名成员 $M = \{m_1, \ldots, m_n\}$，过去一年就 t 个不同议题投票。每个议题上每位成员可投“Yes”“No”或“Abstain”（弃权）；总体效果是：若“Yes”票数严格大于“No”票数（弃权票不计入任何一方），委员会对该议题作出肯定决定，否则作出否定决定。

现在有一张记录每位成员在各议题上投票的大表，考虑如下定义：成员子集 $M' \subseteq M$ 称为**决定性的**（decisive），如果把 M' 成员的投票单独拿出来看，委员会在每一个议题上的决定都会与原来相同（即 M' 内投票的总体结果在每个议题上都与全体委员会投票的总体结果一致）。这样的子集可视为反映委员会整体行为的“核心圈”。

问题是：给定每位成员在各议题上的投票以及参数 k，是否存在由至多 k 名成员组成的决定性子集？称之为 **Decisive Subset 问题**的实例。

例子：设委员会有 4 名成员、3 个议题，要找大小至多 $k = 2$ 的决定性子集，投票如下：

议题	m_1	m_2	m_3	m_4
议题 1	Yes	Yes	Abstain	No
议题 2	Abstain	No	No	Abstain
议题 3	Yes	Abstain	Yes	Yes

则该实例答案为“是”，因为成员 m_1 和 m_3 构成决定性子集。

证明 Decisive Subset 是 NP 完全的。

答案： 属于 NP：证书是至多 k 名成员的子集，可多项式时间检验其在每个议题上的结果与全体一致。归约：从顶点覆盖归约。每条边 e 对应一个议题 I_e，每个节点 v 对应成员 m_v；若 e = (u, v)，则令 u、v 对议题 I_e 投 Yes、其余成员全部弃权（全体结果为 Yes）。则大小 ≤ k 的决定性子集恰对应大小 ≤ k 的顶点覆盖。

习题 19

题目： 假设你担任某个太平洋岛国港务局的顾问。该国每年有数十亿美元的业务，其收入几乎完全受制于卸载到港船舶的速度。

处理危险品给本已复杂的任务增添了额外复杂性。假设早晨一支船队到达，送来共 n 个罐式集装箱，每个集装箱装一种不同的危险物质。码头上有一组 m 辆卡车，每辆最多能装 k 个集装箱。

以下是与危险品处理可能施加的不同类型约束有关的两个问题。对每个问题，给出下列两种答案之一：

- 一个多项式时间算法；或
- 一个 NP 完全性证明。

(a) 对每个集装箱，都规定了可以安全运载它的卡车子集。是否存在一种装法，把全部 n 个集装箱装上 m 辆卡车，使得没有卡车超载，且每个集装箱都装进允许运载它的卡车？

(b) 在这个不同版本中，任何集装箱都可放在任何卡车上；但是有某些集装箱对不能放在同一辆卡车上（它们所含的化学物质接触后可能爆炸）。是否存在一种装法，把全部 n 个集装箱装上 m 辆卡车，使得没有卡车超载，且不应放在一起的两个集装箱不会被放在同一辆卡车？

答案： (a) 可用网络流：源点 s 到每个集装箱节点连容量 1 的边，集装箱 i 到允许的卡车 j 连容量 1 的边，每辆卡车 j 到汇点 t 连容量 k 的边；存在可行装法当且仅当最大流值为 n（整数流即给出装载方案）。在 O(m + n) 个节点、O(mn) 条边的图上求最大流，多项式时间。

(b) NP 完全：从三着色（3-Coloring）归约。每个节点 v_i 对应一个集装箱 i，取 3 辆卡车、每辆容量 k = n；G 中每条边对应的两个集装箱不可同车。则 G 的三着色等价于全部集装箱装入 3 辆卡车。

习题 20

题目： 聚类（clustering）的直观问题有许多种形式化方式，其目标是把一组对象分成彼此“相似”的组。

首先，聚类问题输入的一种自然表达方式是：一组对象 $p_1, p_2, \ldots, p_n$，以及定义在每一对对象上的数值距离 $d(p_i, p_j)$。（只要求 $d(p_i, p_i) = 0$；对互不相同的 p_i, p_j 有 $d(p_i, p_j) > 0$；且距离对称：$d(p_i, p_j) = d(p_j, p_i)$。）

本书第 4.7 节考虑过聚类问题的一种合理表述：把对象分成 k 个集合，使不同集合中任意两个对象之间的最小距离最大化。这可用最小生成树问题的一个巧妙应用求解。

另一种看似相关但不同的表述是：把对象分成 k 个集合，使同一集合内任意两个对象之间的最大距离最小化。注意两者的差别：前一种表述要求没有两个集合“靠得太近”，新表述则要求没有哪个集合太“宽”——即没有集合包含相距很远的两点。

鉴于二者的相似性，这个新表述在计算上难以最优求解或许令人惊讶。为了用 NP 完全性的语言来思考，先把它写成一个 yes/no 判定问题。给定 n 个对象 $p_1, \ldots, p_n$ 及上述距离，以及一个界 B，定义 **Low-Diameter Clustering 问题**如下：能否把对象划分成 k 个集合，使得同一集合中任意两点之间的距离都不超过 B？

证明 Low-Diameter Clustering 是 NP 完全的。

答案： 属于 NP：证书是该划分，检验每组内任意两点距离 ≤ B 即可。归约：从 k 着色（k-Coloring）归约。每个节点 v_i 对应对象 p_i，令 d(p_i, p_i) = 0；若 (v_i, v_j) ∉ E 则 d = 1，若 (v_i, v_j) ∈ E 则 d = 2；取 B = 1。则 G 可 k 着色当且仅当对象可分成 k 组且组内距离 ≤ 1。

习题 21

题目： 在创业自助畅销书《Mine Your Own Business》中沉浸了太多天之后，你终于意识到需要升级办公计算系统。这却带来一些棘手的问题。

配置新系统时，有 k 个**组件**（component）需要选定：操作系统、文本编辑软件、电子邮件程序等等，每个都是独立的组件。对系统的第 j 个组件，你有一组选项 A_j；系统的一个**配置**（configuration）就是从每个选项集 A_1, A_2, …, A_k 中各选一个元素。

麻烦在于，来自不同选项集的某些选项对可能互不兼容。称选项 x_i ∈ A_i 与 x_j ∈ A_j（i ≠ j）构成**不兼容对**（incompatible pair），如果同一个系统不能同时包含二者。（例如 Linux（作为操作系统选项）与 Microsoft Word（作为文本编辑软件选项）构成不兼容对。）称系统的一个配置**完全兼容**（fully compatible），如果它由元素 $x_1 \in A_1$, $x_2 \in A_2$, …, x_k ∈ A_k 组成，且其中没有任何一对 (x_i, x_j) 是不兼容对。

现在可以定义 **Fully Compatible Configuration (FCC) 问题**。FCC 的一个实例由互不相交的选项集 A_1, A_2, …, A_k 以及一个不兼容对集合 P 组成，P 中每一对 (x, y) 的两个元素都属于不同的选项集。问题是要判定是否存在一个完全兼容的配置：从每个选项集各选一个元素，使所选元素中没有任何一对属于 P。

例子：设 k = 3，A_1, A_2, A_3 分别表示操作系统、文本编辑软件和电子邮件程序的选项，且有 A_1 = {Linux, Windows NT}， A_2 = {emacs, Word}， A_3 = {Outlook, Eudora, rmail}， 不兼容对集合为 P = {(Linux, Word), (Linux, Outlook), (Word, rmail)}。 按上述定义，该 FCC 实例的答案为“是”——例如选择 Linux ∈ A_1、emacs ∈ A_2、rmail ∈ A_3 就是完全兼容配置。

证明 Fully Compatible Configuration 是 NP 完全的。

答案： 属于 NP：证书是从每个选项集选出的一个元素，检验其间无 P 中的不兼容对即可。归约：从 3-SAT 归约（与 3-SAT 归约到独立集的做法类似）。每个子句对应一个选项集，含三个与子句文字对应的选项；若两个选项对应同一变量及其否定，则把它们标为不兼容。则可满足赋值等价于存在完全兼容配置。

习题 22

题目： 假设有人给你一个黑盒算法 A，它接受无向图 G = (V, E) 和数 k，行为如下：

- 若 G 不连通，直接返回“G 不连通”；
- 若 G 连通且含大小至少 k 的独立集，返回“yes”；
- 若 G 连通且不含大小至少 k 的独立集，返回“no”。

设 A 的运行时间是 G 与 k 的规模的多项式。说明如何通过调用 A，在多项式时间内求解 **Independent Set 问题**：给定任意无向图 G 和数 k，判定 G 是否包含大小至少 k 的独立集?

答案： 若 V = ∅ 或 k = 1 可直接回答，故设 V ≠ ∅ 且 k > 1。做法一：给 G 加一个新节点 v，*把它与 V 中所有节点相连得连通图 G*，调用 A(G, *k) 并返回其答案；k > 1 时 v* 不会出现在任何大小 ≥ k 的独立集中，故

G 与 G* 的答案相同。做法二：用 BFS 在 O(|E|) 时间求出各连通分量 C，对每个分量用 A(C, k′) 二分（或从 k 到 n 逐个）求出其最大独立集规模 k_C，各分量独立集之并即为全图最大独立集，只需比较 ∑_C k_C 与 k。两种做法都是多项式时间。

习题 23

题目： 给定有限二进制串集合 S = {s_1, …, s_k}，称串 u 是 **S 上的拼接**（concatenation over S），如果存在下标 i_1, …, i_t ∈ {1, …, k}，使 u = s_{i_1} s_{i_2} … s_{i_t}。

你的朋友正在考虑如下问题：给定两组有限二进制串 A = {a_1, …, a_m} 和 B = {b_1, …, b_n}，是否存在一个串 u，使 u 既是 A 上的拼接，又是 B 上的拼接？

朋友宣称：“至少这个问题在 NP 中，因为我只需展示这样一个串 u 就能证明答案为 yes。”你（当然，礼貌地）指出这一说明完全不充分：我们怎么知道最短的这样的串 u 的长度不会是输入规模的指数？若是那样它就不是多项式规模的证书。

然而，这个断言可以被转化为 NP 成员性的证明。具体地，证明下述命题：

若存在一个串 u 既是 A 上的拼接又是 B 上的拼接，则存在这样的串，其长度被 A ∪ B 中所有串长度之和的多项式所界。

答案： 不妨设 m ≤ n，令 L 为 A ∪ B 中最长串的长度，并取 u 为最短的可行串。u 的每个位置在 A 的表示中有一个“类型”（a_i, k，即它是 a_i 的第 k 个字符），在 B 的表示中也有一个类型（b_j, k）；类型对至多 m·n·L^2 种。若 |u| > m·n·L^2（≤ n^2L^2），由鸽巢原理存在位置 p < p′ 的类型对完全相同，则删去 p, p+1, …, p′ − 1 得到的更短串仍是 A 与 B 上的拼接，与 u 的最短性矛盾。故最短 u 的长度 ≤ m n L^2 ≤ n^2L^2，是输入规模的多项式，即存在多项式长度的证书。

习题 24

题目： 设 G = (V, E) 是二分图（bipartite graph），其节点被划分成 X 和 Y 两个集合，使每条边的一端在 X、另一端在 Y。定义 G 的 **(a, b)-骨架**（skeleton）为边集 E′ ⊆ E，使得**至多** a 个 X 中的节点与 E′ 中某条边关联，且**至少** b 个 Y 中的节点与 E′ 中某条边关联。

证明：给定二分图 G 以及数 a 和 b，判定 G 是否存在 (a, b)-骨架是 NP 完全的。

答案： 属于 NP：证书是 E′，可多项式时间统计 X、Y 中与 E′ 关联的节点数。归约：从集合覆盖（Set Cover）归约。X 中节点对应各集合 S_i，Y 中节点对应全集 U 的元素，集合节点与它包含的元素节点相连；取 a = k、b = n（骨架必须触及 Y 的全部节点）。则 G 存在 (a, b)-骨架当且仅当存在大小 k 的集合覆盖。

习题 25

题目： 对函数 g_1, …, g_ℓ，通过 max(g_1, …, g_ℓ) = max(g_1(x), …, g_ℓ(x)) 定义函数 max(g_1, …, g_ℓ)。

考虑如下问题。给定 n 个分段线性（piecewise linear）连续函数 f_1, …, f_n，它们定义在区间 [0, t] 上（t 为某个整数）；另给定整数 B。你要判定：是否存在 k 个函数 f_{i_1}, …, f_{i_k}，使得

$\int_0^t \max(f_{i_1}, \ldots, f_{i_k})\, dx \geq B$?

证明该问题是 NP 完全的。

答案： 属于 NP：若每个 f_i 的分段线性表示至多有 q 个折点，则 k 个函数的逐点最大值至多有 k^2q^2 个折点，可在多项式时间内逐段算出积分以检验证书。归约：从顶点覆盖归约。设 G 有 n 个节点、m 条边，令 t = 2m − 1，每个节点 v_i 对应一个定义在 [0, 2m − 1] 上的 f_i：若边 e_j 关联 i，则 f_i(x) = 1（x ∈ [2j, 2j+1]），否则为 0；在 x = 2j + 3/2 处取 1/2，其余区间用直线连接。取 B = m + (3/4)(m − 1)。则 k 个函数的最大值积分 ≥ B 当且仅当对应节点构成大小 k 的顶点覆盖。

习题 26

题目： 你和朋友在世界各地远足，积攒了一大堆纪念品。当时并没认真考虑哪些你留着、哪些朋友留着，而现在该把所有东西分掉了。

一种可行的做法如下。设有 n 个物品，标号为 1, 2, …, n，物品 i 有双方商定的价值 x_i。（例如可把它看作转售价值；双方对价值看法不一致的情形此处不予讨论。）一种合理的分法是寻找物品的一个**划分**（partition），把它分成两个集合，使两个集合中物品的总价值相等。

这提示我们求解如下 **Number Partitioning 问题**：给定正整数 $x_1, \ldots, x_n$，判定能否把这些数划分成两个集合 S_1 与 S_2，使二者之和相等：

$\sum_{x_i \in S_1} x_i = \sum_{x_i \in S_2} x_i$

证明 Number Partitioning 是 NP 完全的。

答案： 属于 NP：证书是子集 S ⊆ {1, …, n}，其中 $\sum_{i \in S} x_i = \frac{1}{2}\sum_{i=1}^{n} x_i$，多项式时间验证等式即可。归约：从子集和归约。给定 $w_1, \ldots, w_n$ 与目标 W，令 T = ∑w_i，新增 $w_{n+1} = W + 1$、$w_{n+2} = T + 1 - W$，总和为 2T + 2。则存在和为 W 的子集当且仅当这 n + 2 个数可等和划分（加 1 保证两个新数不在同一部分，从而另一部分其余数之和恰为 W）。

习题 27

题目： 考虑下述问题。给定正整数 $x_1, \ldots, x_n$ 以及数 k 和 B。你想知道能否把数 {x_i} **划分**成 k 个集合 $S_1, \ldots, S_k$，使得各集合元素之和的平方和至多为 B：

$\sum_{i=1}^{k} \left(\sum_{x_j \in S_i} x_j\right)^2 \le B$。

证明该问题是 NP 完全的。

答案： 属于 NP：证书是该划分，多项式时间计算各组和并验证平方和 ≤ B。归约：从 Number Partitioning 归约。给定 $x_1, \ldots, x_n$，令 S = ∑x_i，取 k = 2、$B = S^2/2$。等和划分对应平方和 $S^2/2$；反之若两组和为 S_1, S_2 且 $S_1^2 + S_2^2 = S^2/2$，由 $S_1 + S_2 = S$ 可推出唯一的解 $S_1 = S_2 = S/2$。

习题 28

题目： 下面是 Independent Set 问题的一个变体。给定图 G = (V, E) 及整数 k。称集合 I ⊆ V 是**强独立的**（strongly independent），如果对任意两个节点 v, u ∈ I，边 (v, u) 不属于 E，并且不存在从 u 到 v 的长度为 2 的路径，即不存在节点 w 使 (u, w) ∈ E 且 (w, v) ∈ E。**Strongly Independent Set 问题**是判定 G 是否有大小至少 k 的强独立集。

证明 Strongly Independent Set 问题是 NP 完全的。

答案： 属于 NP：证书是 k 个节点，检验两两之间距离至少为 3。归约：从 Independent Set 归约。构造 G′：把每条边 e = (u, v) 替换为长度 2 的路径（新增节点 w_e，加入边 (u, w_e)、(w_e, v)），并在所有新节点之间两两连边。则 G 有大小 k 的独立集当且仅当 G′ 有大小 k 的强独立集（新节点两两距离 ≤ 2，不可能入选；G′ 中的强独立集只能是原节点且它们在 G 中不相邻）。

习题 29

题目： 你在配置一个大型工作站网络，把它建模为无向图 G；G 的节点表示各工作站，边表示直接的通信链路。所有工作站都需要访问一个公共的**核心数据库**（core database），其中存有操作系统基本功能所需的数据。

你可以在每台工作站上复制这份数据库；这样从任意工作站查询都很快，但需要管理大量副本。另一种做法是只在某一台工作站上保存唯一一份数据库，其余工作站通过 G 发出数据请求；但离数据库所在位置很多跳的工作站会产生很大延迟。

因此你决定寻找折中方案：只维护少量副本，但放置得使任何工作站要么自身有数据库副本，要么与一台有副本的工作站直接相连。用图论的术语，这样的位置集合称为**支配集**（dominating set）。

于是把 **Dominating Set 问题**表述如下：给定网络 G 和数 k，是否存在一种方式，把 k 份数据库副本放在 k 个不同节点上，使每个节点要么自身有数据库副本，要么与一个有数据库副本的节点直接相连？

证明 Dominating Set 是 NP 完全的。

答案： 属于 NP：证书是这 k 台服务器，可多项式时间检验每个非服务器节点都与某台服务器相邻。归约：从顶点覆盖归约到支配集。对 G 的每条边加一个平行副本并把每条边细分：对边 (i, j) 新增节点 v_{ij}，加边 (i, v_{ij}) 与 (v_{ij}, j)；设 r 为 G 中孤立节点数，支配集参数取 k + r。则 G 有大小 ≤ k 的顶点覆盖当且仅当 G′ 有大小 ≤ k + r 的支配集（孤立节点必须自带副本，新节点可换回原端点）。

习题 30

题目： 思考传统“连续数学”问题时，有一点并不总是显而易见：离散的、组合的问题会悄悄渗入看似标准的微积分问题。

例如，考虑在区间 $x \in [0, 1]$ 上最小化一元函数 $f(x) = 3 + x - 3x^2$ 的传统问题。导数在 $x = 1/6$ 处为零，但这实际上是最大值而非最小值；要得到最小值，必须留意标准告诫——还要检查区间边界上的取值。（最小值实际上在边界 $x = 1$ 处取得。）

只有一个变量时检查边界不成问题；但假设现在要在 n 个变量 $x_1, x_2, \ldots, x_n$ 的单位立方体上最小化函数，其中每个 $x_i \in [0, 1]$。最小值可能在立方体内部取得，也可能在边界上取得；后一种情形相当令人却步，因为边界由 2^n 个“角”（每个 x_i 等于 0 或 1）以及各种维数的众多切片组成。微积分教材在描述如何应对这种复杂性、处理多变量最小化问题时，往往含糊得令人生疑。

这其实是有原因的：在 n 维单位立方体上最小化一个 n 元函数，与某个 NP 完全问题一样难。为使其具体化，考虑整系数 n 元多项式 $x_1, x_2, \ldots, x_n$ 的特殊情形。回顾术语：**单项式**（monomial）是实数系数 c 与每个变量 x_i 的某个非负整数次幂 a_i 的乘积，可写作 $c\, x_1^{a_1} x_2^{a_2} \cdots x_n^{a_n}$（例如 $2x_1^2x_2x_3^4$ 是单项式）。**多项式**（polynomial）则是有限个单项式之和（例如 $2x_1^2x_2x_3^4 + x_1x_3 - 6x_2^2x_3^2$ 是多项式）。

定义 **Multivariable Polynomial Minimization 问题**如下：给定一个 n 元整系数多项式和一个整数界 B，是否存在实数 $x_1, x_2, \ldots, x_n \in [0, 1]$ 的取值，使该多项式的值 ≤ B？

从本章中选一个已知为 NP 完全的问题 Y，并证明

Y ≤_p Multivariable Polynomial Minimization。

答案： 取 Y = 3-SAT。对每个布尔变量 x_i 引入实变量 y_i ∈ [0, 1]；每个子句对应三个文字之积，其中把 x_i 表示为 y_i、把 ¬x_i 表示为 (1 − y_i)（如子句 x_i ∨ ¬x_j ∨ x_k 对应 (y_i)(1 − y_j)(y_k)）。由分配律每个这样的积展开为至多 8 个单项式，令多项式为全部单项式之和，取界 B = 0。则多项式可取到 ≤ 0 的值当且仅当 3-SAT 实例可满足：每个子句积必须为 0，即该子句中至少有一个文字为真。

习题 31

题目： 给定无向图 G = (V, E)，反馈集（feedback set）是一个集合 X ⊆ V，使得 G − X 不含圈。无向反馈集问题（Undirected Feedback Set Problem）问：给定 G 和 k，G 是否含有大小至多为 k 的反馈集？证明无向反馈集问题是 NP 完全的。

答案： 属于 NP：给出 X 后用深度优先搜索判断 G − X 是否为森林。从顶点覆盖（vertex cover）归约：把每条边 (u, v) 替换为四条边 (u, x^1_uv)、(u, x^2_uv)、(v, x^1_uv)、(v, x^2_uv)，即把该边换成一个 4-圈 u-x^1-v-x^2-u，新顶点只与这两个端点相连。G 有大小 k 的顶点覆盖当且仅当 G′ 有大小 k 的反馈集：最小反馈集总可改取为不含新顶点（把 x 换成 u 或 v 不会增大规模），此时要打断每个 4-圈只能靠 u 或 v，故对应一个顶点覆盖。

习题 32

题目： 基因组作图涉及大量困难的计算问题。设 S = {s_1, s_2, …, s_n} 是 q 字母表上的短 DNA 串集合，每个串长度为 2ℓ（ℓ ≥ 1）；另有一个同字母表上的串库 T = {t_1, t_2, …, t_m}，其中每个串长度也是 2ℓ。若 t_k 的前 ℓ 个符号等于 s_a 的后 ℓ 个符号，且 t_k 的后 ℓ 个符号等于 s_b 的前 ℓ 个符号，则称 t_k 佐证（corroborate）有序对 (s_a, s_b)。若能把 S 中的串排成一个顺序 s_i_1, s_i_2, …, s_i_n（其中 i_1, i_2, …, i_n 是 {1, 2, …, n} 的一个排列），使得对每个 j = 1, 2, …, n − 1 都存在 T 中的串 t_k 佐证对 (s_i_j, s_i_{j+1})（同一个 t_k 可以用于多个相邻对），则称 S 相对于 T 有完美装配（perfect assembly）。完美装配问题（Perfect Assembly Problem）：给定 S 和 T，S 是否有相对于 T 的完美装配？证明完美装配问题是 NP 完全的。

答案： 属于 NP：以给出顺序的串序列为证书，逐对检查即可。从哈密顿路径归约：取 ℓ = 1、字母表为 {v_in, v_out : v ∈ V}，令 S = {v_in v_out : v ∈ V}，T = {v_out w_in : (v, w) ∈ E}。完美装配对应 G 中顶点的一个排列，其中每个相邻对都是边；反之哈密顿路径按经过顺序给出完美装配。

习题 33

题目： 在物物交换经济中，人们直接交换物品与服务，不以货币为中介。设有 n 个人 p_1, …, p_n 和 m 件互不相同的物品 a_1, …, a_m，每件物品归某个人所有。每个人 p_i 有一个估值函数 v_i：v_i(a_j) 是非负整数，表示物品 a_j 对 p_i 的价值（每个人都对每件物品赋值，包括自己并不拥有的物品，不同人对同一物品的估值可以差别很大）。若存在两个人 p_i、p_j，以及分别由 p_i、p_j 拥有的子集 A_i、A_j，使得双方都更想要对方手中自己没有的那部分物品，则称存在一次二人交易；确切地说，需要同时满足： - p_i 对 A_j 中物品的总估值超过其对 A_i 中物品的总估值； - p_j 对 A_i 中物品的总估值超过其对 A_j 中物品的总估值。 （A_i 不必是 p_i 的全部所有物，A_j 亦然，只要满足上述条件即可。）物品估值均为自然数。证明：给定上述物物交换经济的数据，判定是否存在二人交易是 NP 完全的。

答案： 属于 NP：以一对子集 A_i、A_j 为证书，把两边估值求和比较即可。从子集和（Subset Sum）归约：取 n + 1 件物品，p_i 拥有 a_1, ..., a_n，p_j 只拥有 a_{n+1}；对 k = 1, ..., n 令 v_i(a_k) = v_j(a_k) = w_k，并令 v_i(a_{n+1}) = W + 1、v_j(a_{n+1}) = W − 1。因 p_j 只有一件物品，必有 A_j = {a_{n+1}}、A_i ⊆ {a_1, ..., a_n}，两个不等式等价于 W − 1 < ∑_{a∈A_i} v(a) < W + 1，即该子集和恰为 W。

习题 34

题目： 考虑如下行为扩散模型：有向图 G = (V, E) 中节点表示人，若 v 能影响 w 则有边 (v, w)。每个人 w 有一个阈值 θ(w) ∈ [0, 1]：当指向 w 的节点中已成为采纳者（adopter）的比例至少为 θ(w) 时，w 也成为采纳者。阈值越低越容易被说服；θ(w) = 0 的节点无需他人影响立即采纳；无入边的节点当且仅当 θ(w) = 0 时成为采纳者。给定各节点阈值后可如下模拟扩散：初始令所有 θ(w) = 0 的节点为采纳者，其余为非采纳者；然后反复地，对所有非采纳者 w 同时判断，若指向 w 的节点中至少 θ(w) 比例已是采纳者，则 w 变为采纳者；直到采纳者集合不再变化，输出最终采纳者集合。现在给定节点集 S ⊆ V，把 S 中每个节点的阈值重置为 0（即已说服他们试用产品），运行上述过程，记最终采纳者集合的大小为 f(S)，它刻画了以 S 为种子的影响力。影响最大化问题（Influence Maximization Problem）：给定有向图 G、各节点阈值以及参数 k 和 b，是否存在至多 k 个节点的集合 S 使 f(S) ≥ b？证明影响最大化问题是 NP 完全的。

答案： 属于 NP：给定大小不超过 k 的 S，按定义模拟（至多 n 轮，每轮多项式时间）即得 f(S) 并比较 b。从顶点覆盖归约：为每个顶点 v 和每条边 e 各设一个节点，若 v 是 e 的端点则连边 (v, e)，所有阈值取 1/2。边节点恰有 2 个邻居，故一个端点被选中即可被感染；顶点覆盖 S（|S| = k）会使最终采纳者数为 k + m。反之若 k 个种子能影响 k + m 个节点，可把种子中的边节点换成其某个端点而不减少影响，所得 k 个顶点必覆盖所有边。

习题 35

题目： 某游戏中有 n 种不同型号的 droid 和 k 个不同行星。每个行星 p 具有如下属性：型号 j 有 s(j, p) ≥ 0 个可供出售，固定售价为 x(j, p) ≥ 0；同时对型号 j 有需求 d(j, p) ≥ 0，固定收购价为 y(j, p) ≥ 0（同一行星对同一型号不会同时有正供给和正需求，即对每个 j，s(j, p) 与 d(j, p) 至少一个为 0），这里 j = 1, 2, ..., n。玩家从行星 s 出发，携带 z 单位钱，最终必须到达行星 t；行星之间的合法路线由有向无环图 G 上的 s-t 路径给出（取无环是为防止游戏无限延长）。沿 G 中一条 s-t 路径 P，玩家可进行如下交易：每当到达 P 上的行星 p，可以按单价 x(j, p) 买入至多 s(j, p) 个型号 j 的 droid（只要手头钱够），和/或按单价 y(j, p) 卖出至多 d(j, p) 个型号 j 的 droid（j = 1, 2, ..., n）。玩家的最终得分是到达行星 t 时手中的钱数。高分问题（High-Score-on-Droid-Trader!，简称 HSoDT!）：给定上述实例（有向无环图 G 及全部参数）和目标界 B，是否存在 G 中的路径 P 及沿途的交易序列，使最终得分至少为 B？证明 HSoDT! 是 NP 完全的。

答案： 属于 NP：以路径 P 及交易序列为证书，沿路径模拟并检查是否达到 B。从 3-SAT 归约：构造 n + k 层的分层图，前 n 层每层两个节点，分别出售代表 x_i 与 $\bar{x}$_i 的 droid，供给量等于该文字在子句中出现的总次数，售价为 0；后 k 层每层三个节点，对应该子句的三个文字，各需 1 个该型号 droid，收购价为 1；相邻层之间全连接，源接第 1 层、第 n + k 层接汇。路径每层恰过一个节点，故经过 x_i 节点即令 x_i 为真；每个子句层卖出 1 个 droid 得 1 元，总分达界 k 当且仅当每个子句都被满足的文字"供货"，即公式可满足。

习题 36

题目： 某餐馆有 n 种可采购的原料 l_1, l_2, ..., l_n（例如成捆的蒲公英叶、罐装米醋等）。原料 l_j 必须按每包 s(j) 克整包购买，每包价格 c(j) 美元，且自购买之日起 t(j) 天内安全可用。今后 k 天中每天要做一道不同的每日特餐（共 k 道，顺序自定）。第 i 天的特餐使用原料子集 S_i ⊆ {l_1, l_2, ..., l_n}，具体需要 a(i, j) 克原料 l_j。还有一条约束：为保持食材最新鲜，每道特餐所用原料 S_i 被分成两部分——必须在做该特餐当天购买的部分，以及只要仍在保质期内就可任意一天购买的部分。若某天买了某原料的一包而当天用不完，只要紧接着安排另一道用到该原料、且不要求当天新鲜的菜，就能省下再次购买的开销（但把某些菜排得太近又可能使其他菜的安排变难）。每日特餐排程问题（Daily Special Scheduling Problem）：给定上述原料与菜谱数据以及预算 x，是否存在 k 道特餐的日程安排，使 k 天中采购原料的总花费至多 x？证明每日特餐排程问题是 NP 完全的。

答案： 属于 NP：以特餐的一个顺序为证书，按最省方式计算总花费并与 x 比较。从哈密顿路径归约：G 的每个顶点对应一道特餐，每条边对应一种原料；每道特餐需要其对应顶点所有关联边（入边和出边）各 1 单位，原料按 2 单位一份、每份 1 美元出售，两天后失效。不共享时每次使用需买 1 份（1 美元），共 2m 美元；相邻两道特餐共享的那 n − 1 种原料只需买一次，故总花费 2m − n + 1。*存在花费 2m − n + 1 的排程当且仅当 G 有哈密顿路径。*

习题 37

题目： 把星系看成无向图 G = (V, E)，节点是星系，边 (u, v) 表示可从 u 直达 v。死星是节点 s，隐藏的叛军基地是节点 t。某些边代表较远的距离，每条边 e 有整数长度 $\ell_e \geq 0$；某些边是被帝国舰队重点巡逻的路线，每条边 e 有整数风险 r_e ≥ 0，表示通过时遭遇太空战所受损失的期望值。对任意 s-t 路径 P，定义其总长度为各边长度之和，总风险为各边风险之和。银河最短路径问题（Galactic Shortest Path Problem）：给定上述图、s、t 以及整数界 L 和 R，是否存在一条 s-t 路径，其总长度至多 L 且总风险至多 R？证明银河最短路径问题是 NP 完全的。

答案： 属于 NP：以路径为证书，累加长度和风险与 L、R 比较即可。从子集和归约：取链 v_0-v_1-…-v_n，每对相邻顶点间放两条平行边 e_i（长度 w_i、风险 0）和 e′*i（长度 0、风险 w_i）；令 $W_0 = \sum w_i$，取 L = W、R = W_0 − W。任一路径在每对平行边中恰选一条，对应一个子集 S，长度 ≤ W 且风险 $W_0 - \sum_{i \in S} w_i \leq W_0 - W$，两者合并即 $\sum_{i \in S} w_i = W$。*

习题 38

题目： 考虑 Steiner 树问题的如下版本，称为图形 Steiner 树（Graphical Steiner Tree）：给定无向图 G = (V, E)、顶点集 X ⊆ V 以及数 k，判定是否存在至多 k 条边的集合 F ⊆ E，使得在 (V, F) 中 X 的所有顶点属于同一个连通分量。证明图形 Steiner 树问题是 NP 完全的。

答案： 属于 NP：以边集 F 为证书，检查 |F| ≤ k 且 X 中顶点连通即可。从顶点覆盖归约：把每条边 e = (u, v) 细分，插入新顶点 w_e（只连 u、v），记新顶点集为 W；再加一个与 V 中所有原顶点相连的节点 r，令 X = {r} ∪ W、k′ = k + m。若 G 有大小 k 的顶点覆盖 S，取 r 到 S 各点的边以及每个 w_e 到其覆盖端点的边，共 k + m 条即可；反之任一 ≤ k + m 条边的 Steiner 树可整理成每个 w_e 只用一条边，从而必须用 m 条边连接 W，剩下至多 k 条从 r 连向 V，这些端点构成顶点覆盖。

习题 39

题目： 有向不相交路径问题（Directed Disjoint Paths Problem）定义如下：给定有向图 G 和 k 对节点 (s_1, t_1), (s_2, t_2), …, (s_k, t_k)，判定是否存在节点不相交的路径 P_1, P_2, …, P_k，其中 P_i 从 s_i 到 t_i。证明有向不相交路径问题是 NP 完全的。

答案： 属于 NP：以各条路径为证书，检查它们连接对应端点且两两节点不相交即可。从 3-SAT 归约：对每个变量 x_i 造两条各有 k 个节点的 s_i-t_i 路径 P_i 和 P′*i（分别代表 x_i 取真、取假，选 P_i 相当于 x̄_i 的副本未被使用），两路径的第 j 个节点对应第 j 个子句；对每个子句 C_j 增加一对源汇 S_j、T_j，把 S_j 连到该子句三个文字所在路径上的节点 v*{j1}, v_{j2}, v_{j3}，再把这些节点连到 T_j。可满足赋值给出互不相交的路径组，反之不相交路径组中每条 S_j-T_j 路径必须借用某个为真的文字节点，故公式可满足。

习题 40

题目： 考虑网络广播方案设计中出现的问题。给定有向图 G = (V, E)、指定节点 r ∈ V 以及"目标节点"集合 T ⊆ V − {r}。每个节点 v 有一个正整数切换时间（switching time）s_v。时刻 0 节点 r 产生一条消息，希望 T 中每个节点都能收到。为此要找一套方案：r 先告诉某些邻居（按顺序），这些邻居再告诉它们的邻居，如此直至 T 中所有节点都收到。确切地说，广播方案定义如下：节点 r 可在时刻 0 向一个邻居发送消息副本，该邻居在时刻 1 收到；一般地，时刻 t ≥ 0 任何已收到消息的节点 v 可向它的一个邻居发送副本，条件是它在时刻 t − s_v + 1, t − s_v + 2, …, t − 1 中均未发送过副本（即两次发送之间必须停 s_v − 1 步；若 s_v = 1 则无限制）。广播方案的完成时间是 T 中所有节点都已收到消息的最早时刻 t。广播时间问题（Broadcast Time Problem）：给定上述输入和界 b，是否存在完成时间至多 b 的广播方案？证明广播时间问题是 NP 完全的。

答案： 属于 NP：以发送方案为证书，检查切换时间约束并核验 T 中节点都在期限内收到消息即可。从三维匹配归约：设 X、Y、Z 各含 n 个元素、三元组集为 T；建根 r（s_r = 4），每个三元组 t 一个节点 v_t（s = 1），每个元素 a ∈ X ∪ Y ∪ Z 一个节点 w_a（s = 1），边为 (r, v_t) 及 a ∈ t 时的 (v_t, w_a)。若 n 个三元组覆盖全部元素，则 r 在时刻 0, 4, …, 4n − 4 依次向各 v_t 发送，v_t 在 4j + 1, 4j + 2, 4j + 3 转发给三个元素，完成时间 4n − 1；反之完成时间 ≤ 4n − 1 迫使 r 至少发送 n 次且每个 w_a 均收到消息，这些三元组即覆盖全部元素。

习题 41

题目： 给定有向图 G，圈覆盖（cycle cover）是一组节点不相交的圈，使得 G 的每个节点都属于其中某个圈。圈覆盖问题问：给定有向图，它是否有圈覆盖？ (a) 证明圈覆盖问题可在多项式时间内求解。（提示：使用二分图匹配。） (b) 若要求每个圈的边数至多为 3，证明判定图 G 是否有这样的圈覆盖是 NP 完全的。

答案： (a) 把每个顶点 v 拆成 v_in、v_out 分别置于二分图两侧，对每条有向边 (v, w) 连一条边 (v_out, w_in)，得到二分图 G′。G 的圈覆盖与 G′ 的完美匹配一一对应（每个顶点恰有一条入边和一条出边），故用二分图最大匹配在 O(m√n) 时间内判定。 (b) 属于 NP：以各圈为证书检验即可。从三维匹配归约：对每个三元组 t 造一个由三个节点组成的三角形（对应 t 的三个元素），每个三角形就是构造中唯一可能的三角形圈；对每个元素 s，把代表它在各三元组中出现的副本集 A_s 与 |A_s| − 1 个辅助节点 B_s 之间连双向完全二分边，这些节点只能用 2-圈覆盖。完美三匹配存在当且仅当存在每圈至多 3 条边的圈覆盖：匹配的三元组取三角形，其余节点用 2-圈覆盖。

习题 42

题目： 某公司设计一个 n 节点通信网络，模型为有向图 G = (V, E)。出于容错考虑，希望把 G 划分成尽可能多的虚拟"域"：G 中的域（domain）是节点集 X（|X| ≥ 2），使得对 X 中每一对节点 u, v，都存在完全含于 X 的 u 到 v 以及 v 到 u 的有向路径。证明下述域分解问题（Domain Decomposition Problem）是 NP 完全的：给定有向图 G = (V, E) 和数 k，能否把 V 划分成至少 k 个集合，使每个集合都是域?

答案： 属于 NP：以划分为证书，对每个集合检验其内部任意两点互相可达（强连通）即可。从三维匹配归约：对每个元素 x_i ∈ X、y_j ∈ Y、z_k ∈ Z 造节点 x′_i、y′_j、z′_k；对每个三元组 A_m 造三个节点 v^m_x、v^m_y、v^m_z 并连成有向三角形，元素节点与其所属三元组的对应节点之间连双向边。若 C 有完美三匹配，则匹配的 n 个三元组各贡献 3 个二元域，未匹配的 t − n 个三元组各取一个三元三角形域，共 2n + t 个域；反之任何 2n + t 个域的分解中每个元素节点必须与某个三角形节点组成二元域，这些三角形对应的三元组两两不交，即构成完美三匹配。